

# THE HAVEN PROTOCOL

## LEDGER TOKENOMICS & POINT-OF-SALE (PoS) INTEGRATION GUIDE

### Production Protocol & Settlement Architecture Specification

**Document Classification:** Core Protocol Engineering Specification

**Document Number:** HAVEN-DOC-05

**Version:** 2.0

**Status:** Production Architecture Specification

**Network:** Polygon EVM

**Protocol Model:** Stateless Invisible Coordination Protocol

**Primary Enforcement Layer:** Cryptographically Verifiable Settlement & Edge Authorization

**Payment Enforcement Model:** Zero-Trust Payment Routing Enclosure Engine

**Settlement Model:** Hybrid Off-Chain Obligation & On-Chain Batch Settlement

**Verification Model:** Event-Driven Dual-Transport Verification with Lease-Bound Authorization

**Data Protection Model:** Zero-Trust PHI Isolation with Off-Chain Clinical Data Storage

**Security Posture:** Fail-Closed / Zero-Trust / Atomic Settlement / Immutable Routing

**CONFIDENTIAL INSTITUTIONAL Technical Document. This architecture suite is published openly on GitHub and is licensed strictly under the terms of the GNU Affero General Public License (AGPL-3.0). Unauthorized derivative commercial execution or private re-licensing without an enterprise waiver from the author is strictly prohibited.**

# **SYSTEM PRINCIPLE**

## **NO VALID SETTLEMENT. NO NETWORK PARTICIPATION.**

The Haven Protocol establishes a cryptographically enforceable relationship between economic participation and interoperability participation.

Every billable Haven system event creates a protocol settlement obligation. The corresponding infrastructure pass-through fee is therefore not treated as an optional application charge, client-configurable software parameter, or advisory commercial preference.

It is an infrastructure condition of network participation.

The settlement obligation may be accumulated and cryptographically committed into a bounded settlement batch before being settled on-chain. Batching changes the execution mechanism; it does not change the underlying economic obligation of the participating network.

The Point-of-Sale integration layer is consequently designed so that payment-routing and settlement state cannot be trusted merely because a client application reports compliance.

The authoritative economic state is established through independently verifiable settlement evidence.

The authoritative routing mechanism remains governed by immutable protocol rules.

The authoritative synchronization state is independently established by the Haven edge enforcement layer through validated blockchain evidence.

Real-time event delivery may be provided through authenticated WebSocket or gRPC transports, but event-stream presence alone does not constitute settlement truth.

A valid synchronization state is therefore bounded by an explicit authorization lease. When the lease expires, verification becomes stale and the network enters an unauthorized state until fresh independent verification is completed.

Temporary RPC failure, event-stream interruption, stale state, incomplete settlement evidence, inconsistent verification cursors, malformed blockchain responses, or unresolved state must never be interpreted as affirmative authorization.

The safe protocol default is:

**UNKNOWN → UNAUTHORIZED**

**STALE → UNAUTHORIZED**

## **UNVERIFIED → UNAUTHORIZED**

The client application may initiate a transaction, create a settlement obligation, or request Haven-governed coordination, but it does not determine whether the Haven Protocol considers the resulting network state valid.

The protocol therefore separates:

### **CLIENT ACTION**

from

### **PROTOCOL AUTHORITY**

and:

### **EVENT GENERATION**

from

### **SETTLEMENT FINALITY**

and:

### **EVENT DELIVERY**

from

### **VERIFICATION TRUTH**

and:

### **TEMPORARY CONNECTIVITY**

from

### **AUTHORIZATION VALIDITY**

The governing rule is:

**Economic participation must remain cryptographically verifiable, independently observable, time-bounded, and impossible for the participating client to unilaterally redefine.**

# CORE ECONOMIC PARAMETER

## Haven Protocol Micro-Transaction Tax

### Protocol Event Fee:

**\$0.05 / ₦100 per system event block**

The Haven Protocol Event Fee remains the defined infrastructure charge associated with each billable Haven-governed system event.

The applicable fee is structured as an **infrastructure pass-through operational expense** and is incorporated directly into the consumer-facing checkout transaction.

The checkout system therefore presents the applicable fee as part of the transaction economics rather than requiring a separate post-transaction collection mechanism.

## SETTLEMENT EXECUTION MODEL

The existence of a protocol event creates a **settlement obligation**.

It does **not** require an individual blockchain transaction for every event.

The protocol operates through the following settlement sequence:

### Protocol Event

- **Fee Obligation Created**
- **Off-Chain Settlement Ledger**
- **Micro-Batch Accumulation**
- **Cryptographic Batch Commitment**
- **Polygon Settlement Transaction**
- **Independent Settlement Verification**

This separation exists to prevent blockchain transaction overhead from becoming a structural dependency of high-frequency healthcare operations.

The off-chain settlement ledger is authoritative for **event-level accounting and reconciliation state**.

The Polygon settlement layer is authoritative for **cryptographic settlement execution and designated fund routing**.

A settlement batch **MUST** contain, at minimum:

- unique `batchId`;
- monotonically increasing `batchSequence`;
- event count;
- aggregate settlement amount;
- settlement asset;
- batch commitment;
- creation timestamp;
- sealing timestamp;
- settlement status;
- settlement attempt identifier.

The batch commitment **MUST** cryptographically bind the underlying settlement obligations without placing clinical information or patient-identifiable information on-chain.

## **GAS PROTECTION**

Settlement execution **MUST** operate under a defined gas-protection policy.

The settlement engine **MUST** evaluate:

- current gas estimate;
- configured gas ceiling;
- dynamic volatility buffer;
- maximum acceptable execution cost;
- batch economics;
- retry eligibility;
- settlement urgency.

Where network conditions exceed the configured gas policy, settlement **MAY** be temporarily deferred.

**Gas deferral **MUST NOT** be interpreted as authorization to bypass settlement.**

The obligation remains outstanding until successfully settled, reconciled, cancelled under an explicitly permitted state transition, or placed into controlled recovery.

## **CLINICAL DATA ISOLATION**

The Haven coordination layer does not acquire ownership of the underlying clinical transaction, patient funds, merchant inventory, healthcare asset, or client database.

Its economic role remains restricted to enforcing the protocol-defined infrastructure charge associated with the Haven-coordinated event.

**PHI and clinical payloads remain 100% off-chain.**

Polygon EVM settlement records **MUST** contain only the minimum non-reversible cryptographic correlation material required for settlement verification and reconciliation.

No patient name, medical record, diagnosis, clinical narrative, prescription, laboratory result, encounter payload, or other PHI **MUST** be written to blockchain state or EVM event logs.

The blockchain therefore functions as a **settlement and verification substrate**, not as a clinical data store.

## **ECONOMIC INVARIANT**

A Haven-governed event creates an enforceable economic obligation.

The obligation **MAY** be settled individually or through a bounded micro-batch mechanism.

In either case:

**NO VALID SETTLEMENT → NO VALID NETWORK PARTICIPATION.**

# ZERO-TRUST PAYMENT ROUTING ENCLOSURE

The payment and settlement architecture is governed by the following foundational constraints:

## 1. CLIENT SOFTWARE IS NOT AUTHORITATIVE

Checkout configuration variables, frontend flags, environment variables, API responses, JavaScript controls, database fields, SDK parameters, client-side verification values, local payment state, and application-reported compliance status cannot establish protocol settlement compliance.

Client software may **initiate** an event or settlement request.

It cannot declare that settlement has occurred.

The client cannot extend its own authorization lease, manufacture settlement confirmation, rewrite settlement state, or convert an unknown synchronization state into an authorized state.

## 2. THE EVM IS AUTHORITATIVE FOR ON-CHAIN SETTLEMENT ROUTING

The Polygon smart-contract layer contains the authoritative settlement-routing mechanism and designated destination logic.

The production splitter enforces the protocol-defined allocation:

**95% → Designated Settlement Destination**

**5% → Haven Protocol Treasury**

The configured allocation is immutable at runtime unless a separately governed contract migration mechanism explicitly replaces the deployment.

The contract **MUST** reject invalid settlement parameters, invalid destinations, unauthorized callers, duplicate batch settlement, and zero-value settlement execution.

The EVM therefore establishes authoritative **on-chain settlement truth**.

## 3. SETTLEMENT OBLIGATION AND BLOCKCHAIN SETTLEMENT ARE DISTINCT STATES

A Haven event first produces an off-chain settlement obligation.

That obligation **MAY** remain temporarily pending while the settlement engine accumulates a bounded batch.

Accordingly:

**EVENT  $\neq$  BATCH**

**BATCH  $\neq$  SUBMISSION**

**SUBMISSION  $\neq$  INCLUSION**

**INCLUSION  $\neq$  FINAL VERIFICATION**

**VERIFICATION  $\neq$  AUTHORIZATION UNLESS ALL CONTROL CONDITIONS ARE VALID**

A batch becomes authoritative only after the settlement transaction, contract identity, aggregate amount, batch commitment, and required confirmation conditions have been independently verified.

#### **4. SYNCHRONIZATION IS INDEPENDENTLY VERIFIED**

Edge enforcement **MUST NOT** rely on a single transport or locally supplied compliance assertion.

The verification system uses:

**PRIMARY TRANSPORT:**

WebSocket / gRPC event streaming for low-latency event propagation.

**FALLBACK TRANSPORT:**

Optimized JSON-RPC polling and historical backfill.

The event stream provides **transport and observation**.

It does not independently establish settlement truth.

Authoritative settlement verification **MUST** be capable of being reconstructed through JSON-RPC and independently verified against the configured Polygon contract identity.

The verification identity **MUST** bind, at minimum:

- chain ID;
- contract address;
- expected runtime bytecode hash;
- protocol interface;
- immutable settlement destinations;
- 95/5 split configuration;
- protocol version.

## 5. SYNCHRONIZATION IS LEASE-BOUND

Successful verification produces a bounded authorization lease.

The lease MUST contain:

- leaseId;
- network ID;
- verified block number;
- verified block hash;
- verified state;
- issued timestamp;
- expiry timestamp;
- verification source;
- stream sequence;
- verification cursor.

### 5.1 Lease Quantization and Temporal Handshakes

To prevent transient network jitter or minor RPC scheduling delays from triggering cascading false-positive circuit breaks, the operational duration of an issued synchronization lease SHALL be strictly quantized.

• **Absolute Lease Validity Window:** Every successfully generated authorization lease SHALL carry a hard runtime expiration parameter locked to exactly 180 seconds ( $T_{\text{lease}} = 180\text{s}$ ).

• **Polling Ratio Alignment:** The lease duration mathematically enforces a 3:1 safety ratio against Document 02's nominal 60-second polling worker execution interval ( $T_{\text{lease}} = 3 \times T_{\text{poll}}$ ).

• **Automatic Revocation:** If the local edge adapter fails to achieve a fresh, verified, positive contract state assertion from the public network substrate within this 180-second validity window, the active lease SHALL instantly expire by operation of law, forcing an immediate transition to the CIRCUIT\_OPEN state. A valid RPC response alone does not constitute authorization.

Authorization requires:

**STREAM HEALTHY**

**+ LEASE VALID**

**+ STATE VERIFIED**

**+ CURSOR CONSISTENT**

**+ CONTRACT IDENTITY VALID**

**+ SETTLEMENT STATE VALID**

Only when all required conditions evaluate positively may Haven-governed coordination continue.

## 6. FAILED OR UNKNOWN VERIFICATION CAUSES ENFORCEMENT

Where the required synchronization or settlement condition is not satisfied, the edge enforcement layer activates the Haven circuit breaker.

The following states **MUST** be treated as unauthorized:

- UNKNOWN;
- STALE;
- UNVERIFIED;
- EXPIRED;
- CORRUPTED;
- INCONSISTENT;
- REORGED\_PENDING\_REVALIDATION;
- CURSOR\_GAP;
- STREAM/RPC\_DISAGREEMENT.

Temporary RPC failure, WebSocket failure, gRPC failure, event-stream interruption, stale state, incomplete settlement evidence, sequence gaps, malformed responses, unresolved contract state, or lease expiration **MUST NOT** silently preserve authorization.

**UNKNOWN → UNAUTHORIZED**

**STALE → UNAUTHORIZED**

**UNVERIFIED → UNAUTHORIZED**

**EXPIRED → UNAUTHORIZED**

Recovery requires successful re-verification.

## 7. FAIL-CLOSED IS THE DEFAULT

The Haven edge layer **MUST** fail closed whenever authoritative settlement or synchronization state cannot be established with sufficient cryptographic and temporal confidence.

The system **MUST NOT** infer validity from:

- connectivity;
- cached application state;
- previous authorization;
- client assertions;
- successful API responses;
- healthy UI state;
- an apparently healthy event stream;
- an unexpired local cache.

A valid authorization state must be **earned through verification**, not presumed through continuity.

## **GOVERNING RULE**

**Economic participation must remain cryptographically verifiable, independently observable, time-bounded, and impossible for the participating client to unilaterally redefine.**

Therefore:

**NO VALID SETTLEMENT.**

**NO VALID SYNCHRONIZATION.**

**NO VALID LEASE.**

**NO NETWORK PARTICIPATION.**

## **DOCUMENT PURPOSE**

This specification defines the production architecture through which Haven converts its protocol economics into an enforceable, cryptographically verifiable coordination mechanism.

It specifies the relationship between:

- consumer checkout;
- merchant or institutional billing;
- Haven event generation;
- protocol fee calculation;
- settlement obligation creation;
- off-chain settlement accounting;
- micro-batch accumulation;
- batch sealing and cryptographic commitment;
- Polygon settlement submission;
- immutable splitter routing;
- treasury allocation;
- secure vault allocation;
- settlement verification;
- synchronization state;
- event-stream observation;
- edge adapter verification;
- synchronization lease issuance;
- circuit-breaker enforcement;
- JSON Schema validation;
- EVM smart-contract execution;
- transaction observability;
- batch reconciliation;
- stream recovery;

- reorganization handling;
- gas-protection controls;
- security controls; and
- operational recovery.

The objective is not merely to describe how a payment can be collected.

The objective is to define how a protocol economic obligation becomes **machine-enforced infrastructure state** and how that state becomes a prerequisite for continued Haven network coordination.

## **ARCHITECTURAL BOUNDARY**

Haven deliberately separates four concerns that must not be conflated:

### **1. EVENT GENERATION**

The participating healthcare system generates a Haven-governed system event.

The event establishes the economic obligation.

It does not establish settlement finality.

### **2. SETTLEMENT ACCOUNTING**

The Haven settlement layer records the resulting fee obligation in an off-chain settlement ledger.

This layer maintains event-level accounting, correlation, batching state, retry state, and reconciliation state.

It **MUST NOT** contain clinical payloads beyond the minimum metadata required for protocol accounting.

### **3. ON-CHAIN SETTLEMENT**

Eligible settlement obligations are accumulated into bounded settlement batches.

A sealed batch is cryptographically committed and submitted to the Polygon EVM settlement contract.

The contract is authoritative for:

- settlement execution;
- designated destination routing;
- configured allocation;
- replay protection;
- batch settlement finality; and
- on-chain settlement evidence.

### **4. EDGE AUTHORIZATION**

The Haven edge enforcement layer independently verifies settlement and synchronization state.

Only verified state may produce an authorization lease.

Only a valid authorization lease may permit continued Haven-governed cross-network coordination.

## PROTOCOL SEQUENCE

The production architecture therefore operates according to:

### Consumer Checkout

- **Haven Event Generation**
- **Fee Calculation**
- **Settlement Obligation**
- **Off-Chain Settlement Ledger**
- **Micro-Batch Accumulation**
- **Batch Commitment**
- **Polygon Settlement**
- **Independent Verification**
- **Synchronization Lease**
- **Authorized Cross-Network Translation**

This ordering preserves the original enforcement principle while removing unnecessary per-event blockchain settlement friction.

## ENFORCEMENT PRINCIPLE

The payment condition **MUST** remain an upstream authorization condition.

However, **settlement batching MUST NOT weaken enforcement.**

An event may be temporarily represented by an outstanding settlement obligation while a bounded batch is being accumulated.

The system **MUST** therefore distinguish:

### OBLIGATION EXISTENCE

from

### ON-CHAIN SETTLEMENT FINALITY

and:

### SETTLEMENT FINALITY

from

### NETWORK AUTHORIZATION

No layer may silently promote one state into another.

## **DATA PROTECTION BOUNDARY**

Clinical information and PHI remain outside the settlement substrate.

Haven **MUST NOT** use the Polygon blockchain as a clinical data repository.

On-chain state and EVM logs are restricted to non-reversible cryptographic correlation and settlement metadata required for independent verification.

The protocol's economic infrastructure therefore remains decoupled from the underlying clinical record.

## **PRODUCTION OBJECTIVE**

The resulting architecture **MUST** provide:

### **LOW-LATENCY EVENT PROCESSING**

without requiring immediate blockchain execution for every event;

### **CRYPTOGRAPHIC SETTLEMENT ASSURANCE**

without placing PHI on-chain;

### **INDEPENDENT VERIFICATION**

without trusting participating applications;

### **FAIL-CLOSED AUTHORIZATION**

when authoritative state cannot be established;

### **GAS-AWARE EXECUTION**

without allowing gas volatility to invalidate economic accounting;

and

### **DETERMINISTIC RECOVERY**

when transport, RPC, settlement, or network conditions fail.

The architecture is therefore designed not simply to process payments, but to make **economic participation, settlement integrity, synchronization integrity, and network authorization part of one enforceable control system.**

# 1. SYSTEM ARCHITECTURE & ENFORCEMENT MODEL

## 1.1 Architectural Objective

The Haven Protocol PoS architecture establishes a closed enforcement loop between the consumer transaction, the Haven event block, the off-chain settlement obligation, the cryptographic settlement batch, the Polygon settlement rail, and the edge interoperability layer.

The production architecture MUST preserve the distinction between an economic obligation being created and that obligation becoming independently verifiable on-chain settlement.

The canonical execution sequence is:

### **Consumer Checkout**

- **Haven Event Generation**
- **Fee Obligation Creation**
- **Off-Chain Settlement Ledger**
- **Bounded Micro-Batch Accumulation**
- **Cryptographic Batch Commitment**
- **Gas-Aware Polygon Settlement**
- **Event-Driven Settlement Verification**
- **RPC Cross-Check / Historical Recovery**
- **Authorization Lease Issuance**
- **Authorized Cross-Network Coordination**

The architecture is deliberately ordered so that economic participation becomes an upstream authorization condition for continued Haven-governed interoperability.

A Haven event therefore creates an enforceable settlement obligation without requiring a separate blockchain transaction for every individual event.

This distinction is essential for high-frequency healthcare environments. Blockchain settlement overhead MUST NOT become a structural dependency of the consumer checkout path.

The PoS system therefore separates:

### **EVENT CREATION**

from

### **SETTLEMENT EXECUTION**

and:

### **SETTLEMENT OBLIGATION**

from

### **ON-CHAIN SETTLEMENT**

and:

**SETTLEMENT OBSERVATION**  
from  
**SETTLEMENT VERIFICATION**

and:

**VERIFICATION**  
from  
**AUTHORIZATION**

The resulting architecture is:

**Checkout → Obligation → Batch → Commitment → Settlement → Verification → Lease  
→ Authorization**

A conventional integration may operate as:

**Checkout → Data Transmission → Settlement → Audit**

Haven operates according to:

**Checkout → Payment Enclosure → Settlement Obligation → Controlled Batch Settlement  
→ Independent Verification → Lease-Bound Authorization → Authorized Data  
Transmission**

The payment condition is therefore evaluated before Haven-governed interoperability is permitted to continue.

## **1.2 Settlement and Coordination Boundary**

The settlement engine maintains an off-chain ledger containing the event-level economic obligations generated by Haven-governed system events.

Individual obligations MAY be accumulated into bounded settlement batches subject to configured limits for:

- event count;
- aggregate settlement value;
- maximum batch age;
- settlement urgency;
- gas conditions; and
- operational recovery policy.

A sealed batch MUST receive a cryptographic commitment before on-chain submission.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.  
Licensed under GNU AGPL-3.0 rules.

The commitment **MUST** bind the relevant settlement obligations without exposing patient-identifiable information, protected health information, clinical narratives, or other sensitive healthcare payloads to Polygon EVM state or event logs.

The Polygon settlement transaction establishes the authoritative cryptographic settlement record for the corresponding batch.

The Haven edge enforcement layer independently verifies that settlement record before issuing or renewing the authorization lease required for continued coordination.

### **1.3 Trust Transition**

The architecture therefore creates a controlled sequence of trust transitions:

#### **CLIENT STATE**



Untrusted event and transaction input

#### **HAVEN SETTLEMENT STATE**



Off-chain obligation recorded and reconciled

#### **BATCH STATE**



Bounded obligations sealed into a cryptographically committed settlement unit

#### **POLYGON SETTLEMENT STATE**



Settlement transaction submitted and independently observed

#### **VERIFIED SETTLEMENT STATE**



Contract identity, transaction inclusion, commitment, aggregate amount, routing and confirmation conditions independently validated

#### **AUTHORIZATION LEASE**



Time-bounded permission to continue Haven-governed coordination

#### **EDGE ENFORCEMENT**



Cross-network translation permitted only while all authorization conditions remain valid

Any unresolved transition **MUST** prevent the system from treating the downstream state as valid.

The protocol therefore does not equate transaction initiation, batch creation, transaction submission, blockchain inclusion, settlement verification, or synchronization with authorization.

Authorization is the terminal state of a successful chain of independently verifiable conditions.

## **1.4 ENFORCEMENT LOOP**

The production enforcement loop is defined as:

### **EVENT**

A Haven-governed transaction generates a deterministic event block and corresponding protocol settlement obligation.

↓

### **OBLIGATION**

The applicable infrastructure fee is recorded in the off-chain settlement ledger as an outstanding economic obligation.

↓

### **BATCH**

Eligible outstanding obligations are accumulated into a bounded settlement batch according to configured event-count, value, age, urgency, and gas-policy constraints.

↓

### **COMMIT**

The sealed batch receives a cryptographic commitment that binds the settlement obligations without exposing PHI or clinical payloads on-chain.

↓

### **SETTLE**

The settlement engine evaluates current gas conditions, configured execution ceilings, dynamic volatility protection, batch economics, and retry policy before submitting the settlement transaction to Polygon.

A temporary gas-policy violation MAY defer settlement execution.

Gas deferral **MUST NOT** cancel the underlying obligation and **MUST NOT** create an authorization bypass.



## **OBSERVE**

The Haven verification layer receives settlement-related blockchain events through the authenticated primary transport:

### **WebSocket / gRPC Event Stream**

The event stream provides low-latency observation and sequencing information.

Event-stream presence alone **MUST NOT** establish settlement truth.



## **CROSS-CHECK**

The edge verification engine independently cross-checks the observed state against Polygon JSON-RPC.

JSON-RPC provides authoritative blockchain-state reconstruction, transaction lookup, receipt verification, contract-code verification, historical backfill, and recovery following event-stream interruption or cursor gaps.

The system **MUST** be capable of reconstructing authoritative verification state without relying exclusively on the live event stream.



## **VERIFY**

The verification engine independently validates, as applicable:

- expected Polygon chain ID;
- expected splitter contract address;
- expected contract runtime identity;
- transaction inclusion;
- transaction execution result;
- expected settlement commitment;
- expected aggregate settlement amount;
- expected 95/5 allocation;
- expected routing destinations;
- required confirmation or finality condition;

- event sequence continuity;
- verification cursor consistency; and
- absence of unresolved reorganization or state divergence.

↓

## **LEASE**

Successful verification produces or renews a time-bounded authorization lease containing the verified synchronization state, verification cursor, source information, issuance time, expiry time, and relevant blockchain reference.

The lease **MUST** expire automatically unless refreshed through successful independent verification.

↓

## **DECIDE**

The Haven edge adapter evaluates the complete authorization predicate:

### **STREAM HEALTHY**

- **LEASE VALID**
- **STATE VERIFIED**
- **CURSOR CONSISTENT**
- **CONTRACT IDENTITY VALID**
- **SETTLEMENT STATE VALID**

Only when every required condition evaluates positively may the participating network remain authorized.

↓

## **ALLOW / HALT**

If all required conditions are valid:

### **AUTHORIZED → CONTINUE HAVEN-GOVERNED COORDINATION**

If any required condition is false, unknown, stale, expired, inconsistent, corrupted, or otherwise unverifiable:

### **UNAUTHORIZED → CIRCUIT BREAKER ENGAGED**

A WebSocket or gRPC interruption **MUST** therefore not automatically authorize continued operation merely because a previous state was valid.

Likewise, a successful RPC response **MUST NOT** independently authorize operation if the associated lease has expired, the verification cursor is inconsistent, the contract identity is invalid, or settlement evidence remains incomplete.

The governing enforcement principle is:

**VERIFICATION MUST BE CONTINUOUSLY EARNED, NOT ASSUMED FROM PRIOR CONNECTIVITY.**

The edge layer therefore remains fail-closed across transport failure, RPC failure, network partition, stale state, cursor gaps, event-stream recovery, settlement reorganization, and unresolved verification conditions.

## 2. ECONOMIC MODEL & LEDGER TOKENOMICS

### 2.1 Protocol Economic Primitive

The Haven Protocol does not require a native speculative token to perform its core coordination function.

Its primary economic primitive is the **Protocol Event Fee** attached to an eligible system event block.

The fee represents the infrastructure cost of maintaining the Haven coordination rail across otherwise fragmented healthcare software environments.

The baseline commercial parameter is:

**Protocol Event Fee = \$0.05 / ₦100 per eligible system event block**

The protocol implementation must distinguish between:

1. the **underlying healthcare transaction value**;
2. the **merchant or institutional service charge**;
3. the **Haven infrastructure pass-through fee**; and
4. any applicable external payment processor, banking, network, tax, or regulatory charges.

Haven must never silently conflate these economic categories.

### 2.2 Event Block Definition

For tokenomic accounting purposes, an **Event Block** is a deterministic representation of a Haven-governed state transition or coordination event.

An Event Block may represent, depending upon the deployment profile:

- patient registration synchronization;
- encounter-state synchronization;
- referral-state transition;
- laboratory-result availability notification;
- prescription-state transition;
- pharmacy fulfillment-state transition;
- HMO authorization-state transition;
- claims coordination;
- eligibility verification;
- provider-network coordination;
- payment-state synchronization; or
- another explicitly registered interoperability event.

The event itself remains independent of the payment asset.

Haven coordinates the state transition.

The protocol fee represents the economic obligation associated with processing that coordination event.

## 2.3 Pass-Through Economic Model

The fee must be represented as an infrastructure pass-through rather than disguised as a clinical service charge.

The checkout calculation is conceptually:

**Consumer Transaction Total = Base Transaction Amount + Applicable Haven Infrastructure Fee + Other Applicable Charges**

Where:

**Applicable Haven Infrastructure Fee = Eligible Event Count × Applicable Event Rate**

For a single eligible event:

**Fee = \$0.05**

or, under the defined local-currency representation:

**Fee = ₦100**

where the deployment's commercial configuration establishes the applicable denomination and conversion methodology.

A production deployment must not silently alter the economic meaning of the fee through frontend rounding.

## 2.4 Fee Calculation Authority

The consumer-facing checkout may display a calculated estimate.

However, the display layer is not the ultimate enforcement authority.

The production architecture should maintain three separate concepts:

### Display Calculation

Used to communicate the expected fee to the consumer.

## Transaction Calculation

Used by the payment orchestration layer to construct the transaction.

## Settlement Verification

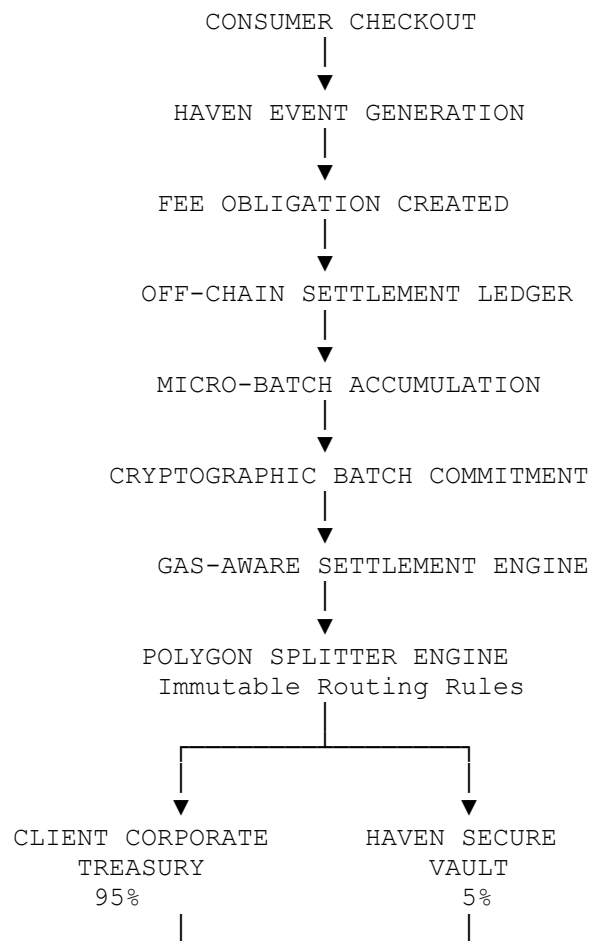
Used by the Haven enforcement layer to establish whether the required economic condition has actually been satisfied.

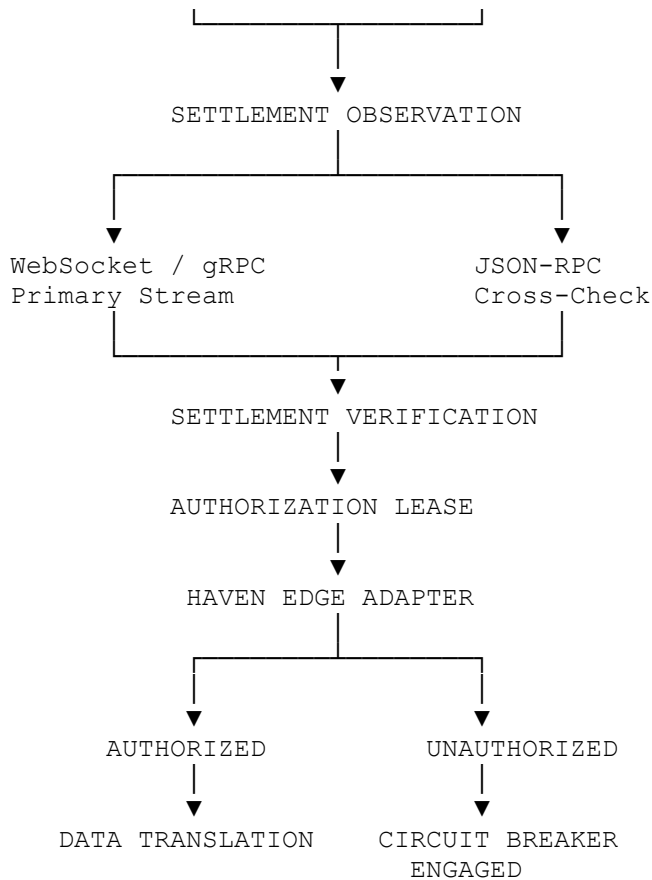
These values must be independently traceable.

A malicious or defective client application must therefore be unable to obtain authorization simply by changing the value rendered to the customer.

## 2.5 ECONOMIC ROUTING MODEL

The designated settlement architecture is conceptually represented as:





The critical architectural property is that the payment route remains upstream of authorization while the blockchain execution path is decoupled from the individual event-generation path.

A Haven event creates an economic obligation.

The obligation is recorded off-chain and MAY be accumulated with other eligible obligations into a bounded settlement batch.

The batch is sealed and cryptographically committed before settlement execution.

The settlement engine then evaluates the current Polygon execution environment before submitting the batch for on-chain settlement.

### 2.5.1 Batch Settlement Boundary

Batching is an execution optimization and MUST NOT constitute an economic waiver.

The underlying event-level obligations remain individually attributable within the off-chain settlement ledger even when they are aggregated for blockchain execution.

The batch MUST preserve deterministic correlation between:

- event identifiers;
- network identifiers;
- applicable fee obligations;
- batch identifier;
- batch sequence;
- aggregate settlement amount;
- settlement asset;
- cryptographic commitment; and
- resulting Polygon transaction evidence.

The batch therefore functions as a cryptographically bounded settlement container rather than as a replacement for event-level accounting.

### **2.5.2 Gas-Aware Settlement**

The settlement engine MUST prevent Polygon gas volatility from becoming a consumer checkout dependency.

The PoS transaction path MUST NOT require an individual on-chain transaction for every Haven event.

Before submitting a batch, the settlement engine evaluates:

- current gas estimate;
- configured gas ceiling;
- dynamic gas volatility buffer;
- maximum acceptable execution cost;
- batch aggregate value;
- minimum economic efficiency threshold;
- settlement urgency;
- retry eligibility; and
- recovery state.

Where execution conditions exceed the configured gas policy, the batch MAY remain pending in the off-chain settlement ledger.

The system MUST NOT reinterpret gas deferral as settlement completion.

The outstanding obligation remains enforceable until one of the following controlled states is reached:

**SETTLED**

or

**RECONCILED**

or

**EXPLICITLY CANCELLED UNDER A PERMITTED STATE TRANSITION**

or

**CONTROLLED RECOVERY**

No frontend, merchant administrator, client backend, payment processor, or edge adapter may convert a gas-deferred obligation into an authorized settlement state.

### **2.5.3 Immutable Allocation**

Once a valid batch reaches the Polygon splitter, the designated allocation remains governed by the contract's immutable routing rules:

**95% → Designated Client Corporate Treasury**

**5% → Haven Secure Vault**

The client application **MUST NOT** determine, modify, or negotiate these destinations at execution time.

The batch settlement mechanism therefore changes **when and how** settlement reaches the blockchain, but does not change **what economic allocation the protocol requires**.

### **2.5.4 Verification Boundary**

The blockchain settlement record is observed through the authenticated primary event transport and independently reconstructed through JSON-RPC.

The primary transport provides low-latency propagation.

The JSON-RPC path provides independent blockchain-state verification and recovery.

Neither transport is sufficient by itself to establish authorization.

The authoritative verification state is established only when the required blockchain evidence, contract identity, batch commitment, aggregate amount, routing outcome, confirmation conditions, synchronization cursor, and authorization lease requirements have been independently satisfied.

The Haven edge adapter therefore operates downstream of settlement verification rather than downstream of client-reported payment status.

The governing sequence is:

**Economic Obligation → Controlled Settlement → Independent Verification → Lease → Authorization**

and never:

**Client Payment Flag → Authorization.**

## **3. ZERO-TRUST PAYMENT ROUTING ENCLOSURE**

### **3.1 Purpose**

The Zero-Trust Payment Routing Enclosure Engine establishes a protected economic boundary around every Haven-enabled transaction.

Its purpose is to prevent a participating organization from modifying the application layer in a manner that causes Haven coordination traffic to continue while the corresponding protocol fee is diverted, suppressed, reduced, or eliminated.

The engine therefore treats payment routing as an infrastructure security property.

### **3.2 Enclosure Boundary**

The enclosure begins at the consumer checkout transaction and terminates at independently verifiable on-chain settlement state.

Within that boundary, the protocol controls:

- fee-route destination;
- designated splitter contract;
- allocation logic;
- synchronization state;
- authorization interpretation;
- edge enforcement;
- failure handling; and
- recovery conditions.

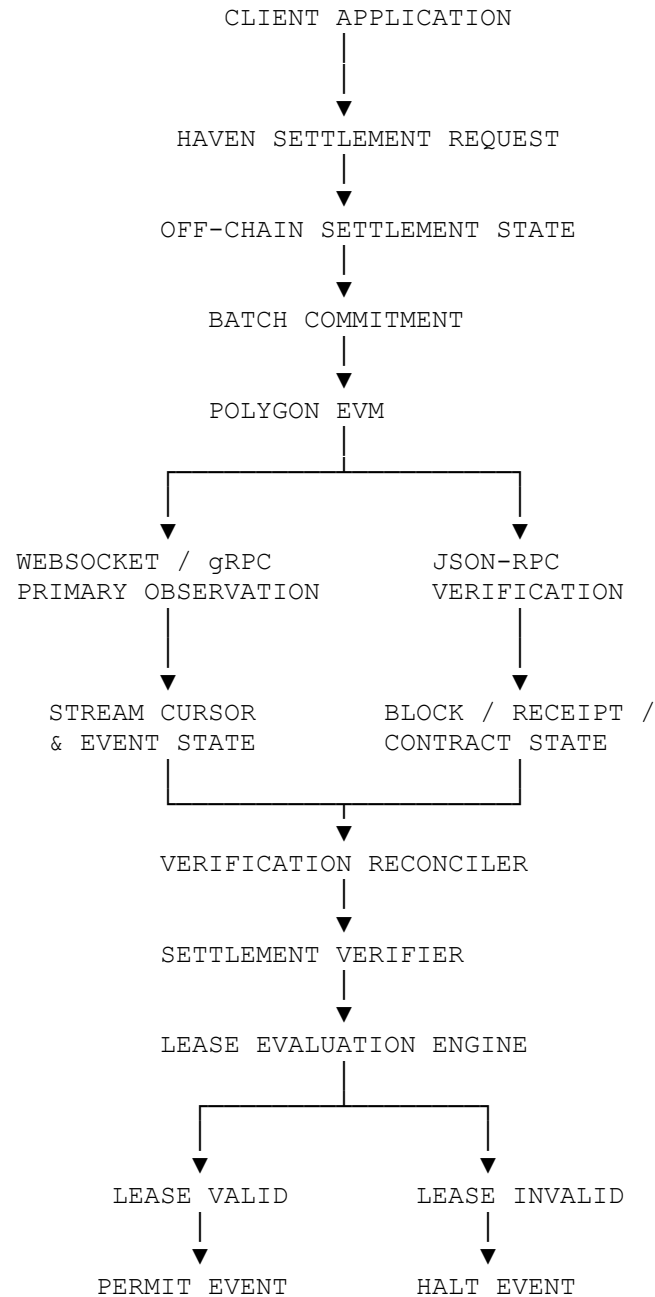
The client controls its own internal business software but does not control the Haven enforcement decision.

### **3.3 INDEPENDENT VERIFICATION PATH**

Client-side payment state is not authoritative because the party generating the application state may also control the value being evaluated.

The Haven enforcement model therefore requires an independent, dual-transport verification path.

The production verification architecture is:



### 3.3.1 Primary Event Transport

WebSocket or gRPC event streaming is the primary low-latency transport for settlement observation.

The primary stream is responsible for delivering:

- settlement-related events;
- block references;
- event sequence information;
- stream cursor information;
- contract-origin information; and
- transport health signals.

A healthy stream does not itself establish settlement truth.

An event received through the stream **MUST** be independently correlated against the expected contract identity and blockchain state before it can contribute to authorization.

### **3.3.2 JSON-RPC Verification and Recovery**

Optimized JSON-RPC is the independent verification and recovery transport.

It **MUST** support, as applicable:

- block retrieval;
- transaction lookup;
- transaction receipt retrieval;
- contract-code verification;
- historical event backfill;
- settlement commitment verification;
- confirmation tracking;
- reorganization detection; and
- recovery from stream interruption or cursor discontinuity.

JSON-RPC polling **MUST NOT** be implemented as a fixed high-latency substitute for event streaming under normal operation.

Polling intervals **MUST** be dynamically controlled according to synchronization health, cursor state, block progression, recovery requirements, and configured verification policy.

### **3.3.3 Stream/RPC Reconciliation**

The verification engine **MUST** continuously reconcile the primary event stream against independently reconstructed blockchain state.

For an observed settlement event:

#### **STREAM EVENT**

**MUST** correlate with:

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.  
Licensed under GNU AGPL-3.0 rules.

## **EXPECTED CONTRACT IDENTITY**

and:

## **VALID TRANSACTION / RECEIPT**

and:

## **EXPECTED BATCH COMMITMENT**

and:

## **EXPECTED AGGREGATE SETTLEMENT AMOUNT**

and:

## **EXPECTED ROUTING OUTCOME**

and:

## **REQUIRED CONFIRMATION CONDITION**

If the stream and RPC sources disagree, the system **MUST NOT** select the more permissive result.

The resulting state **MUST** be:

## **STREAM/RPC\_DISAGREEMENT → UNAUTHORIZED**

until independent reconciliation resolves the discrepancy.

{

"\$schema": "https://protocol.infrastructure",

"title": "HavenProtocolPoSEnclosurePayload",

"type": "object",

"required": [

"transactionHeader",

"coordinationCommitment",

"economicEnclosureGateway"

```

],
"properties": {
  "transactionHeader": {
    "type": "object",
    "required": ["posTransactionId", "timestamp", "networkIdentifier"],
    "properties": {
      "posTransactionId": { "type": "string", "format": "uuid" },
      "timestamp": { "type": "string", "format": "date-time" },
      "networkIdentifier": { "type": "string", "pattern": "^NET-[0-9]{4}$" }
    }
  },
  "coordinationCommitment": {
    "type": "object",
    "required": ["statelessEventId", "payloadDigest"],
    "properties": {
      "statelessEventId": { "type": "string", "format": "uuid" },
      "payloadDigest": { "type": "string", "pattern": "^sha256:[a-fA-F0-9]{64}$" }
    }
  },
  "economicEnclosureGateway": {
    "type": "object",
    "required": ["authoritativeContractAddress", "infrastructureTaxDenomination",
"protocolVersion"],
    "properties": {

```

```

    "authoritativeContractAddress": {
      "type": "string",
      "const": "0xC02E7Fe6AAfFF15F0bc4636B53E6d9D20898c4ac"
    },
    "infrastructureTaxDenomination": {
      "type": "string",
      "const": "NGN_100"
    },
    "protocolVersion": {
      "type": "string",
      "const": "2.0"
    }
  }
}
}
}
}
}

```

### 3.3.4 Cursor Continuity

Every event-stream verification session **MUST** maintain a monotonic verification cursor.

The cursor **MUST** permit the verification engine to determine:

- the last verified block;
- the last verified event sequence;
- the corresponding block hash;
- the associated settlement state; and
- the point from which historical recovery must resume.

A cursor gap **MUST** trigger controlled backfill.

The edge adapter **MUST NOT** assume that an event was received merely because later events were successfully delivered.

The recovery sequence is:

### **CURSOR GAP**

- **SUSPEND AUTHORIZATION**
- **RECONSTRUCT MISSING RANGE**
- **RECONCILE STREAM AND RPC STATE**
- **REVALIDATE SETTLEMENT**
- **ISSUE / RENEW LEASE**
- **RESUME AUTHORIZED COORDINATION**

### **3.3.5 Transport Failure**

If WebSocket or gRPC connectivity is interrupted, the edge verifier **MAY** transition to controlled RPC-based recovery.

However, RPC recovery does not automatically preserve an existing authorization lease.

If the interruption causes the verification cursor, settlement state, or lease freshness to become uncertain, authorization **MUST** be suspended until the required state is independently re-established.

Likewise, an RPC failure **MUST NOT** cause the system to assume that the previously verified state remains valid indefinitely.

The protocol therefore maintains the following rule:

**TRANSPORT FAILURE → VERIFICATION REQUIRED**

and:

**UNRESOLVED VERIFICATION → UNAUTHORIZED**

### **3.3.6 Verification Authority**

The client application remains an input source.

The event stream remains an observation source.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.  
Licensed under GNU AGPL-3.0 rules.

JSON-RPC remains an independent blockchain-state reconstruction source.

The settlement verifier is responsible for determining whether the observed evidence satisfies protocol requirements.

The lease engine is responsible for converting verified state into time-bounded authorization.

The edge circuit breaker is responsible for enforcing the resulting authorization decision.

No client-side Boolean, API response, cached payment state, database field, frontend variable, or application-generated confirmation may override this hierarchy.

The authoritative path is therefore:

#### **CLIENT INPUT**

→ **SETTLEMENT EVIDENCE**

→ **DUAL-TRANSPORT OBSERVATION**

→ **INDEPENDENT RECONCILIATION**

→ **VERIFIED STATE**

→ **LEASE**

→ **EDGE AUTHORIZATION**

Any break in this chain causes enforcement to fail closed.

### **3.4 Immutable Routing Requirement**

The designated payment endpoint must be a contract whose routing logic cannot be changed by a client administrator, merchant operator, frontend developer, API developer, or ordinary backend administrator.

The production deployment must therefore establish a clearly defined contract immutability policy.

Where the protocol requires an **immutable, non-upgradable splitter**, an implementation using proxy upgradeability must not be substituted merely for operational convenience.

Any upgrade mechanism creates an additional trust surface.

If an upgrade mechanism is ever required for a separate deployment profile, it must be treated as a different security model and documented as such rather than represented as equivalent to an immutable deployment.

### **3.5 Destination Separation**

The splitter must maintain predetermined destination addresses for the economic allocations.

At minimum, the architecture identifies:

#### **Client Corporate Treasury**

The destination receiving the designated corporate allocation.

#### **Haven Secure Vault**

The destination receiving the designated Haven allocation.

The secure vault must be controlled through a hardware-backed multisignature custody arrangement.

The private keys must not be embedded inside:

- checkout applications;
- frontend code;
- client backend servers;
- edge adapters;
- environment files;
- CI/CD pipelines;
- smart-contract source variables exposed to clients; or
- ordinary cloud secrets.

### **3.6 Treasury Security Model**

The preferred custody architecture is a Safe multisignature wallet controlled by hardware-secured signing devices.

The operational principle is:

**Smart Contract → Safe Multisig → Hardware Signers → Air-Gapped Treasury Control**

The smart contract does not require possession of the treasury's private key.

It requires only the immutable destination address.

The signing authority remains separated from the payment-routing execution layer.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.  
Licensed under GNU AGPL-3.0 rules.

This separation limits the blast radius of a compromise.

A compromised client application should not be capable of obtaining treasury signing authority.

A compromised edge adapter should not be capable of modifying splitter destinations.

A compromised checkout frontend should not be capable of redefining the protocol's economic route.

### 3.7 Synchronization State

The splitter architecture may expose a synchronization registry of the form:

```
mapping(address => bool) public isNetworkSynchronized;
```

The mapping must not be interpreted as a client-controlled boolean.

The critical security question is **who is authorized to cause a state transition**.

The production implementation must therefore explicitly define:

- which address can modify synchronization state;
- what event causes synchronization to become valid;
- what event causes synchronization to become invalid;
- whether synchronization is automatically derived from settlement;
- how the edge adapter interprets the state;
- how stale state is detected;
- how revoked participants are handled; and
- how unauthorized state mutation is rejected.

The mere existence of a public mapping does not itself establish security.

The authorization model surrounding its mutation is part of the protocol's security boundary.

### 3.8 Circuit-Breaker Principle

The edge adapter must operate under a fail-closed model.

Where the adapter cannot establish the required synchronization state, it must not assume that synchronization exists.

The default condition is therefore:

**Unknown = Not Authorized**

This includes:

- JSON-RPC timeout;
- unavailable RPC provider;
- malformed response;
- contract-call failure;
- stale synchronization state;
- invalid contract address;
- unexpected chain ID;
- incorrect contract bytecode;
- failed state verification;
- synchronization explicitly set to false; or
- evidence of payment-route divergence.

The edge enforcement layer must consequently distinguish between:

**Verified Synchronized**

and

**Everything Else**

Only the first condition permits continued Haven-governed cross-network coordination.

## **4. POINT-OF-SALE TRANSACTION LIFECYCLE**

### **4.1 End-to-End Transaction Sequence**

A production Haven PoS implementation shall treat consumer checkout as the initiation point of a deterministic economic and coordination lifecycle.

The checkout transaction does **not** directly create an on-chain settlement obligation. Instead, the system creates a Haven-governed event and its associated economic obligation, which may subsequently be accumulated into a bounded settlement batch before Polygon execution.

The canonical lifecycle is:

#### **1. Consumer Initiates Checkout**

The consumer selects a healthcare service, product, claim-related transaction, or other billable interaction.

#### **2. PoS Constructs Transaction Context**

The client PoS system creates a transaction context containing the commercial transaction identifier, participating network identifier, event classification, amount, currency, timestamp, and other permitted metadata.

#### **3. Haven Event Is Classified**

The integration layer determines whether the transaction produces one or more Haven-governed event blocks.

#### **4. Protocol Fee Obligation Is Determined**

The trusted Haven enforcement boundary calculates the applicable infrastructure obligation according to the configured event-rate policy.

The client interface may display the resulting pass-through amount, but the client application does not become the authority for redefining the protocol fee.

#### **5. Haven Economic Obligation Is Recorded**

The event and its associated fee obligation are committed to the off-chain settlement ledger.

The obligation receives a deterministic correlation identity linking the Haven event, commercial transaction, network, and payment attempt without placing PHI on the public blockchain.

## **6. Checkout Payment Is Authorized**

The consumer completes the commercial payment through the configured payment processor or payment rail.

Commercial payment authorization and Haven protocol settlement remain separate state transitions.

## **7. Settlement Evidence Is Correlated**

The payment orchestration layer records the processor reference and associated evidence required to correlate the commercial payment with the Haven economic obligation.

## **8. Obligation Enters the Bounded Settlement Queue**

The confirmed Haven obligation becomes eligible for accumulation into a bounded micro-batch.

The batching layer may aggregate multiple valid obligations while preserving per-event correlation, amount integrity, ordering requirements, and idempotency controls.

## **9. Cryptographic Batch Commitment Is Created**

Before on-chain submission, the settlement service creates a deterministic cryptographic commitment representing the obligations contained within the batch.

The commitment establishes an integrity boundary between the off-chain obligation ledger and the corresponding on-chain settlement transaction.

## **10. Gas-Aware Settlement Decision Is Evaluated**

The settlement engine evaluates current Polygon execution conditions against the configured gas policy and dynamic volatility buffer.

Normal conditions permit settlement execution.

Elevated gas conditions may defer or re-batch eligible obligations without cancelling the underlying economic obligation.

## **11. Polygon Settlement Is Executed**

An authorized settlement executor submits the committed batch through the production Haven Protocol Splitter.

The splitter remains the authoritative on-chain allocation mechanism for the protocol settlement.

## **12. Settlement Is Independently Verified**

Settlement evidence is observed through the primary event transport and independently cross-checked through the fallback JSON-RPC verification path.

Stream presence alone does not constitute settlement truth.

## **13. Authorization Lease Is Issued or Renewed**

Only after the required settlement and synchronization conditions have been independently verified may the participating edge environment receive or renew a bounded authorization lease.

## **14. Cross-Network Coordination Is Released**

The Haven edge adapter permits the corresponding governed coordination operation only while the required verification state and authorization lease remain valid.

Any unresolved, stale, contradictory, or expired condition results in authorization denial.

The lifecycle therefore preserves the following separation:

**Event Creation  $\neq$  Economic Settlement**

**Economic Obligation  $\neq$  On-Chain Settlement**

**Payment Processor Success  $\neq$  Polygon Settlement**

**Event Stream Presence  $\neq$  Settlement Truth**

**Settlement Verification  $\neq$  Authorization**

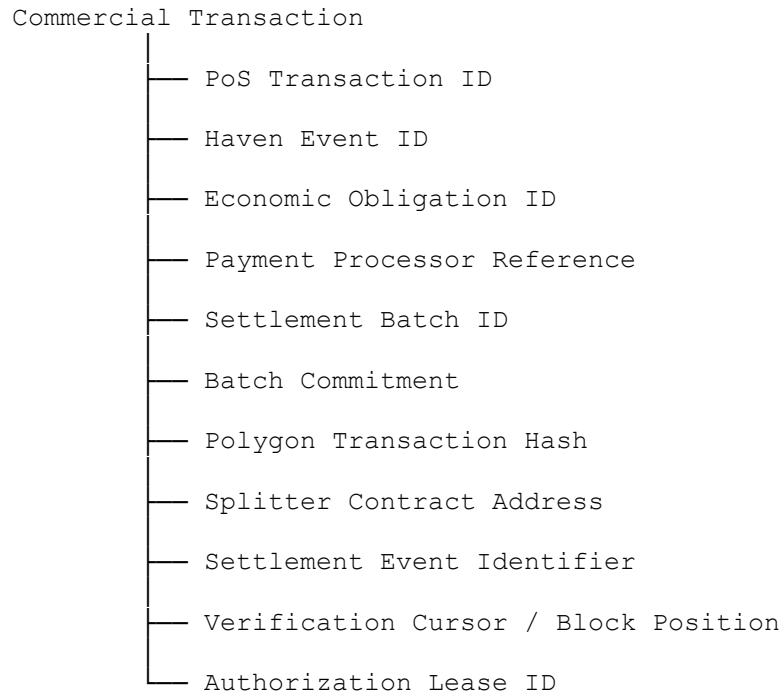
**Authorization  $\neq$  Clinical Data Ownership**

This separation is mandatory for production operation.

## **4.2 TRANSACTION CORRELATION AND SETTLEMENT CONTINUITY**

Every Haven-governed PoS transaction must possess a deterministic correlation identity capable of linking the commercial transaction to its protocol event, economic obligation, settlement batch, and independently verified Polygon settlement evidence.

The canonical correlation chain is:



The correlation layer must support deterministic reconstruction of the complete transaction path without requiring the public blockchain to contain the underlying clinical or commercial record.

#### 4.2.1 Off-Chain Correlation Authority

The off-chain settlement ledger maintains the authoritative relationship between:

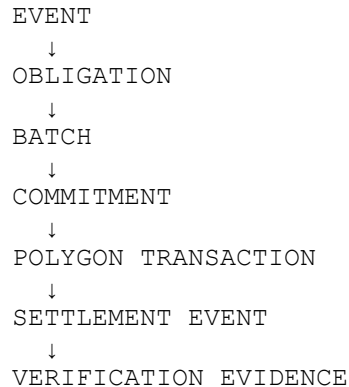
- PoS transaction;
- Haven event;
- economic obligation;
- payment attempt;
- settlement batch;
- batch commitment;
- settlement status;
- verification evidence;
- synchronization state; and
- authorization lease.

The ledger must support idempotent updates and must prevent a duplicated payment webhook, transport retry, RPC response, or settlement submission from creating an uncontrolled duplicate obligation.

## 4.2.2 Batch Correlation Integrity

When multiple obligations are accumulated into a settlement batch, every individual obligation must remain independently traceable to the batch commitment.

A batch must therefore preserve:



A valid batch settlement does not permit the system to lose the identity or amount of an individual obligation.

A settlement batch must not be considered complete merely because a Polygon transaction exists. The system must establish that the observed settlement corresponds to the expected batch commitment and designated Haven Protocol Splitter.

## 4.2.3 Dual-Transport Verification Correlation

Settlement observation shall use two logically independent transport paths:

### Primary Event Transport

WebSocket or gRPC event streaming provides low-latency notification of relevant Polygon events and supports sub-second edge propagation under normal network conditions.

### Independent Verification Transport

JSON-RPC provides authoritative historical inspection, block-level verification, receipt retrieval, cursor recovery, and reconciliation when the event stream is delayed, interrupted, duplicated, or inconsistent.

#### 4.2.3.1 Blockchain Consensus Finality Threshold

The Independent Verification Transport SHALL explicitly insulate the edge authorization gate from public network forks and transient chain reorganizations (reorgs) native to the Polygon EVM environment.

- **Confirmation Depth Constraint:** A batch settlement transaction status query executed via JSON-RPC (`eth_getTransactionReceipt`) SHALL NOT evaluate to TRUE for the purpose of lease renewal until the target transaction hash is embedded within a block that has achieved a minimum depth of exactly 16 block confirmations.

- **Reorg Isolation:** Any on-chain settlement evidence resting at a block depth of less than 16 confirmations SHALL be treated as non-final metadata (`UNVERIFIED`). The verification engine SHALL withhold authorization propagation for that specific batch until the consensus depth threshold is securely surpassed.

The verification engine must reconcile:

```
Stream Event
    +
RPC Observation
    +
Expected Batch Commitment
    +
Expected Contract Identity
    +
Expected Settlement State
```

A stream event that cannot be reconciled with the independently verified chain state must not authorize downstream coordination.

#### 4.2.4 Cursor Continuity

The edge verification layer must maintain a monotonic observation cursor for the Polygon event stream.

If a cursor gap is detected, the adapter must:

1. stop treating subsequent stream events as sufficient authorization evidence;
2. mark the verification state as unresolved;
3. recover the missing range through JSON-RPC historical inspection;
4. reconcile recovered events against the expected settlement state;
5. restore stream continuity; and
6. only then issue or renew the authorization lease.

Duplicate events must be safely ignored through deterministic event identity and idempotent processing.

### 4.2.5 Privacy Boundary

The correlation mechanism must not require sensitive clinical information to be written onto the public blockchain.

Clinical payloads, identifiable patient records, diagnostic information, medical documents, and other sensitive healthcare data remain outside the public ledger.

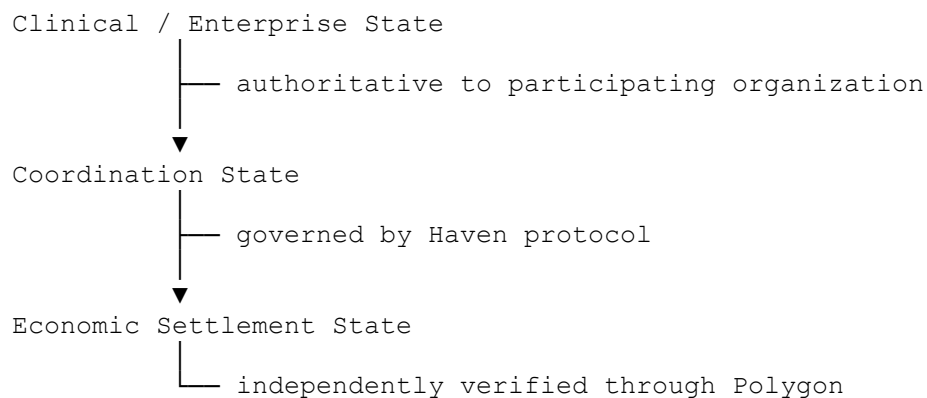
Polygon EVM settlement records shall contain only the minimum non-reversible cryptographic correlation material and economic settlement information required for protocol verification.

The public chain therefore functions as a settlement and integrity-verification boundary rather than a clinical data repository.

### 4.2.6 Stateless Coordination Constraint

Haven does not become the system of record for the participating healthcare enterprise.

The architecture continues to separate:



The participating organization remains responsible for its authoritative clinical and commercial records.

Haven provides the coordination rail through which permitted state transitions are transmitted, verified, sequenced, and acknowledged.

No successful Polygon settlement, event-stream observation, or authorization lease shall be interpreted as transferring ownership of the underlying clinical or commercial record to Haven.

## 4.3 Stateless Coordination Constraint

Haven does not become the system of record for the participating healthcare enterprise.

The PoS integration therefore must not transform the protocol into a centralized clinical database.

The architecture separates:

### **Clinical / Enterprise State**

from

### **Coordination State**

from

### **Economic Settlement State**

The participating organization remains responsible for its authoritative clinical and commercial records.

Haven provides the coordination rail through which permitted state transitions can be transmitted, verified, sequenced, and acknowledged.

## 4.4 Event-to-Fee Binding

The economic obligation must be deterministically associated with the event that generates it.

A conceptual event envelope is:

```
{
  "eventId": "EVT-...",
  "networkId": "HAVEN-NET-...",
  "eventType": "REFERRAL_STATE_CHANGE",
  "eventVersion": "1.0",
  "occurredAt": "2026-09-02T10:00:00Z",
  "feePolicy": {
    "required": true,
    "amount": 100,
    "currency": "NGN"
  }
}
```

The client may construct this object.

The client cannot unilaterally establish that the associated fee has been settled.

The authoritative settlement condition must be established independently.

## 5. CHECKOUT PORTAL INTEGRATION

### 5.1 Integration Boundary

The checkout portal is the first application boundary at which the Haven economic requirement becomes visible to the consumer.

A production integration should therefore separate the **presentation layer** from the **protocol enforcement layer**.

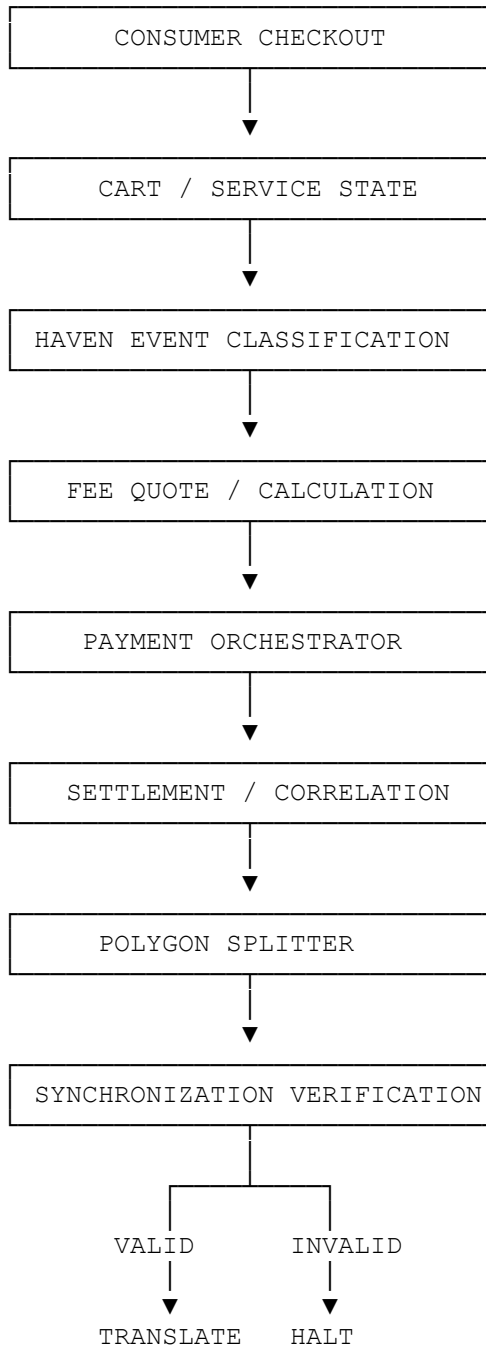
The consumer interface may render:

|                          |         |
|--------------------------|---------|
| Healthcare Service       | ₦10,000 |
| Haven Infrastructure Fee | ₦100    |
| Other Applicable Charges | ₦0      |
| -----                    |         |
| Total                    | ₦10,100 |

The interface must not contain privileged logic capable of disabling or redefining the underlying protocol enforcement mechanism.

## 5.2 Canonical Checkout Flow

The recommended implementation map is:

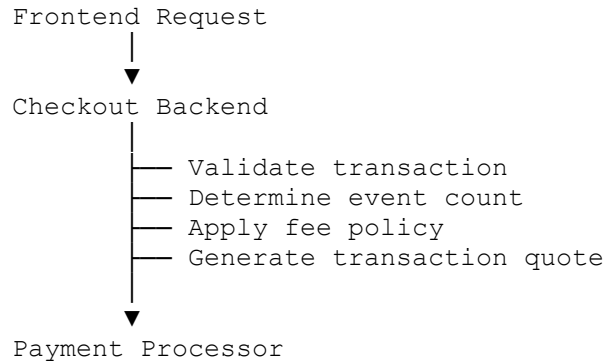


## 5.3 Server-Side Fee Enforcement

The checkout frontend must never be the sole location where the Haven fee is calculated.

A production system should calculate or validate the fee at a trusted server-side boundary before payment authorization.

Conceptually:



The frontend receives the resulting transaction quote.

It does not receive authority to redefine the protocol's fee policy.

## 5.4 Idempotency

Payment and event processing must be idempotent.

A duplicated HTTP request, browser retry, webhook retry, payment processor retry, or network timeout must not create an uncontrolled duplicate protocol charge or duplicate Haven event.

Each transaction should therefore contain a unique idempotency key.

Example:

```
idempotencyKey =  
  HASH(  
    merchantId ||  
    transactionId ||  
    eventSet ||  
    paymentAttempt  
  )
```

The exact construction must be defined by the implementation and must avoid incorporating unnecessary sensitive clinical information.

The backend must persist sufficient state to recognize previously accepted transaction attempts.

## **5.5 Payment Processor Separation**

The payment processor and Polygon settlement layer should be treated as separate infrastructure components.

The processor may confirm the consumer's commercial payment.

Polygon establishes the designated on-chain protocol settlement state.

Therefore:

### **Payment Processor Success $\neq$ Haven Synchronization**

and:

### **Haven Synchronization $\neq$ Clinical Transaction Ownership**

The integration must explicitly correlate these states without incorrectly treating one as proof of the other.

## **5.6 Failure Handling**

A production checkout must define behavior for:

### **Payment Declined**

The transaction remains incomplete and the associated Haven event must not be represented as successfully settled.

### **Payment Processor Timeout**

The system must place the transaction into a recoverable pending state rather than blindly retrying an unknown payment.

### **Polygon Submission Failure**

The commercial transaction and protocol settlement state become temporarily divergent and must enter a reconciliation workflow.

### **Splitter Rejection**

The protocol settlement attempt is unsuccessful and the Haven synchronization condition must not be represented as valid merely because the consumer payment succeeded.

### **JSON-RPC Failure**

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.  
Licensed under GNU AGPL-3.0 rules.

The edge adapter cannot establish authorization and must fail closed.

## **Synchronization False**

The adapter engages the circuit breaker and blocks Haven-governed cross-network translation until the required condition is restored.

The architecture must therefore distinguish **payment completion**, **on-chain settlement**, **synchronization**, and **data authorization** as separate state transitions.

## 6. PROTOCOL EVENT & FEE DATA MODEL

### 6.1 Canonical Event Envelope

Every Haven billable system event **MUST** be represented by a canonical protocol event envelope before it can enter the economic obligation and settlement pipeline.

The canonical envelope is an **off-chain protocol object**. It is not a clinical record, does not constitute a patient database entry, and **MUST NOT** contain Protected Health Information (PHI).

The upgraded canonical structure is:

```
{
  "schemaVersion": "2.0.0",
  "eventId": "evt_...",
  "networkId": "polygon-mainnet",
  "participantId": "participant_...",
  "eventType": "CLINICAL_COORDINATION",
  "eventVersion": "1.0",
  "occurredAt": "2026-09-04T08:30:00Z",
  "correlationId": "corr_...",
  "idempotencyKey": "idem_...",
  "economicObligation": {
    "required": true,
    "amount": "0.05",
    "currency": "USD",
    "feePolicyVersion": "2.0"
  },
  "settlement": {
    "status": "OBLIGATION_CREATED",
    "batchId": null,
    "batchCommitment": null,
    "transactionHash": null
  },
  "payloadReference": "ref_...",
  "integrity": {
    "algorithm": "SHA-256",
    "eventDigest": "..."
  }
}
```

### 6.2 Event Identity

`eventId` **MUST** uniquely identify the Haven protocol event.

`correlationId` **MUST** provide deterministic linkage between the commercial transaction, PoS transaction, Haven event, economic obligation, settlement batch, and subsequent settlement verification record.

`idempotencyKey` **MUST** prevent duplicate economic obligations from being created when the same transaction is retried, replayed, or delivered through multiple transport paths.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.  
Licensed under GNU AGPL-3.0 rules.

Event identity MUST remain independent of clinical identity.

No public-chain address, transaction hash, or wallet identifier may be treated as a substitute for the canonical Haven event identity.

### 6.3 Economic Obligation Object

The `economicObligation` object represents the protocol's enforceable economic requirement generated by a qualifying Haven event.

The standard Haven protocol charge remains:

**\$0.05 / ₦100 per system event block.**

The creation of the economic obligation occurs independently of the eventual Polygon settlement transaction.

Accordingly:

**Event Creation  $\neq$  Settlement Execution**

**Economic Obligation  $\neq$  On-Chain Settlement**

A temporary inability to submit an on-chain settlement transaction MUST NOT erase, mutate, or cancel a valid economic obligation.

The obligation MUST remain recorded in the off-chain settlement ledger until it is either included in an authorized settlement batch or placed into an explicitly governed exception/recovery state.

### 6.4 Settlement Object

The `settlement` object provides the state transition from economic obligation to independently verified settlement.

Permitted lifecycle states include:

```
OBLIGATION_CREATED  
BATCH_ELIGIBLE  
BATCH_COMMITTED  
SETTLEMENT_SUBMITTED  
SETTLEMENT_OBSERVED  
SETTLEMENT_VERIFIED
```

The following fields **MUST** remain nullable until their corresponding lifecycle stage exists:

- `batchId`
- `batchCommitment`
- `transactionHash`

A client application **MUST NOT** populate settlement fields with fabricated or assumed values merely to satisfy schema validation.

Settlement truth **MUST** be established independently by the Haven settlement and verification layer.

## 6.5 Payload Reference and Privacy Boundary

`payloadReference` **MUST** identify the off-chain data or workflow object associated with the protocol event without exposing the underlying clinical payload to the Polygon network.

PHI, clinical notes, diagnostic information, patient-identifying information, medical documents, and other sensitive healthcare payloads **MUST** remain outside the public ledger.

The Polygon EVM layer **MUST** receive only the minimum cryptographic correlation material necessary to establish settlement integrity.

The public-chain representation therefore acts as a **settlement proof and economic coordination anchor**, not as a clinical data repository.

The protocol **MUST** preserve the following invariant:

```
100% PHI OFF-CHAIN
+
NON-REVERSIBLE HASH CORRELATIONS ON-CHAIN
=
CLINICAL PRIVACY PRESERVATION
```

No event schema may introduce a pathway through which PHI can be serialized into transaction calldata, event logs, batch commitments, settlement metadata, or public-chain correlation fields.

## 6.6 Event Integrity

Every canonical Haven event **MUST** contain sufficient integrity metadata to detect alteration, duplication, truncation, or unauthorized substitution before the event can create or modify an economic obligation.

The canonical integrity mechanism is based on a deterministic event digest.

```
eventDigest = SHA-256(canonicalized_event_payload)
```

The canonicalization procedure **MUST** be deterministic across all compliant Haven implementations.

The integrity layer **MUST** verify:

1. Schema validity.
2. Event identity.
3. Network identity.
4. Correlation identity.
5. Idempotency identity.
6. Economic obligation integrity.
7. Payload-reference integrity.
8. Event digest integrity.

An integrity failure **MUST** prevent the event from advancing into settlement eligibility.

## 6.7 Public-Ledger Data Minimization

The Polygon EVM layer **MUST** operate under strict data minimization.

The following classes of information **MUST NOT** be written to the public ledger:

- Patient names.
- Patient identifiers.
- Medical record numbers.
- Clinical notes.
- Diagnoses.
- Prescriptions.
- Laboratory results.
- Imaging information.
- Provider notes.
- Insurance details.
- Contact information.
- Raw clinical payloads.
- Any reversible identifier capable of reconstructing the underlying patient identity.

Only non-reversible cryptographic correlation material and the minimum settlement information required by the protocol **MAY** cross the public-chain boundary.

The public ledger therefore establishes **economic and cryptographic settlement evidence**, while the clinical and transactional payload remains under the appropriate off-chain governance boundary.

## 6.8 Event-to-Settlement State Progression

A valid Haven event **MUST** progress through a controlled state sequence:

```
OBLIGATION_CREATED
↓
BATCH_ELIGIBLE
↓
BATCH_COMMITTED
↓
SETTLEMENT_SUBMITTED
↓
SETTLEMENT_OBSERVED
↓
SETTLEMENT_VERIFIED
```

A state transition **MUST NOT** be inferred solely from the existence of an application callback, frontend response, payment processor response, or event-stream message.

The authoritative transition requires the corresponding settlement evidence to satisfy the Haven verification rules.

In particular:

```
PAYMENT_SUCCESS
≠
POLYGON_SETTLEMENT

STREAM_OBSERVATION
≠
SETTLEMENT_TRUTH

SETTLEMENT_OBSERVATION
≠
AUTHORIZATION
```

## 6.9 Validation and Fail-Closed Semantics

Before an event becomes economically actionable, the following conjunction **MUST** evaluate to true:

```
Schema Valid
AND
Event Identity Valid
AND
Network Identity Valid
AND
Event Type Recognized
AND
Correlation Valid
AND
Idempotency Valid
```

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.  
Licensed under GNU AGPL-3.0 rules.

AND  
Economic Obligation Valid  
AND  
Integrity Metadata Valid

If any required condition evaluates to false, UNKNOWN, STALE, or UNVERIFIED, the event MUST NOT advance into an authorized settlement-dependent workflow.

Unknown event types MUST fail closed.

Malformed economic metadata MUST fail closed.

Duplicate or replayed events MUST fail closed until idempotency resolution is completed.

An event whose integrity cannot be independently established MUST fail closed.

## 6.10 Settlement Correlation Requirements

Once an economic obligation enters a settlement batch, the correlation chain MUST remain immutable:

Event ID  
→  
Economic Obligation ID  
→  
Settlement Batch ID  
→  
Batch Commitment  
→  
Polygon Transaction Hash  
→  
Splitter Settlement Event  
→  
Verification Cursor / Block Position  
→  
Authorization Lease

The settlement batch MUST preserve deterministic linkage to every obligation included within it.

A batch commitment MUST NOT be accepted as proof of settlement by itself.

The protocol MUST independently verify the corresponding Polygon transaction and settlement event before marking the associated obligations as `SETTLEMENT_VERIFIED`.

Only after successful verification may downstream authorization logic treat the corresponding settlement state as authoritative.

## 7. PRODUCTION JSON SCHEMA DEFINITIONS

### 7.1 Schema Design Requirements

Haven production interfaces **MUST** use machine-readable JSON Schema contracts conforming to **JSON Schema Draft 2020-12**.

Schemas **MUST** establish explicit boundaries between:

1. Haven event creation.
2. Economic obligation creation.
3. Settlement batch construction.
4. On-chain settlement evidence.
5. Dual-transport observation.
6. Verification state.
7. Authorization lease state.
8. Recovery and reconciliation state.

No client-controlled payload may establish settlement truth or authorization state merely by satisfying the event schema.

All schemas **MUST** use:

```
"$schema": "https://json-schema.org/draft/2020-12/schema"
```

and **MUST** reject undeclared properties where the object represents an authoritative protocol state.

The production schema layer **MUST** preserve the following invariants:

```
VALID EVENT
  ↓
ECONOMIC OBLIGATION
  ↓
BATCH ELIGIBILITY
  ↓
BATCH COMMITMENT
  ↓
SETTLEMENT SUBMISSION
  ↓
INDEPENDENT OBSERVATION
  ↓
SETTLEMENT VERIFICATION
  ↓
AUTHORIZATION LEASE
```

A schema-valid message is therefore **syntactically valid**, not automatically **economically authoritative, settlement-authoritative, or authorization-authoritative**.

## 7.2 Haven Event Envelope Schema

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://protocol.haven/schemas/event-envelope.schema.json",
  "title": "Haven Protocol Event Envelope",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "schemaVersion",
    "eventId",
    "networkId",
    "participantId",
    "eventType",
    "eventVersion",
    "occurredAt",
    "correlationId",
    "idempotencyKey",
    "economicObligation",
    "settlement",
    "payloadReference",
    "integrity"
  ],
  "properties": {
    "schemaVersion": {
      "type": "string",
      "const": "2.0.0"
    },
    "eventId": {
      "type": "string",
      "minLength": 16,
      "maxLength": 128,
      "pattern": "^[A-Za-z0-9._:-]+$"
    },
    "networkId": {
      "type": "string",
      "const": "polygon-mainnet"
    },
    "participantId": {
      "type": "string",
      "minLength": 3,
      "maxLength": 128
    },
    "eventType": {
      "type": "string",
      "minLength": 3,
      "maxLength": 128
    },
    "eventVersion": {
      "type": "string",
      "pattern": "^[0-9]+\\.\\.[0-9]+$"
    },
    "occurredAt": {
      "type": "string",
      "format": "date-time"
    }
  },
}
```

```

"correlationId": {
  "type": "string",
  "minLength": 8,
  "maxLength": 128
},
"idempotencyKey": {
  "type": "string",
  "minLength": 16,
  "maxLength": 256
},
"economicObligation": {
  "$ref": "#/$defs/economicObligation"
},
"settlement": {
  "$ref": "#/$defs/settlement"
},
"payloadReference": {
  "$ref": "#/$defs/payloadReference"
},
"integrity": {
  "$ref": "#/$defs/integrity"
}
},
"$defs": {
  "economicObligation": {
    "type": "object",
    "additionalProperties": false,
    "required": [
      "required",
      "amount",
      "currency",
      "feePolicyVersion"
    ],
    "properties": {
      "required": {
        "type": "boolean"
      },
      "amount": {
        "type": "string",
        "pattern": "^[0-9]+(\\.[0-9]{1,18})? $"
      },
      "currency": {
        "type": "string",
        "pattern": "[A-Z]{3} $"
      },
      "feePolicyVersion": {
        "type": "string",
        "minLength": 3,
        "maxLength": 64
      }
    }
  },
  "settlement": {
    "type": "object",
    "additionalProperties": false,
    "required": [

```

```

    "status"
  ],
  "properties": {
    "status": {
      "type": "string",
      "enum": [
        "OBLIGATION_CREATED",
        "BATCH_ELIGIBLE",
        "BATCH_COMMITTED",
        "SETTLEMENT_SUBMITTED",
        "SETTLEMENT_OBSERVED",
        "SETTLEMENT_VERIFIED"
      ]
    },
    "batchId": {
      "type": [
        "string",
        "null"
      ]
    },
    "batchCommitment": {
      "type": [
        "string",
        "null"
      ]
    },
    "transactionHash": {
      "type": [
        "string",
        "null"
      ],
      "pattern": "^0x[a-fA-F0-9]{64}$"
    }
  },
  "payloadReference": {
    "type": "object",
    "additionalProperties": false,
    "required": [
      "type",
      "referenceId"
    ],
    "properties": {
      "type": {
        "type": "string",
        "const": "EXTERNAL_REFERENCE"
      },
      "referenceId": {
        "type": "string",
        "minLength": 8,
        "maxLength": 256
      }
    }
  },
  "integrity": {
    "type": "object",

```

```

    "additionalProperties": false,
    "required": [
      "algorithm",
      "eventDigest"
    ],
    "properties": {
      "algorithm": {
        "type": "string",
        "const": "SHA-256"
      },
      "eventDigest": {
        "type": "string",
        "pattern": "^[a-fA-F0-9]{64}$"
      }
    }
  }
}

```

The event schema **MUST NOT** permit clinical content to be embedded directly within the event envelope.

The schema validates the protocol object's structure. It does not authorize settlement, release an authorization lease, or establish that a Polygon transaction has been confirmed.

### 7.3 Production Settlement Evidence Schema

Settlement evidence **MUST** be represented separately from the originating Haven event.

The settlement evidence object **MUST** capture sufficient information to independently determine whether an economic obligation has reached the designated Polygon settlement mechanism.

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://protocol.haven/schemas/settlement-evidence.schema.json",
  "title": "Haven Settlement Evidence",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "schemaVersion",
    "eventId",
    "correlationId",
    "batchId",
    "batchCommitment",
    "chainId",
    "contractAddress",
    "transactionHash",
    "blockNumber",
    "settlementStatus",
    "observedAt",
    "verificationState"
  ],
  "properties": {

```

```

"schemaVersion": {
  "type": "string",
  "const": "2.0.0"
},
"eventId": {
  "type": "string",
  "minLength": 16,
  "maxLength": 128
},
"correlationId": {
  "type": "string",
  "minLength": 8,
  "maxLength": 128
},
"batchId": {
  "type": "string",
  "minLength": 8,
  "maxLength": 128
},
"batchCommitment": {
  "type": "string",
  "pattern": "^[a-fA-F0-9]{64}$"
},
"chainId": {
  "type": "integer",
  "const": 137
},
"contractAddress": {
  "type": "string",
  "pattern": "^0x[a-fA-F0-9]{40}$"
},
"transactionHash": {
  "type": "string",
  "pattern": "^0x[a-fA-F0-9]{64}$"
},
"blockNumber": {
  "type": "integer",
  "minimum": 0
},
"settlementStatus": {
  "type": "string",
  "enum": [
    "SUBMITTED",
    "OBSERVED",
    "VERIFIED",
    "REJECTED",
    "REORGED"
  ]
},
"observedAt": {
  "type": "string",
  "format": "date-time"
},
"verificationState": {
  "type": "string",
  "enum": [

```

```

        "UNVERIFIED",
        "VERIFIED",
        "STALE",
        "DISPUTED"
    ]
}
}
}

```

## 7.4 Settlement Evidence Authority

The settlement observer **MUST NOT** trust settlement evidence supplied by the client application.

The following values **MUST** be independently obtained or independently cross-checked against the Polygon network:

```

Chain ID
Contract Address
Transaction Hash
Block Number
Settlement Event
Batch Commitment
Settlement Status

```

The production verification rule is:

```

RPC Chain ID = 137
AND
Expected Contract Code Exists
AND
Transaction Exists
AND
Transaction Targets Expected Contract
AND
Settlement Event Exists
AND
Batch Commitment Matches
AND
Settlement Allocation Is Valid

```

Only when the required conditions are satisfied may the observer transition the evidence to:

```

verificationState = VERIFIED

```

## 7.5 Dual-Transport Observation Boundary

Settlement and authorization infrastructure **MUST** support two independent transport paths:

```

PRIMARY:
WebSocket / gRPC Event Stream

```

```

SECONDARY:

```

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.  
Licensed under GNU AGPL-3.0 rules.

The event stream provides low-latency observation and cursor continuity.

JSON-RPC provides independent verification, historical recovery, and transport reconciliation.

A stream message **MUST NOT** by itself establish authoritative settlement.

Likewise, a stale RPC response **MUST NOT** be treated as current state.

The adapter **MUST** maintain a verification cursor containing sufficient information to detect gaps, replay, and stale observations.

Conceptually:

```
streamCursor
+
lastVerifiedBlock
+
lastVerifiedEvent
+
observedAt
+
transportHealth
```

If stream and RPC observations disagree, the adapter **MUST** enter a non-authoritative state and initiate reconciliation.

## 7.6 Schema-Level Fail-Closed Rule

Schema validation **MUST** be treated as a prerequisite rather than the final authorization decision.

The following states **MUST** be treated as non-authoritative:

```
UNKNOWN
STALE
UNVERIFIED
DISPUTED
```

Any downstream component requiring verified settlement **MUST** therefore evaluate:

```
SCHEMA VALID
AND
SETTLEMENT VERIFIED
AND
TRANSPORT STATE ACCEPTABLE
AND
CURSOR CONSISTENT
```

Failure of any condition **MUST** prevent authorization-dependent coordination from proceeding.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.

Licensed under GNU AGPL-3.0 rules.

The protocol MUST never convert an incomplete schema object, a stream-only observation, a stale RPC response, or a client-supplied settlement assertion into an authorization decision.

## 7.7 Edge Verification State Schema

The Edge Verification State object defines the minimum machine-readable state required by a Haven edge adapter before downstream coordination may be authorized.

The edge adapter MUST maintain verification state independently from the client application and MUST NOT derive authorization solely from client-supplied status.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://protocol.haven/schemas/edge-verification-
state.schema.json",
  "title": "Haven Edge Verification State",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "schemaVersion",
    "participantId",
    "networkId",
    "contractAddress",
    "streamState",
    "rpcState",
    "cursorState",
    "settlementState",
    "leaseState",
    "authorizationState",
    "evaluatedAt"
  ],
  "properties": {
    "schemaVersion": {
      "type": "string",
      "const": "2.0.0"
    },
    "participantId": {
      "type": "string",
      "minLength": 3,
      "maxLength": 128
    },
    "networkId": {
      "type": "string",
      "const": "polygon-mainnet"
    },
    "contractAddress": {
      "type": "string",
      "pattern": "^0x[a-fA-F0-9]{40}$"
    },
    "streamState": {
      "type": "string",
      "enum": [
        "HEALTHY",
        "DEGRADED"
      ]
    }
  }
}
```

```

        "DISCONNECTED",
        "RECONNECTING",
        "UNKNOWN"
    ]
},
"rpcState": {
    "type": "string",
    "enum": [
        "HEALTHY",
        "DEGRADED",
        "UNAVAILABLE",
        "UNKNOWN"
    ]
},
"cursorState": {
    "type": "string",
    "enum": [
        "CONSISTENT",
        "GAP_DETECTED",
        "BACKFILLING",
        "STALE",
        "UNKNOWN"
    ]
},
"settlementState": {
    "type": "string",
    "enum": [
        "UNVERIFIED",
        "OBSERVED",
        "VERIFIED",
        "DISPUTED",
        "REORGED",
        "UNKNOWN"
    ]
},
"leaseState": {
    "type": "string",
    "enum": [
        "VALID",
        "EXPIRING",
        "EXPIRED",
        "REVOKED",
        "UNKNOWN"
    ]
},
"authorizationState": {
    "type": "string",
    "enum": [
        "AUTHORIZED",
        "UNAUTHORIZED",
        "UNKNOWN"
    ]
},
"evaluatedAt": {
    "type": "string",
    "format": "date-time"
}

```

```
}  
}  
}
```

## 7.8 Authorization Evaluation Constraint

The schema above describes state; it does not itself grant authorization.

Authorization **MUST** require the simultaneous satisfaction of all mandatory verification conditions:

```
STREAM HEALTHY  
AND  
LEASE VALID  
AND  
STATE VERIFIED  
AND  
CURSOR CONSISTENT  
AND  
CONTRACT IDENTITY VALID  
AND  
SETTLEMENT STATE VALID
```

The resulting authorization decision **MUST** therefore be evaluated as a conjunction rather than as a single status flag.

Any of the following **MUST** result in:

```
authorizationState = UNAUTHORIZED  
UNKNOWN  
STALE  
UNVERIFIED  
DISCONNECTED  
GAP_DETECTED  
EXPIRED  
REVOKED  
DISPUTED
```

The edge adapter **MUST NOT** preserve a previous `AUTHORIZED` state after the authorization lease expires, the verification cursor becomes stale, or the transport layer loses the ability to establish current settlement state.

## 7.9 Stale-State Protection

Every verification state **MUST** be associated with an evaluation timestamp and a valid authorization lease.

A disconnected edge adapter **MUST NOT** continue operating indefinitely on its last known verified state.

The permitted sequence during transport loss is:

```
CONNECTED
  ↓
DISCONNECTED
  ↓
RECONNECTING
  ↓
BACKFILL / RECONCILIATION
  ↓
CURRENT STATE VERIFIED
  ↓
LEASE VALIDATED
  ↓
AUTHORIZED
```

If current state cannot be independently re-established, authorization **MUST** remain blocked.

This prevents an edge node from converting historical truth into present authorization during network partitions, RPC degradation, stream failure, or delayed event delivery.

## 7.10 Event-Stream Observation Envelope Schema

The primary low-latency observation channel **MUST** use a structured event-stream envelope for WebSocket or gRPC delivery.

The stream envelope is an **observation mechanism**, not an independent settlement authority.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://protocol.haven/schemas/event-stream-
observation.schema.json",
  "title": "Haven Event Stream Observation",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "schemaVersion",
    "transport",
    "networkId",
    "contractAddress",
    "eventName",
    "eventId",
    "blockNumber",
    "transactionHash",
    "cursor",
    "observedAt"
  ],
  "properties": {
    "schemaVersion": {
      "type": "string",
      "const": "2.0.0"
    },
    "transport": {
```

```

        "type": "string",
        "enum": [
            "WEBSOCKET",
            "GRPC"
        ]
    },
    "networkId": {
        "type": "string",
        "const": "polygon-mainnet"
    },
    "contractAddress": {
        "type": "string",
        "pattern": "^0x[a-fA-F0-9]{40}$"
    },
    "eventName": {
        "type": "string",
        "minLength": 1,
        "maxLength": 128
    },
    "eventId": {
        "type": "string",
        "minLength": 16,
        "maxLength": 128
    },
    "blockNumber": {
        "type": "integer",
        "minimum": 0
    },
    "transactionHash": {
        "type": "string",
        "pattern": "^0x[a-fA-F0-9]{64}$"
    },
    "cursor": {
        "$ref": "#/$defs/cursor"
    },
    "observedAt": {
        "type": "string",
        "format": "date-time"
    }
},
"$defs": {
    "cursor": {
        "type": "object",
        "additionalProperties": false,
        "required": [
            "blockNumber",
            "transactionHash",
            "eventIndex"
        ],
        "properties": {
            "blockNumber": {
                "type": "integer",
                "minimum": 0
            },
            "transactionHash": {
                "type": "string",

```



## 7.12 Cursor Continuity and Recovery

The verification cursor **MUST** provide deterministic continuity across event-stream reconnects.

When a cursor gap is detected, the adapter **MUST**:

1. Freeze authorization-dependent state.
2. Record the last verified cursor.
3. Reconnect to the primary event transport.
4. Request or derive the missing event range.
5. Backfill the missing range through the appropriate transport.
6. Cross-check recovered events through JSON-RPC.
7. Reconstruct settlement state.
8. Revalidate the authorization lease.
9. Resume authorization only after the complete state is verified.

A reconnect **MUST NOT** reset the cursor to the newest observed event and silently discard the missing range.

## 7.13 Schema Authority Boundary

All schemas in this section define machine-readable contracts for protocol communication.

They do not permit a client application to:

- disable the payment-routing enclosure;
- modify the protocol fee;
- substitute another settlement contract;
- declare a transaction settled;
- manufacture verification evidence;
- extend an expired authorization lease;
- bypass stream/RPC reconciliation; or
- convert an UNKNOWN, STALE, or UNVERIFIED condition into authorization.

The Zero-Trust Payment Routing Enclosure Engine remains the authoritative enforcement boundary.

Schema validity permits processing.

**It does not confer authority.**

## 7.14 RUNTIME CHAIN AND CONTRACT IDENTITY VALIDATION

The JSON Schema constrains the expected Polygon environment, but schema validation alone shall never establish blockchain identity or settlement authority.

For Haven production execution, the edge verification layer shall independently validate all chain and contract identity attributes against trusted runtime configuration.

The authoritative identity tuple is:

```
NETWORK_ID
+
CHAIN_ID = 137
+
EXPECTED_SPLITTER_CONTRACT_ADDRESS
+
DEPLOYED_BYTECODE_IDENTITY
+
VERIFIED_EVENT_INTERFACE
```

A client supplied `chainId`, `contractAddress`, `transactionHash`, or `settlement status` shall be treated only as an untrusted observation until independently verified.

### 7.14.1 Chain Identity Requirements

The verifier MUST independently establish:

```
RPC Chain ID = 137
AND
Configured Network = Polygon Mainnet
AND
Expected Contract Address = Runtime Configured Address
```

A mismatch in any identity attribute shall invalidate the observation.

The following conditions are therefore non-authoritative:

```
CHAIN_ID_MISMATCH
CONTRACT_ADDRESS_MISMATCH
NETWORK_ID_MISMATCH
UNKNOWN_CONTRACT
UNVERIFIED_BYTECODE
UNEXPECTED_EVENT_INTERFACE
```

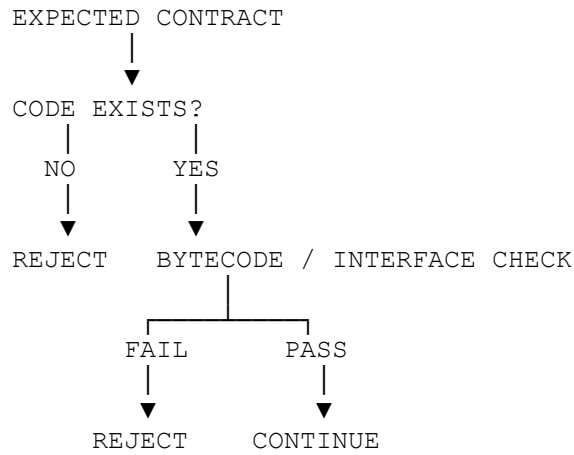
No authorization lease may be issued or renewed while any of these conditions remains unresolved.

### 7.14.2 Contract Identity Verification

The expected splitter address shall be held outside client-controlled checkout configuration.

The edge verifier shall confirm that executable code exists at the expected address before accepting settlement evidence.

Where the deployment configuration includes a verified bytecode identity or deployment commitment, the verifier shall additionally establish that the deployed contract corresponds to the approved production artifact.



A valid address without valid deployed code shall not be considered a valid settlement endpoint.

### 7.14.3 Transaction and Event Verification

A transaction hash shall not be accepted merely because it matches the syntactic format of an EVM transaction hash.

The verifier shall establish, as applicable:

```

TRANSACTION EXISTS
  AND
TRANSACTION TARGET = EXPECTED SPLITTER
  AND
RECEIPT EXISTS
  AND
EXPECTED EVENT SIGNATURE EXISTS
  AND
EVENT CORRELATION IS VALID
  AND
BLOCK POSITION IS CONSISTENT
  
```

For batched settlement, the verification target shall be the batch commitment and its associated settlement evidence rather than an individual checkout event being treated as a standalone on-chain payment.

### 7.14.4 Runtime Observation Authority

WebSocket/gRPC event streaming provides low-latency observation.

JSON-RPC provides independent state verification and recovery.

Neither transport shall independently confer authorization.

The production verification boundary is:

```
STREAM OBSERVATION
+
RPC CROSS-CHECK
+
CURSOR CONTINUITY
+
CONTRACT IDENTITY
+
SETTLEMENT STATE
+
LEASE VALIDITY
=
AUTHORIZATION ELIGIBILITY
```

If stream and RPC observations disagree, the verifier shall enter a reconciliation state and authorization shall remain frozen until the disagreement is resolved.

### 7.14.5 Fail-Closed Identity Rule

Any unresolved chain, contract, transaction, event, cursor, or settlement identity condition shall produce:

```
VERIFICATION = UNKNOWN
AUTHORIZATION = UNAUTHORIZED
```

The system shall never convert an unverified client assertion into authoritative settlement state.

Runtime identity verification therefore forms a mandatory security boundary between **machine-readable payload validity** and **permission to continue Haven coordination**.

## 8. CHECKOUT TRANSACTION SCHEMA & PAYMENT STATE MACHINE

### 8.1 Production Point-of-Sale Transaction Schema

The checkout transaction is the commercial transaction from which Haven event obligations are derived.

The PoS transaction object **MUST** establish commercial correlation and economic obligation without granting the client authority to declare settlement.

The production schema is:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://protocol.haven/schemas/pos-transaction.schema.json",
  "title": "Haven PoS Transaction",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "schemaVersion",
    "transactionId",
    "merchantId",
    "networkId",
    "currency",
    "subtotal",
    "havenFee",
    "total",
    "eventIds",
    "createdAt",
    "idempotencyKey",
    "economicObligation"
  ],
  "properties": {
    "schemaVersion": {
      "type": "string",
      "const": "2.0.0"
    },
    "transactionId": {
      "type": "string",
      "minLength": 8,
      "maxLength": 128
    },
    "merchantId": {
      "type": "string",
      "minLength": 3,
      "maxLength": 128
    },
    "networkId": {
      "type": "string",
      "minLength": 3,
      "maxLength": 128
    }
  }
}
```

```

},
"currency": {
  "type": "string",
  "pattern": "^[A-Z]{3}$"
},
"subtotal": {
  "type": "integer",
  "minimum": 0
},
"havenFee": {
  "type": "integer",
  "minimum": 0
},
"total": {
  "type": "integer",
  "minimum": 0
},
"eventIds": {
  "type": "array",
  "minItems": 1,
  "uniqueItems": true,
  "items": {
    "type": "string",
    "minLength": 8,
    "maxLength": 128
  }
},
"createdAt": {
  "type": "string",
  "format": "date-time"
},
"idempotencyKey": {
  "type": "string",
  "minLength": 16,
  "maxLength": 256
},
"economicObligation": {
  "type": "object",
  "additionalProperties": false,
  "required": [
    "status",
    "amount",
    "currency",
    "feePolicyVersion"
  ],
  "properties": {
    "status": {
      "type": "string",
      "enum": [
        "CREATED",
        "BATCH_ELIGIBLE",
        "SETTLED",
        "DISPUTED"
      ]
    },
    "amount": {

```

```

        "type": "integer",
        "minimum": 0
      },
      "currency": {
        "type": "string",
        "pattern": "^[A-Z]{3}$"
      },
      "feePolicyVersion": {
        "type": "string",
        "minLength": 1,
        "maxLength": 64
      }
    }
  }
}

```

The schema defines the transaction contract. It does **not** make the client the authority for fee legitimacy, settlement completion, or authorization.

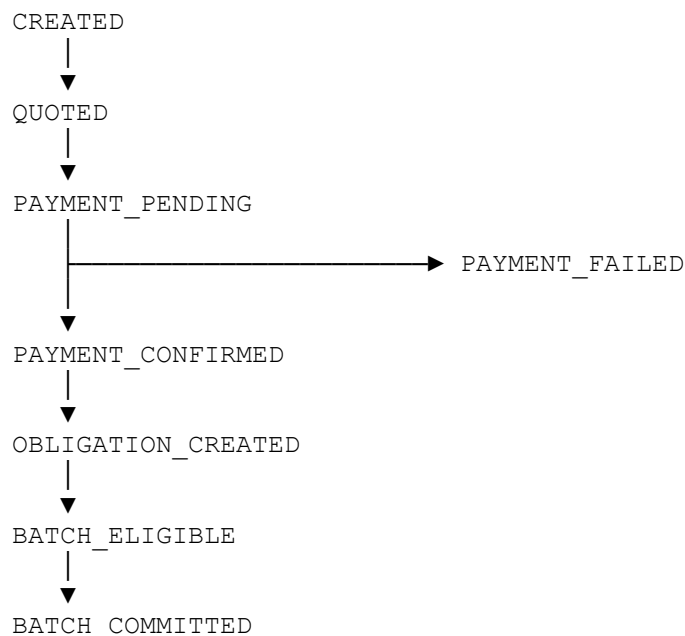
The authoritative fee policy remains outside the consumer-controlled presentation layer.

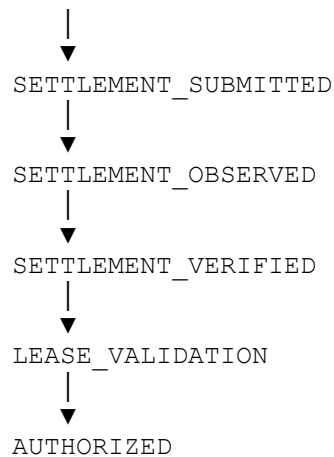
## 8.2 Payment and Settlement State Machine

The Haven payment lifecycle shall be modeled as an explicit state machine rather than as a single Boolean payment result.

A successful payment processor response shall never transition the Haven protocol directly to an authorized state.

The production lifecycle is:





### 8.2.1 State Definitions

#### **CREATED**

The commercial transaction has been instantiated but has not yet entered the payment process.

#### **QUOTED**

The applicable transaction amount and Haven economic obligation have been calculated according to the authoritative fee policy.

#### **PAYMENT\_PENDING**

The commercial payment has been initiated but has not received an authoritative processor result.

#### **PAYMENT\_CONFIRMED**

The payment processor has independently confirmed the commercial payment.

This state does **not** establish Haven protocol settlement.

#### **OBLIGATION\_CREATED**

The Haven economic obligation has been recorded against the corresponding billable event or events.

#### **BATCH\_ELIGIBLE**

The obligation is eligible for inclusion in a bounded settlement batch.

## **BATCH\_COMMITTED**

The eligible obligations have been cryptographically committed into the settlement batch and are no longer dependent on mutable checkout presentation data.

## **SETTLEMENT\_SUBMITTED**

The committed batch has been submitted to the Polygon settlement contract.

## **SETTLEMENT\_OBSERVED**

A corresponding settlement transaction or contract event has been observed through an approved event transport.

Observation alone does not establish settlement truth.

## **SETTLEMENT\_VERIFIED**

The settlement has passed independent verification against the approved contract, transaction, event, block, correlation, and chain identity requirements.

## **LEASE\_VALIDATION**

The authorization layer evaluates whether the authorization lease is valid, current, unrevoked, and bound to the verified settlement state.

## **AUTHORIZED**

Cross-network Haven coordination may proceed only after all mandatory authorization predicates are satisfied.

## **8.2.2 Non-Equivalent States**

The following states shall remain architecturally distinct:

PAYMENT\_CONFIRMED

≠

OBLIGATION\_SETTLED

OBLIGATION\_CREATED

≠

BATCH\_COMMITTED

BATCH\_COMMITTED

≠

SETTLEMENT\_VERIFIED

```

SETTLEMENT_OBSERVED
    ≠
SETTLEMENT_VERIFIED

SETTLEMENT_VERIFIED
    ≠
AUTHORIZATION_LEASE_VALID

STREAM_CONNECTED
    ≠
SETTLEMENT_TRUTH

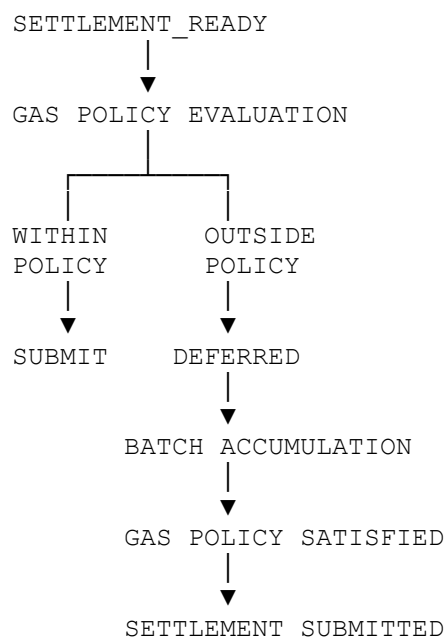
```

This separation prevents a payment processor response, client assertion, event-stream observation, or stale synchronization state from being promoted into authorization.

### 8.2.3 Gas-Aware Settlement Deferral

Polygon gas volatility shall affect settlement timing without invalidating the underlying Haven economic obligation.

Where the configured gas policy, volatility buffer, batch threshold, or network condition requires settlement to be temporarily withheld, the state shall transition to a deferred settlement condition rather than a payment failure.



A gas spike shall not:

- cancel an economic obligation;
- reduce an economic obligation;
- cause the PoS transaction to be recharged;

- permit the client to bypass the settlement mechanism; or
- cause the obligation to disappear from the settlement ledger.

The obligation remains authoritative until settled, reconciled, or placed into an explicitly governed exception state.

### 8.2.4 Event-Stream Observation Boundary

The primary WebSocket/gRPC event transport shall provide low-latency settlement observation.

JSON-RPC shall provide independent state verification and recovery.

Neither transport shall independently confer settlement authority.

The authorization layer shall require reconciliation between the available observation channels before accepting settlement as verified.

### 8.2.5 State Transition Integrity

Every state transition shall be associated with a valid correlation context and shall be monotonic with respect to the authoritative settlement lifecycle, except where an explicitly detected blockchain reorganization, dispute, recovery, or reconciliation process requires controlled rollback.

A client-controlled request shall not be permitted to submit an arbitrary target state.

The state engine shall evaluate:

```

CURRENT STATE
+
VALID TRANSITION
+
AUTHORIZED ACTOR
+
CORRELATION INTEGRITY
+
SETTLEMENT EVIDENCE
=
PERMITTED STATE TRANSITION

```

An invalid transition shall be rejected without mutating the authoritative state.

Page 45 therefore terminates at **Section 8.2.5**, with the next page continuing from **Section 8.2.6**.

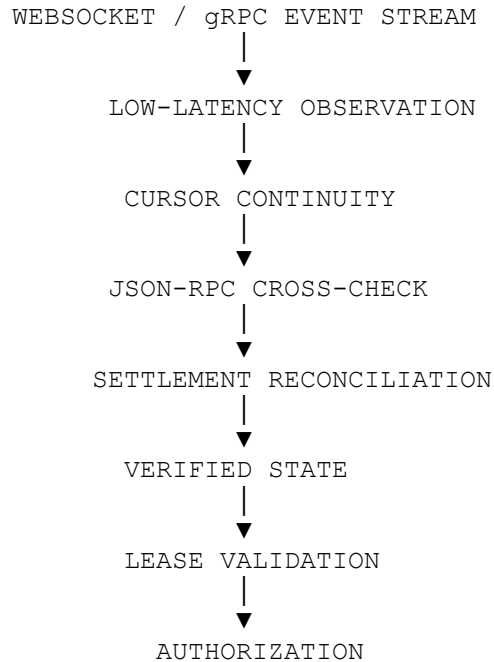
### 8.2.6 Event-Stream Failure, Lease Expiry and Recovery

The Haven authorization state shall remain fail-closed whenever settlement observation becomes unreliable, stale, disconnected, or inconsistent.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.  
Licensed under GNU AGPL-3.0 rules.

The event-stream transport is a low-latency observation mechanism. It shall not become an independent authority for settlement truth.

The production verification boundary is:



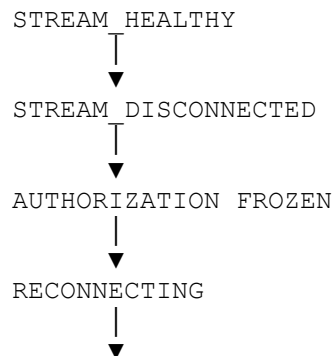
## Stream Disconnect

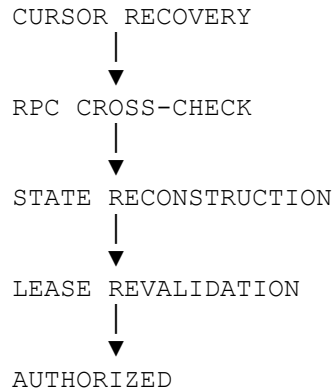
If the WebSocket or gRPC transport becomes disconnected, the system shall immediately mark the stream state as:

STREAM\_DISCONNECTED

Authorization shall not continue solely on the basis of the last previously observed stream state.

The system shall retain the **last verified cursor** and transition the affected authorization context into a non-authorized recovery state.





## Cursor Gap Detection

Every event-stream observation shall carry a cursor sufficient to establish ordering and continuity.

At minimum:

```
CURSOR =  
{  
    blockNumber,  
    transactionHash,  
    eventIndex  
}
```

A detected gap, discontinuity, duplicate sequence, unexpected block progression, or transaction mismatch shall produce:

CURSOR\_GAP\_DETECTED

The system shall then:

1. freeze authorization;
2. retain the last verified cursor;
3. reconnect or establish a healthy transport;
4. request the missing event range;
5. independently query Polygon through JSON-RPC;
6. reconcile the reconstructed event sequence;
7. verify the settlement transaction and contract identity;
8. reconstruct the authoritative settlement state; and
9. revalidate the authorization lease.

No authorization lease shall be considered valid merely because the stream has subsequently reconnected.

## RPC Unavailability

If independent JSON-RPC verification is unavailable, the event stream may continue to receive observations, but those observations shall remain non-authoritative for authorization.

The resulting state shall be:

```
STREAM = HEALTHY
RPC = UNAVAILABLE
VERIFICATION = UNKNOWN
AUTHORIZATION = UNAUTHORIZED
```

The system shall not infer verified settlement from stream availability alone.

## Lease Expiry

Authorization leases shall be time-bound.

If the lease expires before the required verification conditions are satisfied:

```
LEASE_VALID
  |
  ▼
LEASE_EXPIRED
  |
  ▼
AUTHORIZATION = UNAUTHORIZED
```

A previously authorized state shall not remain authorized merely because the underlying commercial transaction remains valid.

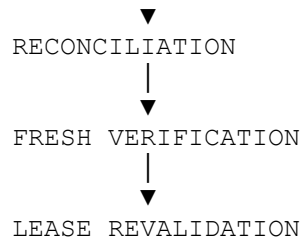
Lease renewal shall require a fresh evaluation of the underlying settlement evidence.

A client request shall never be permitted to extend, recreate, or self-authorize an expired lease.

## Reorganization and Disputed Settlement

If Polygon state indicates that a previously observed settlement has been reorganized, invalidated, or placed into a disputed verification state, authorization shall immediately be withdrawn or prevented from being issued.

```
SETTLEMENT_VERIFIED
  |
  ▼
REORG / DISPUTE DETECTED
  |
  ▼
AUTHORIZATION FROZEN
  |
```



The settlement lifecycle therefore remains authoritative only while its supporting evidence remains independently verifiable.

## Recovery Principle

The governing recovery rule is:

NO CURRENT VERIFICATION  
=  
NO CURRENT AUTHORIZATION

Historical success may be retained as audit evidence, but historical success shall not be treated as perpetual authorization.

The protocol shall prefer temporary interruption of coordination over continuation under uncertain settlement state.

### 8.2.7 Authorization State and Final Settlement Gate

The final Haven authorization decision shall be produced only after the payment, economic, settlement, transport, identity, cursor, and lease conditions have independently satisfied their required predicates.

Authorization shall therefore be evaluated as a conjunction rather than as a single payment flag.

PAYMENT\_CONFIRMED  
AND  
ECONOMIC\_OBLIGATION\_VALID  
AND  
BATCH\_COMMITMENT\_VALID  
AND  
SETTLEMENT\_VERIFIED  
AND  
STREAM\_STATE\_HEALTHY  
AND  
RPC\_STATE\_VALID  
AND  
CURSOR\_STATE\_CONSISTENT  
AND  
CONTRACT\_IDENTITY\_VALID  
AND  
AUTHORIZATION\_LEASE\_VALID

=  
AUTHORIZED

If any mandatory predicate evaluates to false, unknown, stale, disputed, or otherwise unresolved:

AUTHORIZATION = UNAUTHORIZED

## **Mandatory Authorization Conditions**

### **1. Payment Confirmation**

The originating commercial transaction must have a valid payment confirmation.

A payment processor callback that has not passed transaction validation shall not satisfy this condition.

### **2. Economic Obligation Validity**

The Haven economic obligation must correspond to a recognized billable event and valid fee policy.

The obligation must remain correlated to its originating transaction and idempotency context.

### **3. Batch Commitment Validity**

Where the obligation is settled through the bounded micro-batch mechanism, the relevant batch must possess a valid cryptographic commitment.

The commitment establishes the integrity of the obligation set presented for settlement.

### **4. Settlement Verification**

A settlement observation shall not be sufficient.

The settlement must be independently verified against:

CHAIN ID  
+  
CONTRACT ADDRESS  
+  
TRANSACTION HASH  
+  
BLOCK POSITION  
+  
EXPECTED EVENT INTERFACE  
+  
BATCH CORRELATION  
+  
SETTLEMENT STATE

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.  
Licensed under GNU AGPL-3.0 rules.

## 5. Transport Integrity

The event-stream transport must be in an acceptable operational state.

A connected transport with an unresolved cursor gap shall not satisfy authorization.

Similarly:

```
STREAM_CONNECTED  
  ≠  
STREAM_VERIFIED
```

## 6. RPC Verification

Independent JSON-RPC verification shall remain available whenever required by the verification policy.

A stream-only observation shall not be promoted to authoritative settlement during an RPC outage.

## 7. Cursor Consistency

The event sequence must be continuous and reconciled.

The following conditions shall prevent authorization:

```
GAP_DETECTED  
STALE  
BACKFILLING  
UNKNOWN
```

until the affected state has been completely reconciled.

## 8. Contract Identity

The observed settlement must correspond to the approved Haven splitter contract and Polygon network identity.

A client-supplied contract address shall not be trusted as authoritative evidence.

The verification boundary shall independently establish:

```
NETWORK IDENTITY  
+  
CHAIN ID = 137  
+  
EXPECTED CONTRACT IDENTITY  
+
```

DEPLOYED CODE / INTERFACE VALIDITY

## 9. Authorization Lease

The authorization lease must be:

VALID  
NOT EXPIRED  
NOT REVOKED  
BOUND TO VERIFIED STATE

An expired or revoked lease shall result in immediate denial.

## Fail-Closed Authorization Matrix

| Condition                                | Authorization |
|------------------------------------------|---------------|
| Payment confirmed, settlement unverified | UNAUTHORIZED  |
| Settlement observed, RPC unavailable     | UNAUTHORIZED  |
| Stream disconnected                      | UNAUTHORIZED  |
| Cursor gap detected                      | UNAUTHORIZED  |
| Settlement disputed                      | UNAUTHORIZED  |
| Settlement reorged                       | UNAUTHORIZED  |
| Contract identity unresolved             | UNAUTHORIZED  |
| Lease expired                            | UNAUTHORIZED  |
| Lease revoked                            | UNAUTHORIZED  |
| State stale                              | UNAUTHORIZED  |
| State unknown                            | UNAUTHORIZED  |
| All mandatory predicates valid           | AUTHORIZED    |

## Final Gate

The authorization engine shall implement the following invariant:

```
IF
    paymentConfirmed
    AND economicObligationValid
    AND batchCommitmentValid
    AND settlementVerified
    AND streamHealthy
    AND rpcVerified
    AND cursorConsistent
    AND contractIdentityValid
    AND leaseValid
THEN
    AUTHORIZED
ELSE
```

UNAUTHORIZED

No client application, PoS frontend, payment processor, event publisher, or integration developer shall be permitted to override this gate.

The authorization result is therefore an output of the Haven verification architecture, not a value supplied by the transaction originator.

## State-Machine Completion Principle

The complete lifecycle is:

```
COMMERCIAL PAYMENT
  ↓
ECONOMIC OBLIGATION
  ↓
BATCH ELIGIBILITY
  ↓
BATCH COMMITMENT
  ↓
ON-CHAIN SETTLEMENT
  ↓
EVENT OBSERVATION
  ↓
INDEPENDENT VERIFICATION
  ↓
CURSOR RECONCILIATION
  ↓
CONTRACT IDENTITY VALIDATION
  ↓
LEASE VALIDATION
  ↓
AUTHORIZATION
```

This establishes the final separation required by the Haven Zero-Trust Payment Routing Enclosure Engine:

```
PAYMENT
≠
SETTLEMENT
≠
OBSERVATION
≠
VERIFICATION
≠
AUTHORIZATION
```

Each boundary must independently succeed before the next authority-bearing state can be entered.

## 9. ON-CHAIN SPLITTER ENGINE

### 9.1 Contract Design Objective

The On-Chain Splitter Engine is the economic enforcement primitive of the Haven architecture.

Its purpose is to receive protocol-designated funds and deterministically allocate them to predefined destinations without relying on client application logic.

The contract must establish:

- immutable routing destinations;
- deterministic allocation;
- authorized network synchronization;
- explicit chain assumptions;
- event emission;
- rejection of unauthorized state mutation;
- protection against accidental direct token transfers where applicable;
- controlled handling of native Polygon assets; and
- transparent on-chain accounting.

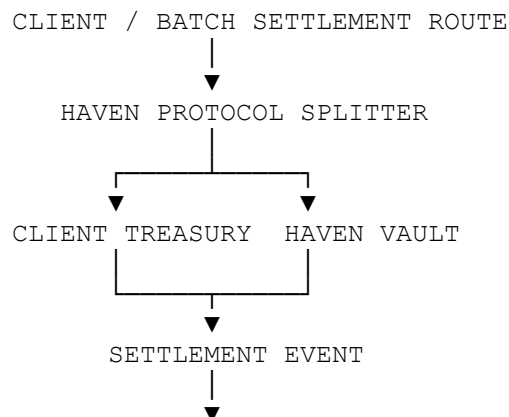
## 9. ON-CHAIN SPLITTER ENGINE

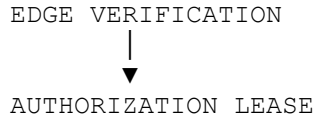
### 9.2 Critical EVM Design Correction

The Haven splitter is the **on-chain settlement enforcement boundary** for protocol obligations that have been committed for Polygon settlement.

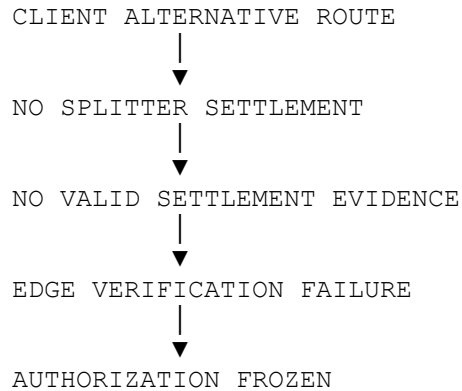
The splitter does not attempt to determine whether a client independently transferred value outside the splitter. EVM execution cannot establish a negative fact about transactions that never entered the contract. Accordingly, Haven enforcement is divided into two distinct controls:

#### ON-CHAIN SETTLEMENT CONTROL





## OFF-CONTRACT BYPASS DETECTION



The two controls must not be conflated.

A successful payment processor response is not Polygon settlement evidence.

A WebSocket or gRPC event is not independently sufficient settlement proof.

A JSON-RPC observation is not sufficient if contract identity, transaction inclusion, event integrity, or cursor continuity cannot be independently established.

The production settlement condition is therefore:

VALID ECONOMIC OBLIGATION  
AND  
VALID BATCH COMMITMENT  
AND  
VALID SPLITTER EXECUTION  
AND  
INDEPENDENT SETTLEMENT VERIFICATION  
AND  
VALID CONTRACT IDENTITY  
AND  
CONSISTENT OBSERVATION CURSOR

Only after these conditions are satisfied may the off-chain Edge Verification System evaluate authorization.

## 9.3 Production Solidity Reference Implementation

The following implementation defines the production settlement boundary for the native Polygon-value routing model.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.  
Licensed under GNU AGPL-3.0 rules.

The contract is intentionally **non-upgradeable**.

No proxy administrator, delegatecall upgrade path, client-controlled destination mutation, or post-deployment fee-ratio mutation is permitted.

The economic allocation is permanently constrained to:

```
CLIENT = 95%
HAVEN   = 5%
TOTAL   = 100%
```

The contract uses:

- immutable destination addresses;
- immutable synchronization authority;
- packed immutable basis-point configuration;
- custom errors instead of revert strings;
- zero-value guards;
- explicit split conservation checks;
- deterministic settlement events;
- authorization-gated network synchronization;
- no PHI;
- no reversible patient identifier;
- no client-side mechanism for disabling protocol routing.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

/**
 * @title HavenProtocolSplitter
 * @notice Immutable native-asset settlement splitter for Haven Protocol.
 *
 * @dev
 * Production invariants:
 *
 * 1. clientTreasury is immutable.
 * 2. havenVault is immutable.
 * 3. synchronizationAuthority is immutable.
 * 4. Client allocation is permanently constrained to 9,500 BPS.
 * 5. Haven allocation is permanently constrained to 500 BPS.
 * 6. Every accepted settlement must have non-zero value.
 * 7. The two destination allocations must conserve msg.value exactly.
 * 8. No PHI is accepted or emitted.
 * 9. No upgrade or proxy mechanism exists.
 * 10. Synchronization state cannot authorize payment settlement.
 *
 * The contract proves only settlement that entered this contract.
 * It cannot prove that an external payment route did not occur.
 */
```

```

contract HavenProtocolSplitter {
    uint16 public constant CLIENT_BPS = 9_500;
    uint16 public constant HAVEN_BPS = 500;
    uint16 public constant BPS_DENOMINATOR = 10_000;

    address payable public immutable clientTreasury;
    address payable public immutable havenVault;
    address public immutable synchronizationAuthority;

    mapping(address => bool) public isNetworkSynchronized;

    error InvalidAddress();
    error InvalidValue();
    error Unauthorized();
    error InvalidNetwork();
    error TransferFailed();
    error InvalidConfiguration();

    event FeeReceived(
        address indexed payer,
        uint256 amount,
        uint256 clientAmount,
        uint256 havenAmount
    );

    event NetworkSynchronizationUpdated(
        address indexed network,
        bool synchronizedState
    );

    constructor(
        address payable _clientTreasury,
        address payable _havenVault,
        address _synchronizationAuthority
    ) {
        if (
            _clientTreasury == address(0) ||
            _havenVault == address(0) ||
            _synchronizationAuthority == address(0)
        ) {
            revert InvalidAddress();
        }

        if (_clientTreasury == _havenVault) {
            revert InvalidConfiguration();
        }

        clientTreasury = _clientTreasury;
        havenVault = _havenVault;
        synchronizationAuthority = _synchronizationAuthority;
    }

    /**
     * @notice Routes a native-asset settlement according to the immutable
     *         95/5 protocol allocation.
     */

```

```

* @dev
* The caller cannot provide alternative destinations or basis points.
* This prevents client integrations from substituting a different route.
*/
function settle()
    external
    payable
{
    uint256 amount = msg.value;

    if (amount == 0) {
        revert InvalidValue();
    }

    uint256 havenAmount;
    uint256 clientAmount;

    unchecked {
        havenAmount = (amount * HAVEN_BPS) / BPS_DENOMINATOR;
        clientAmount = amount - havenAmount;
    }

    if (clientAmount == 0 || havenAmount == 0) {
        revert InvalidConfiguration();
    }

    _sendValue(clientTreasury, clientAmount);
    _sendValue(havenVault, havenAmount);

    emit FeeReceived(
        msg.sender,
        amount,
        clientAmount,
        havenAmount
    );
}

/**
* @notice Records verified network synchronization state.
*
* @dev
* Synchronization is a coordination signal only.
* It does not modify the settlement route and cannot change the
* 95/5 allocation.
*/
function setNetworkSynchronization(
    address network,
    bool synchronizedState
)
    external
{
    if (msg.sender != synchronizationAuthority) {
        revert Unauthorized();
    }

    if (network == address(0)) {

```

```

        revert InvalidNetwork();
    }

    isNetworkSynchronized[network] = synchronizedState;

    emit NetworkSynchronizationUpdated(
        network,
        synchronizedState
    );
}

function _sendValue(
    address payable destination,
    uint256 amount
)
    internal
{
    bool success;

    assembly {
        success := call(
            gas(),
            destination,
            amount,
            0,
            0,
            0,
            0
        )
    }

    if (!success) {
        revert TransferFailed();
    }
}

receive() external payable {
    revert InvalidConfiguration();
}

fallback() external payable {
    revert InvalidConfiguration();
}
}

```

The splitter therefore exposes **one authoritative settlement route**.

It does not expose setter functions for treasury destination, Haven destination, fee ratio, synchronization authority, or upgrade implementation.

Any requirement to alter those values constitutes a deployment-level configuration change and requires deployment of a new contract identity rather than mutation of an existing production settlement boundary.

### 9.3.1 Splitter Execution Invariants

The production splitter must preserve the following invariants for every successful settlement:

```
msg.value > 0  
  
AND  
  
clientAmount > 0  
  
AND  
  
havenAmount > 0  
  
AND  
  
clientAmount + havenAmount = msg.value  
  
AND  
  
clientAmount = msg.value - havenAmount  
  
AND  
  
havenAmount = floor(msg.value × 500 / 10,000)
```

Therefore:

```
HAVEN = 5%  
CLIENT = 95%
```

The subtraction-based client calculation deliberately derives the client allocation from the original settlement amount after Haven allocation rather than independently rounding both allocations.

This ensures that:

```
CLIENT + HAVEN = ORIGINAL SETTLEMENT VALUE
```

with no residual amount remaining in the contract.

The production implementation must never accept caller-supplied:

- destination addresses;
- basis-point ratios;
- Haven fee percentage;
- treasury percentage;
- synchronization state;
- replacement splitter address.

Those values are outside the caller's authority boundary.

### 9.3.2 Gas Optimization and Settlement Safety

The splitter is deliberately optimized for the settlement operation itself.

Production implementation requirements include:

1. **Custom errors**

Revert conditions use custom errors rather than dynamically allocated revert strings.

2. **Immutable configuration**

Settlement destinations and synchronization authority are immutable, eliminating repeated storage reads and preventing post-deployment mutation.

3. **Minimal storage mutation**

The settlement path does not write mutable state for every payment.

This is intentional.

Haven does not require an on-chain payment ledger for every commercial event because economic obligation accumulation occurs off-chain.

4. **Event-based settlement evidence**

Successful settlement emits the minimum required non-PHI evidence:

```
payer  
amount  
clientAmount  
havenAmount
```

5. **No PHI**

Patient names, identifiers, diagnoses, clinical records, encounter content, provider notes, or other PHI must never enter contract calldata or event logs.

6. **No patient-level reversible identifier**

If an external correlation is required, it remains an off-chain identifier or cryptographic correlation mechanism governed by the Haven privacy boundary.

## 7. No upgrade proxy

The contract contains no `delegatecall` upgrade architecture.

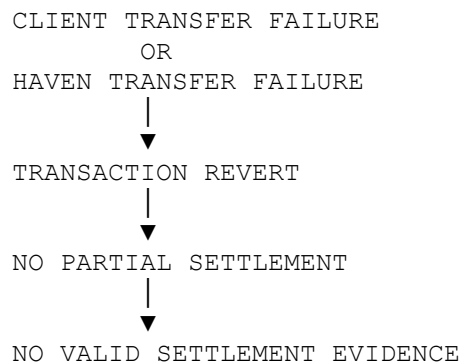
## 8. Assembly value transfer

The internal native-value transfer path may use a constrained low-level `call` to avoid unnecessary abstraction overhead while explicitly checking the returned success flag.

### 9.3.3 Settlement Failure Semantics

Settlement is atomic.

If either destination transfer fails:



The Edge Verification System must therefore never treat a partially observed execution attempt as a valid settlement.

A reverted transaction is not a successful protocol settlement.

A submitted transaction is not a successful protocol settlement.

A mempool observation is not a successful protocol settlement.

A payment processor confirmation is not a successful protocol settlement.

Only independently verified successful contract execution can transition the settlement state to:

SETTLEMENT\_VERIFIED

### 9.3.4 Synchronization Authority Boundary

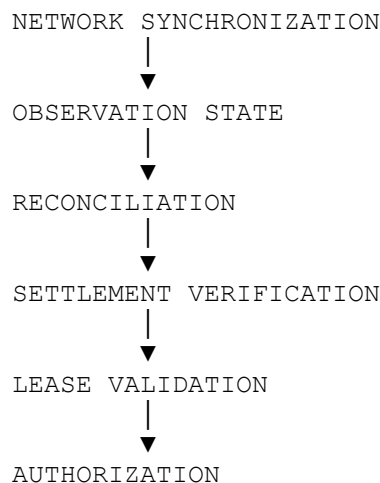
`setNetworkSynchronization()` exists only to publish a coordination state associated with an authorized network identity.

It does **not**:

- modify settlement destinations;
- modify the 95/5 split;
- authorize an unpaid obligation;
- mark a failed transaction as settled;
- manufacture settlement evidence;
- extend an authorization lease;
- override Edge Verification;
- disable the payment-routing enclosure.

The synchronization state must therefore remain subordinate to the independent settlement-verification pipeline.

The production authorization relationship is:



No synchronization flag can skip a preceding stage.

### 9.3.5 On-Chain / Off-Chain Responsibility Boundary

The production architecture intentionally separates responsibilities:

| Layer            | Authoritative Responsibility                     |
|------------------|--------------------------------------------------|
| PoS / Client     | Commercial transaction initiation                |
| Haven Edge       | Event classification and protocol enforcement    |
| Off-Chain Ledger | Economic obligation and micro-batch accounting   |
| Batch Engine     | Bounded aggregation and cryptographic commitment |
| Polygon Splitter | Immutable 95/5 value routing                     |
| WebSocket/gRPC   | Low-latency settlement observation               |
| JSON-RPC         | Independent verification and recovery            |

| <b>Layer</b>        | <b>Authoritative Responsibility</b> |
|---------------------|-------------------------------------|
| Cursor Manager      | Ordering and continuity             |
| Verification Engine | Settlement truth                    |
| Lease Engine        | Time-bounded authorization          |
| Clinical Systems    | PHI and clinical records            |

This boundary is mandatory.

The Polygon contract must not become a clinical data store.

The clinical system must not become the settlement authority.

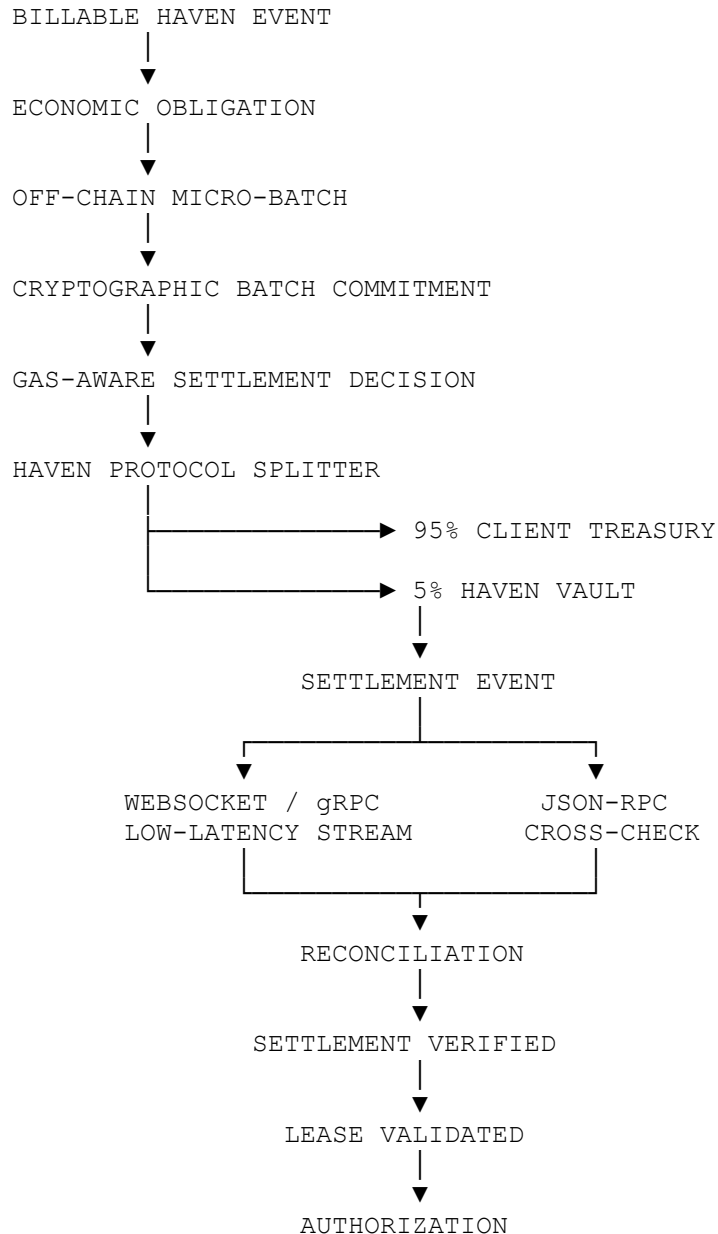
The PoS frontend must not become the fee authority.

The event stream must not become the authorization authority.

The client must not become the payment-routing authority.

### 9.3.6 Production Settlement Principle

The complete settlement path is:



The central production rule is:

**Economic obligations may accumulate off-chain, but settlement authority cannot be manufactured off-chain.**

Only the immutable splitter execution, independently verified through the dual-transport verification architecture, can establish the settlement evidence required by the Edge Authorization System.

Any unresolved settlement state remains non-authoritative and therefore cannot satisfy the final authorization gate.

## 9.4 Split Configuration

For the stated reference economic model:

|                            |      |
|----------------------------|------|
| Client Corporate Treasury: | 95%  |
| Haven Secure Vault:        | 5%   |
| Total:                     | 100% |

The corresponding basis-point parameters are:

|                        |                     |
|------------------------|---------------------|
| <code>clientBps</code> | <code>= 9500</code> |
| <code>havenBps</code>  | <code>= 500</code>  |

The values must be fixed at deployment for an immutable splitter.

A production deployment should never allow a participating client to modify these values after deployment.

## 9.5 Important Atomicity Property

The splitter executes both destination transfers within the same transaction.

If either transfer fails, the entire transaction reverts.

This creates an atomic routing condition:

**Either the complete split succeeds, or the complete split fails.**

The contract therefore avoids a partially completed economic distribution in which one destination receives funds while the other does not.

The edge layer must treat a reverted splitter transaction as unsuccessful settlement.

## 9.6 Synchronization Authority Separation

The synchronization authority is deliberately separated from the consumer checkout.

It must not be:

- a browser wallet;
- a frontend user;
- a client-side JavaScript variable;
- a checkout administrator;
- a mutable environment variable; or
- an arbitrary PoS terminal.

The authority must be explicitly governed by the Haven deployment's trust model.

For stronger production deployments, synchronization transitions should be derived from independently verifiable settlement conditions or controlled by a dedicated protocol authorization mechanism rather than manually toggled by an operator.

## 10. ON-CHAIN PAYMENT ROUTING SPECIFICATION

### 10.1 Payment Endpoint Binding

The Haven payment endpoint is a protocol-controlled Polygon contract address.

The production deployment shall establish the splitter address as a **deployment constant** across all Haven edge configurations.

A client integration must not be permitted to submit an arbitrary destination address as proof of Haven settlement.

The conceptual configuration is:

```
HAVEN_SPLITTER_ADDRESS
├── PoS Integration
├── Payment Orchestrator
├── Settlement Observer
└── Edge Enforcement Adapter
```

All four components must independently know the expected contract address.

The address must be distributed through a controlled configuration mechanism and integrity-protected.

It must not be accepted from an ordinary checkout request.

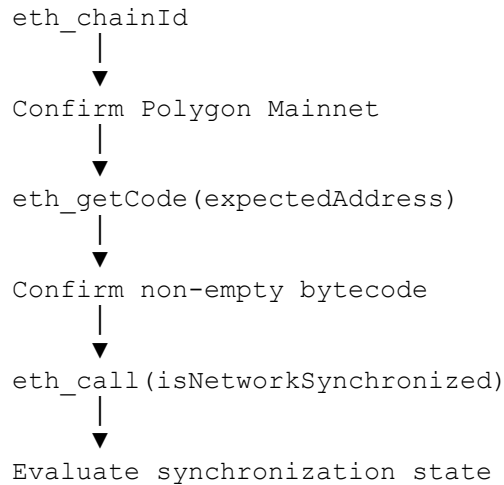
### 10.2 Contract Identity Verification

An adapter must verify more than the textual contract address.

Before treating a response as authoritative, the adapter should establish:

```
Expected Chain ID
      +
Expected Contract Address
      +
Expected Contract Code
      +
Expected Contract Interface
      +
Expected State
```

The verification sequence is:



A client-provided RPC endpoint may be used for transport only if the response is independently validated.

The response itself must never be treated as trustworthy merely because it came from an endpoint supplied by the client.

## 10.3 JSON-RPC Verification

The edge adapter periodically executes an `eth_call` against the splitter contract.

Conceptually:

```
{
  "jsonrpc": "2.0",
  "method": "eth_call",
  "params": [
    {
      "to": "EXPECTED_SPLITTER_ADDRESS",
      "data": "ENCODED_IS_NETWORK_SYNCHRONIZED_CALL"
    },
    "latest"
  ],
  "id": 1
}
```

The adapter decodes the returned ABI-encoded Boolean.

The expected result is:

```
true
```

Any other result must be handled according to the fail-closed enforcement policy.

## 10.4 Verification Interval

The reference synchronization polling interval is:

**60 seconds**

This means the adapter must establish synchronization at least once during each configured verification cycle.

The 60-second value should not be misunderstood as a guarantee that unauthorized traffic can never occur for less than 60 seconds.

It establishes a maximum intended verification interval under normal operation.

A production implementation should additionally support event-driven invalidation and immediate local enforcement where stronger guarantees are required.

## 10.5 Polling Algorithm

Reference logic:

EVERY 60 SECONDS:

```
Verify configured chain ID

IF chain ID != expected:
    ENTER FAIL-CLOSED STATE

Verify expected contract code

IF code absent or unexpected:
    ENTER FAIL-CLOSED STATE

Query isNetworkSynchronized(networkAddress)

IF result == true:
    maintain synchronized state

ELSE:
    engage circuit breaker
```

The adapter must not cache `true` indefinitely.

A previously valid state must expire unless renewed.

## 10.6 Stale-State Protection

The adapter should maintain a local synchronization lease.

Conceptually:

```
lastVerifiedAt = timestamp of successful verification
```

```
if currentTime - lastVerifiedAt > verificationTTL:  
    synchronized = false
```

A successful verification therefore establishes a bounded authorization period rather than a permanent local permission.

This prevents an adapter from remaining operational indefinitely after losing connectivity to the authoritative settlement layer.

## 11. CIRCUIT-BREAKER ENGINE

### 11.1 Purpose

The circuit breaker is the final local enforcement mechanism between a participating enterprise's systems and the Haven coordination network.

Its purpose is to ensure that inability to establish valid protocol synchronization results in **denial of Haven-governed coordination**, rather than silent continuation.

The circuit breaker does not destroy client data.

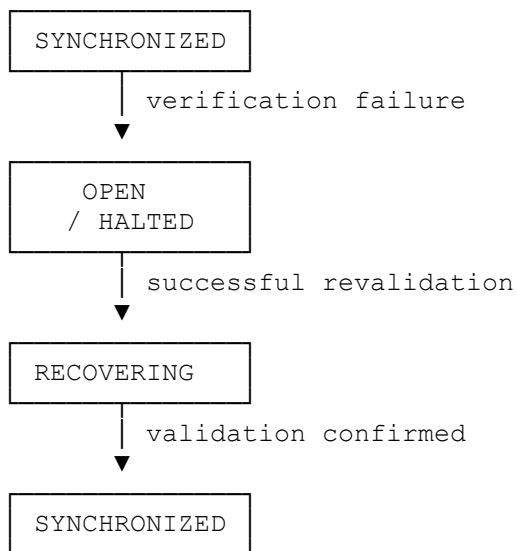
It does not delete clinical records.

It does not shut down the entire healthcare enterprise.

It isolates the Haven-governed coordination pathway.

### 11.2 Circuit States

The adapter shall maintain at least three logical states:



An implementation may use additional states such as **DEGRADED**, **UNKNOWN**, or **RECONCILING**, provided that none of them accidentally grant unauthorized network access.

## 11.3 Fail-Closed Rule

The central decision function is:

```
ALLOW =  
    chainValid  
    AND contractValid  
    AND synchronizationValid  
    AND synchronizationFresh  
    AND adapterHealthy
```

Therefore:

```
IF ANY CONDITION == FALSE  
    ALLOW = FALSE
```

The adapter must not implement:

```
IF verificationFails  
    assumePreviousState();
```

unless an explicitly approved emergency policy permits temporary degradation.

The Haven baseline architecture is fail-closed.

## 11.4 Protected Operations

When the circuit breaker is open, the adapter must prevent Haven-governed operations including:

- cross-network event transmission;
- new coordination-state publication;
- Haven-mediated workflow progression;
- outbound event translation;
- inbound event acceptance where synchronization is a prerequisite;
- protocol acknowledgements that imply successful coordination; and
- creation of new network-authorized coordination state.

Local enterprise systems may continue operating according to their own independent business-continuity procedures.

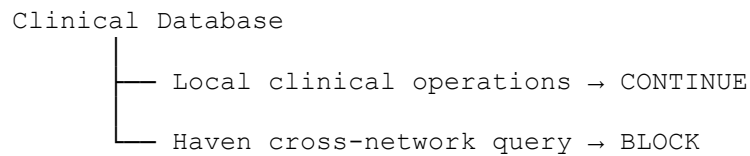
## 11.5 Database Query Boundary

The requirement to freeze "database queries" must be implemented carefully.

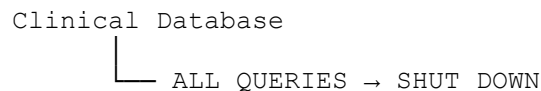
The Haven adapter should not indiscriminately terminate every query against the client's clinical database.

Instead, it must block the **Haven-authorized cross-network query path**.

Correct:



Incorrect:



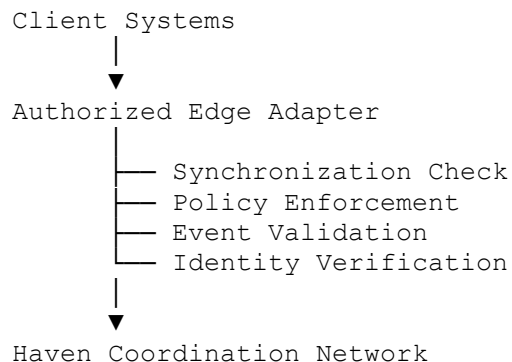
This distinction prevents a payment synchronization failure from unnecessarily becoming a clinical-system availability incident.

## 11.6 Bypass Detection

A client may attempt to circumvent the edge adapter by connecting directly to Haven services.

The production network architecture must therefore require that Haven coordination endpoints accept traffic only through authenticated and authorized edge pathways.

Conceptually:



A direct client-to-Haven connection that bypasses the enforcement adapter must be rejected at the network authorization layer.

Otherwise, the local circuit breaker would be operationally irrelevant.

## 11.7 Enforcement Principle

The architecture therefore has two complementary controls:

## **On-Chain Control**

The splitter determines how funds entering the protocol settlement endpoint are distributed.

## **Off-Chain Enforcement Control**

The edge adapter determines whether a participating network may continue Haven-governed coordination based on independently verified settlement synchronization.

Neither mechanism should be misrepresented as capable of doing the other's job.

The smart contract cannot observe a payment that never reaches it.

The edge adapter cannot move funds on-chain merely by observing a payment.

The security of the Haven model emerges from their composition:

**On-Chain Settlement + Independent Verification + Fail-Closed Edge Enforcement.**

## 12. FEE ROUTING CONTRACT INTERFACE

### 12.1 Contract Interface Definition

The Haven PoS integration layer must interact with the splitter through a narrowly defined contract interface.

The client integration should not depend upon implementation-specific contract internals beyond the functions and events required for settlement verification.

The canonical interface is:

```
interface IHavenProtocolSplitter {  
  
    function isNetworkSynchronized(  
        address network  
    ) external view returns (bool);  
  
    function clientTreasury()  
        external  
        view  
        returns (address);  
  
    function havenVault()  
        external  
        view  
        returns (address);  
  
    function clientBps()  
        external  
        view  
        returns (uint16);  
  
    function havenBps()  
        external  
        view  
        returns (uint16);  
}
```

The interface establishes the minimum state required by an external verifier.

The adapter should independently verify that the deployed bytecode exposes the expected ABI behavior.

## 12.2 Deployment Configuration

The production deployment manifest should contain immutable operational references such as:

```
{
  "network": "polygon",
  "chainId": 137,
  "splitterAddress": "0xEXPECTED_SPLITTER_ADDRESS",
  "clientTreasury": "0xEXPECTED_TREASURY_ADDRESS",
  "havenVault": "0xEXPECTED_VAULT_ADDRESS",
  "clientBps": 9500,
  "havenBps": 500,
  "verificationIntervalSeconds": 60
}
```

The example addresses are intentionally placeholders.

A real deployment must substitute the addresses produced by the controlled deployment ceremony.

The configuration must be cryptographically integrity-protected and distributed through a controlled configuration-management process.

## 12.3 Configuration Integrity

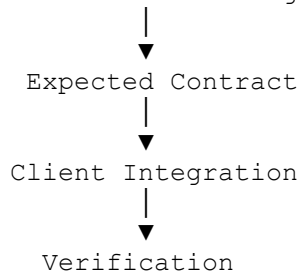
A client must not be able to modify the authoritative contract destination through an ordinary PoS configuration screen.

The following must therefore be rejected as authoritative input:

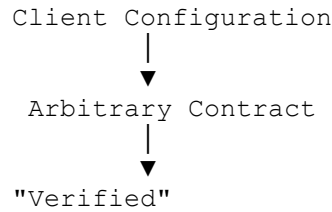
```
{
  "splitterAddress": "CLIENT_SELECTED_ADDRESS"
}
```

The correct architecture is:

Controlled Haven Configuration



rather than:



## 12.4 Event Verification

The splitter emits:

```
event FeeReceived(  
    address indexed payer,  
    uint256 amount,  
    uint256 clientAmount,  
    uint256 havenAmount  
);
```

The settlement observer should consume this event as settlement evidence.

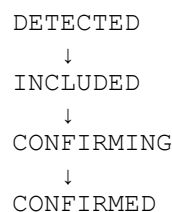
A production observer must verify:

1. the transaction was included on the expected chain;
2. the transaction targeted the expected splitter;
3. the transaction was not reverted;
4. the event originated from the expected contract;
5. the event parameters are ABI-valid;
6. the distributed amounts correspond to the expected split;
7. sufficient confirmation has been reached under the deployment's finality policy; and
8. the event can be correlated to the corresponding Haven transaction.

## 12.5 Confirmation Policy

A transaction appearing in an RPC response is not necessarily equivalent to final operational settlement.

The observer should maintain:



The required confirmation depth should be defined operationally according to the Polygon deployment's risk tolerance and the value of the transaction.

The edge adapter must not release sensitive cross-network workflow transitions merely because an unconfirmed transaction hash has been observed.

## 13. TOKENOMICS ACCOUNTING & RECONCILIATION

### 13.1 Economic Ledger Separation

Haven requires three separate accounting layers:

#### Layer 1 — Commercial Ledger

Records the consumer-facing transaction.

Transaction ID  
Merchant ID  
Subtotal  
Haven Fee  
Total  
Payment Status

#### Layer 2 — Protocol Event Ledger

Records the Haven coordination obligation.

Event ID  
Network ID  
Event Type  
Fee Requirement  
Correlation ID  
Event Status

#### Layer 3 — Blockchain Settlement Ledger

Records the on-chain settlement evidence.

Transaction Hash  
Contract Address  
Block Number  
Amount  
Destination Allocations  
Confirmation Status

These layers must be correlated without being collapsed into a single database object.

### 13.2 Reconciliation Key

A production implementation should establish a deterministic reconciliation key.

Conceptually:

```
RECONCILIATION KEY =  
    HASH(  
        transactionId ||  
        eventId ||  
        networkId ||  
        paymentReference  
    )
```

The key allows the settlement observer to associate a blockchain transaction with the commercial and protocol records.

The exact hashing and canonicalization specification must be frozen before production deployment.

### 13.3 Reconciliation States

The protocol should support at least:

```
MATCHED  
PENDING_CHAIN  
MISSING_CHAIN_SETTLEMENT  
AMOUNT_MISMATCH  
DESTINATION_MISMATCH  
CONTRACT_MISMATCH  
DUPLICATE  
REORGED  
MANUAL_REVIEW  
RESOLVED
```

A reconciliation mismatch must not automatically be overwritten by the latest client status.

The blockchain observation and the commercial transaction record must remain separately auditable.

### 13.4 Amount Reconciliation

For every confirmed splitter transaction:

```
Received Amount  
=  
Client Treasury Allocation  
+  
Haven Vault Allocation
```

For a 95/5 configuration:

```
Client Allocation = Received × 0.95  
Haven Allocation  = Received × 0.05
```

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.  
Licensed under GNU AGPL-3.0 rules.

The implementation should avoid floating-point arithmetic.

On-chain allocation must use integer arithmetic and explicit basis points.

For example:

```
uint256 clientAmount =  
    (amount * clientBps) / 10_000;  
  
uint256 havenAmount =  
    amount - clientAmount;
```

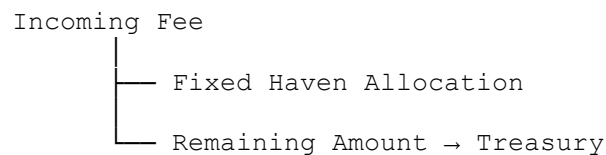
Using the remainder for the second allocation ensures that integer division does not create an unallocated remainder.

### 13.5 Fixed-Fee Alternative

The architecture may also support a fixed-fee allocation model where the Haven allocation is defined as a fixed amount rather than a percentage.

This must be treated as a distinct contract configuration.

For example:



The contract must explicitly define what happens when the incoming amount is smaller than the required fixed allocation.

The safe behavior is to reject the transaction rather than silently underfund the Haven destination.

### 13.6 No Silent Rounding

The checkout layer, payment processor, and smart contract must use explicitly defined monetary units.

For fiat:

```
NGN -> minor currency unit  
USD -> cents
```

For EVM native assets:

POL → wei

The system must never use binary floating-point values for settlement-critical monetary calculations.

Incorrect:

```
const fee = 0.05;
```

Preferred conceptual model:

```
const feeMinorUnits = 5;
```

where the monetary precision has been explicitly defined by the applicable currency model.

## 13.7 Treasury Reconciliation

The corporate treasury must reconcile:

```
Expected Treasury Allocation
    |
    ▼
Blockchain Observed Allocation
    |
    ▼
Treasury Wallet Balance / Transaction History
```

The Haven vault follows the same process:

```
Expected Haven Allocation
    |
    ▼
Blockchain Observed Allocation
    |
    ▼
Vault Transaction History
```

The reconciliation engine must flag discrepancies rather than silently correcting them.

## 13.8 Accounting Immutability

Historical settlement records must not be rewritten to accommodate a later client-side configuration change.

If an error is discovered, the system should append a correction or reconciliation record.

It should not mutate the original evidence.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.  
Licensed under GNU AGPL-3.0 rules.

This establishes an append-oriented forensic record:

ORIGINAL EVENT  
+  
SETTLEMENT EVIDENCE  
+  
RECONCILIATION RESULT  
+  
CORRECTION / RESOLUTION

The result is a traceable economic history suitable for operational review, financial reconciliation, and technical forensic analysis.

## 14. EDGE ADAPTER IMPLEMENTATION

### 14.1 Adapter Responsibility

The Haven Edge Adapter is the enforcement component installed within or adjacent to the participating enterprise's local integration environment.

Its function is not simply to transform data.

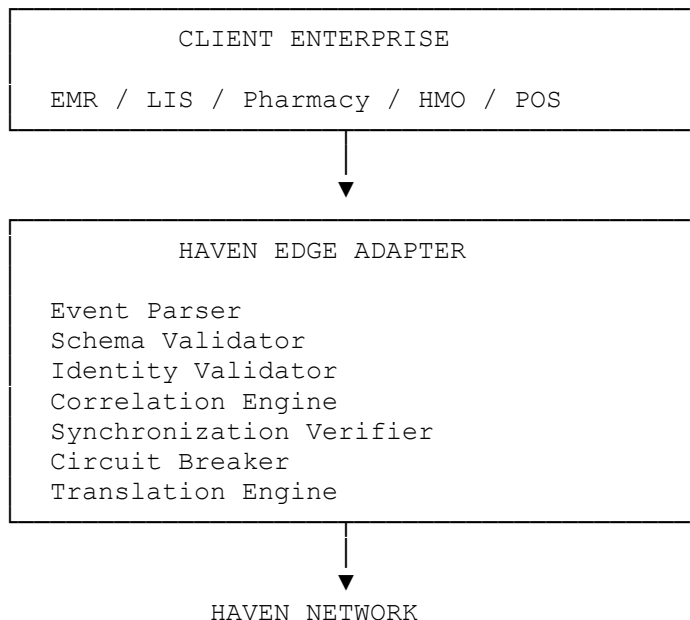
It establishes the **technical authorization boundary** through which Haven-governed events must pass.

The adapter performs four principal functions:

1. **Event Intake**
2. **Protocol Validation**
3. **On-Chain Synchronization Verification**
4. **Cross-Network Release or Rejection**

The adapter must be designed as a fail-closed enforcement service.

### 14.2 Logical Architecture



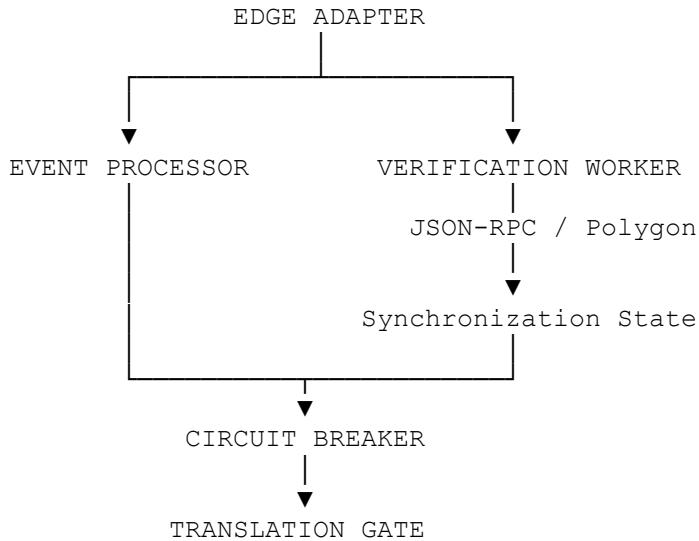
The adapter must prevent unvalidated client traffic from reaching the external coordination layer.

## 14.3 Verification Worker

A dedicated verification worker should execute independently from the event-processing worker.

This prevents a blocked or overloaded transaction pipeline from preventing the synchronization state from being checked.

Conceptually:



The verification worker should not depend on the client application's representation of payment status.

## 14.4 Verification Pseudocode

```
function verifyNetwork():  
  
    chainId = rpc.eth_chainId()  
  
    if chainId != EXPECTED_CHAIN_ID:  
        breaker.open("CHAIN_MISMATCH")  
        return  
  
    code = rpc.eth_getCode(SPLITTER_ADDRESS)  
  
    if code == "0x":  
        breaker.open("CONTRACT_NOT_FOUND")  
        return  
  
    synchronized =  
        rpc.eth_call(  
            SPLITTER_ADDRESS,  
            isNetworkSynchronized(NETWORK_ADDRESS)  
        )  
  
    if synchronized != true:  
        breaker.open("NETWORK_UNSYNCHRONIZED")  
        return  
  
    lastVerifiedAt = now()  
    breaker.close()
```

The actual implementation must include RPC error handling, retry policy, timeout controls, observability, and cryptographic/ABI validation.

## 14.5 Verification Frequency

The baseline verification interval is:

**60 seconds**

The scheduler should not permit overlapping verification jobs.

A robust implementation should use a single-flight mechanism:

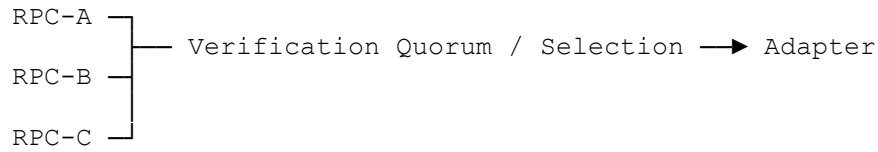
```
IF verification already running:  
    do not start second verification  
  
ELSE:  
    execute verification
```

This prevents network instability from creating a large number of simultaneous RPC calls.

## 14.6 RPC Redundancy

A production deployment should avoid creating a single RPC-provider dependency.

The adapter may maintain multiple approved RPC endpoints:



The exact quorum policy depends on the deployment's threat model.

At minimum, an RPC timeout must not be interpreted as `true`.

An unavailable verification source produces an **unknown** condition, which the fail-closed policy treats as unauthorized.

## 15. ADAPTER STATE & CIRCUIT-BREAKER IMPLEMENTATION

### 15.1 Local State Record

The adapter should maintain a protected local state object:

```
{
  "networkId": "HAVEN-NET-001",
  "synchronized": true,
  "lastVerifiedAt": "2026-09-02T10:00:00Z",
  "verificationSource": "POLYGON",
  "chainId": 137,
  "contractAddress": "0xEXPECTED_SPLITTER_ADDRESS",
  "breakerState": "CLOSED"
}
```

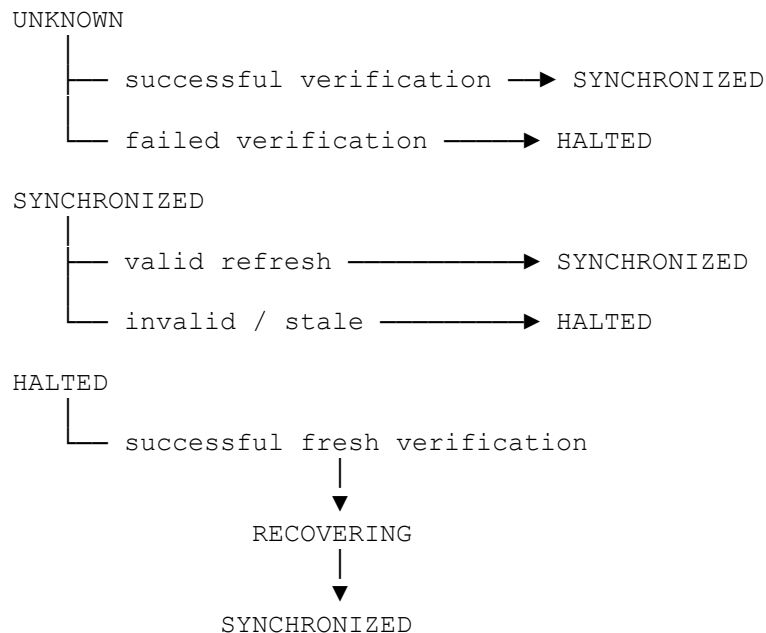
This object is operational state.

It is not the authoritative source of synchronization.

The adapter must be capable of rebuilding it from authoritative verification.

### 15.2 Protected State Transitions

The synchronization state machine is:



A recovery state allows the implementation to perform any required validation before reopening the translation gate.

## 15.3 Circuit-Breaker Pseudocode

```
function authorizeTranslation():  
  
    if breakerState != CLOSED:  
        return DENY  
  
    if currentTime - lastVerifiedAt > VERIFICATION_TTL:  
        breaker.open("STALE_VERIFICATION")  
        return DENY  
  
    if synchronized != true:  
        breaker.open("UNSYNCHRONIZED")  
        return DENY  
  
    return ALLOW
```

The authorization decision must execute immediately before release of Haven-governed traffic.

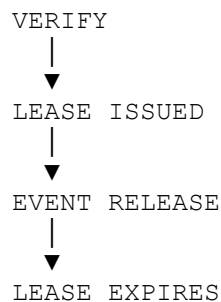
A check performed only when the adapter starts is insufficient.

## 15.4 Race-Condition Protection

The adapter must account for a synchronization state changing between verification and event release.

The implementation should therefore minimize the authorization window.

For higher-assurance deployments, the event pipeline may use a short-lived authorization lease:



The lease must not become an alternative to on-chain verification.

It is simply a bounded local representation of a recently verified state.

## 15.5 Fail-Closed Transport Policy

When the breaker is open:

```
Haven Outbound Event → REJECT  
Haven Inbound Event → REJECT / HOLD  
Haven Cross-Network Query → REJECT  
Haven Workflow Transition → REJECT
```

The adapter should return an explicit protocol-level error rather than silently dropping the event.

Example:

```
{  
  "code": "HAVEN_NETWORK_LOCKED",  
  "reason": "ON_CHAIN_SYNCHRONIZATION_INVALID",  
  "retryable": true,  
  "networkId": "HAVEN-NET-001"  
}
```

The response must not expose sensitive internal verification information to an untrusted caller.

## 15.6 Local Queue Behavior

Events received while the breaker is open should enter a controlled queue only where the deployment explicitly permits deferred processing.

The queue must have:

- maximum capacity;
- retention period;
- encryption at rest;
- integrity protection;
- duplicate detection;
- replay protection;
- dead-letter handling; and
- deterministic ordering where required.

The existence of a queue must not become a bypass mechanism.

A queued event remains **unauthorized** until the adapter establishes a valid synchronization state.

## 15.7 Recovery Procedure

When synchronization becomes valid again:

1. Verify Polygon chain.
2. Verify splitter contract.
3. Verify synchronization state.
4. Confirm verification freshness.
5. Transition breaker to RECOVERING.
6. Validate queued events.
7. Remove duplicates.
8. Reconcile settlement references.
9. Re-establish translation authorization.
10. Transition breaker to CLOSED.

Recovery must not blindly replay every queued event.

Each event must be revalidated against its idempotency key and current protocol state.

## 15.8 Operational Principle

The adapter must never interpret an inability to verify as evidence that synchronization exists.

The fundamental rule is:

**No Proof = No Authorization.**

This principle applies equally to:

- missing RPC responses;
- malformed contract calls;
- unknown contract code;
- stale state;
- incorrect chain;
- invalid settlement evidence;
- synchronization set to false; and
- suspected routing divergence.

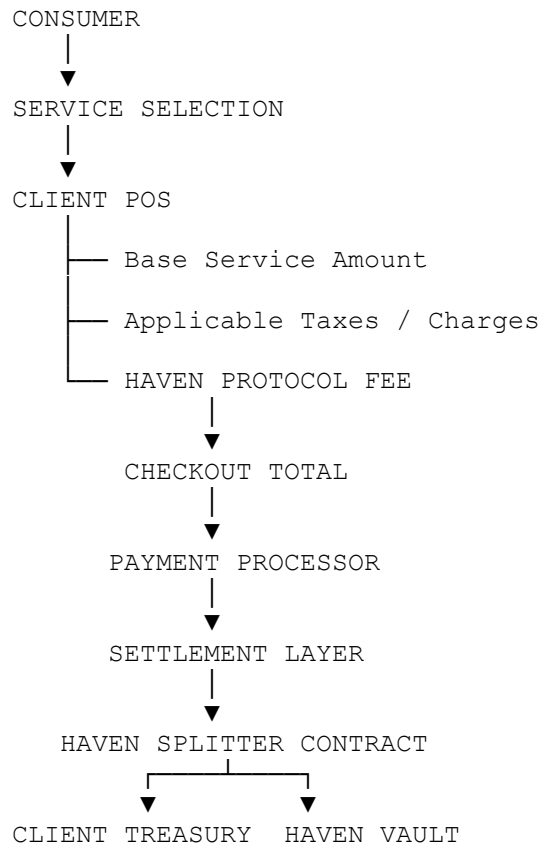
The payment enclosure therefore remains effective even when the client application itself is compromised.

## 16. POINT-OF-SALE PAYMENT GATEWAY INTEGRATION

### 16.1 Consumer Checkout Architecture

The Haven payment mechanism must be embedded into the consumer checkout path as an infrastructure charge rather than presented as an optional post-processing contribution.

The canonical transaction structure is:



The Haven fee must be calculated before payment authorization.

The payment request therefore represents the complete consumer obligation rather than requiring a secondary charge after the transaction.

### 16.2 Fee Construction

For a percentage or fixed protocol configuration, the checkout service must construct a deterministic fee object.

Reference fixed-fee model:

```
{
  "feeType": "FIXED_PROTOCOL_EVENT_FEE",
  "currency": "NGN",
  "amountMinor": 10000,
  "displayAmount": "₦100",
  "eventBlockCount": 1,
  "protocolVersion": "1.0",
  "mandatory": true
}
```

The amount is represented in minor currency units.

For Nigerian Naira:

₦100 = 10,000 minor units

The implementation must not use floating-point arithmetic for monetary calculations.

## 16.3 Checkout Total

The canonical calculation is:

```
TOTAL =
  SERVICE_AMOUNT
+ TAXES
+ OTHER_APPLICABLE_CHARGES
+ HAVEN_PROTOCOL_FEE
```

Example:

|                    |         |
|--------------------|---------|
| Service            | ₦10,000 |
| Haven Protocol Fee | ₦100    |
| -----              |         |
| Consumer Total     | ₦10,100 |

The client interface may display this information to the consumer.

However, the frontend must not be the authoritative source of the fee.

The server-side transaction engine must independently calculate and enforce the amount.

## 16.4 Mandatory Fee Integrity

The following request must be rejected:

```
{
  "subtotal": 1000000,
  "havenFee": 0,
  "total": 1000000
}
```

when the event classification requires:

```
HAVEN_PROTOCOL_FEE = 10000
```

The checkout API should return:

```
{
  "code": "HAVEN_FEE_REQUIRED",
  "message": "Protocol fee requirement not satisfied.",
  "retryable": false
}
```

The frontend cannot override this decision.

## 16.5 Checkout API Contract

Reference request:

```
POST /api/v1/checkout/quote
Content-Type: application/json
Idempotency-Key: <unique-key>
{
  "merchantId": "MERCHANT-001",
  "eventType": "ENCOUNTER_CREATED",
  "currency": "NGN",
  "subtotalMinor": 1000000,
  "quantity": 1,
  "metadata": {
    "posSessionId": "POS-SESSION-001"
  }
}
```

Reference response:

```
{
  "transactionId": "POS-TXN-001",
  "currency": "NGN",
  "subtotalMinor": 1000000,
  "havenFeeMinor": 10000,
  "totalMinor": 1010000,
  "feeRequired": true,
  "protocolVersion": "1.0",
  "expiresAt": "2026-09-02T10:15:00Z"
}
```

The quote should have a controlled expiration period.

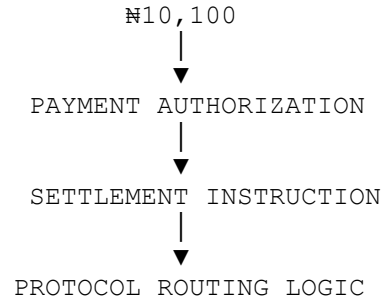
A payment submitted against an expired quote must be recalculated.

## 16.6 Payment Authorization

The payment processor receives the total amount authorized by the checkout service.

The client must not create a second payment path for the Haven fee.

Conceptually:



The system should maintain a deterministic relationship between the consumer transaction and its corresponding protocol event.

Required correlation fields include:

```
transactionId
eventId
idempotencyKey
paymentReference
settlementReference
```

## 17. PAYMENT ROUTING ENFORCEMENT

### 17.1 Routing Invariant

The payment routing invariant is:

```
IF Haven Event Requires Fee
AND Consumer Transaction Is Authorized
THEN
Required Protocol Fee MUST Enter Approved Routing Path
```

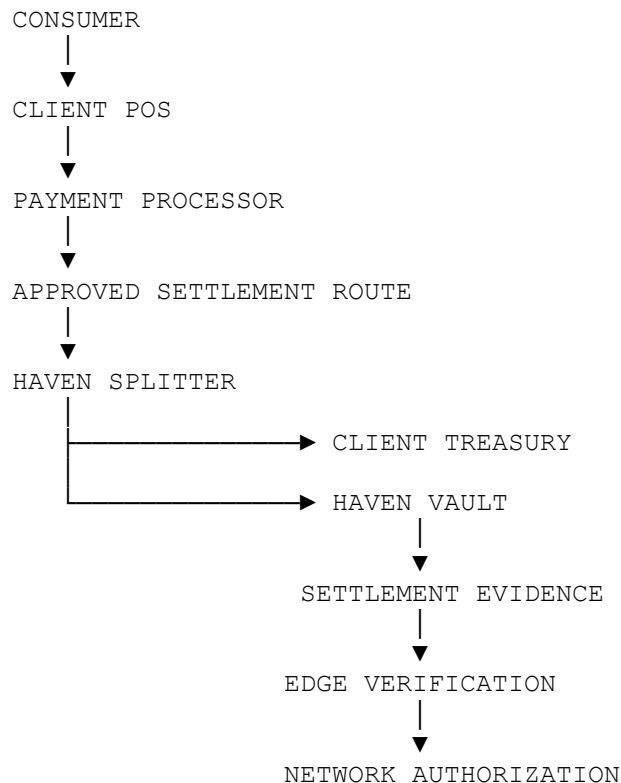
The system must reject any transaction state that claims successful Haven participation without corresponding settlement evidence.

This produces the core invariant:

**No Split, No Data Traffic.**

### 17.2 Approved Routing Path

The approved routing path is fixed:



Any alternative route must be treated as non-compliant.

## 17.3 Route-Binding Configuration

The production deployment must maintain a cryptographically protected configuration containing the expected routing parameters.

Reference:

```
{
  "protocolVersion": "1.0",
  "network": "polygon",
  "chainId": 137,
  "splitterAddress": "0xEXPECTED_SPLITTER_ADDRESS",
  "clientTreasury": "0xEXPECTED_TREASURY_ADDRESS",
  "havenVault": "0xEXPECTED_VAULT_ADDRESS",
  "clientBps": 9500,
  "havenBps": 500,
  "verificationIntervalSeconds": 60,
  "verificationTtlSeconds": 120
}
```

This configuration must be treated as deployment policy rather than user-editable application configuration.

## 17.4 Configuration Tamper Detection

At startup, and periodically during operation, the adapter should calculate a canonical configuration digest.

Example:

```
CONFIG_DIGEST =
SHA-256(
  canonical(protocolVersion)
  || canonical(chainId)
  || canonical(splitterAddress)
  || canonical(clientTreasury)
  || canonical(havenVault)
  || canonical(clientBps)
  || canonical(havenBps)
)
```

The resulting digest should be compared against the approved deployment manifest.

A mismatch must produce:

```
CONFIGURATION_TAMPERED
```

and transition the adapter into a fail-closed state.

The digest does not replace on-chain verification.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.  
Licensed under GNU AGPL-3.0 rules.

It detects local configuration divergence.

## 17.5 Client Developer Protection Boundary

The architecture must assume that a client developer may have administrative access to the POS application.

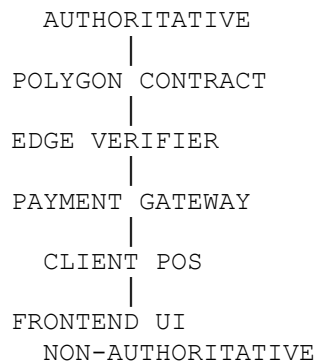
Therefore, the POS application itself cannot be the ultimate enforcement boundary.

A developer with access to:

Frontend  
Backend  
Database  
Environment Variables  
Application Configuration

must not be able to modify the protocol's authoritative payment destination.

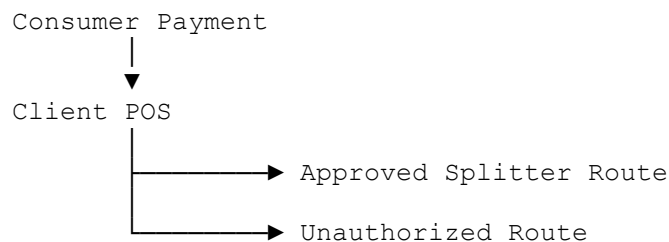
The enforcement hierarchy is:



The lower layers cannot supersede the higher layers.

## 17.6 Bypass Detection

A bypass attempt may occur when:



The splitter cannot observe the unauthorized transaction because it never receives it.

Therefore, bypass detection is performed through reconciliation and edge authorization.

The adapter compares:

Expected Protocol Obligation  
VS  
Observed Settlement Evidence

If the expected Haven fee has no valid corresponding settlement evidence, the event remains unauthorized.

## 17.7 Bypass Response

A detected routing divergence produces:

```
ROUTING_DIVERGENCE
  |
  ▼
NETWORK STATE = UNSYNCHRONIZED
  |
  ▼
CIRCUIT BREAKER = OPEN
  |
  ▼
CROSS-NETWORK TRAFFIC = BLOCKED
```

The client application may continue its permitted local operations.

However, Haven-governed cross-network coordination cannot continue until the routing condition is restored and independently verified.

## 17.8 Prohibited Override Mechanisms

The following mechanisms must not exist in production:

```
skipHavenFee()
disableProtocolFee()
overrideSplitter()
setAlternativeTreasury()
bypassSettlement()
forceSynchronized()
adminDisableLock()
temporaryFeeWaiver()
developerModeBypass()
```

No application-level administrative privilege should be capable of converting an unauthorized payment route into an authorized Haven network state.

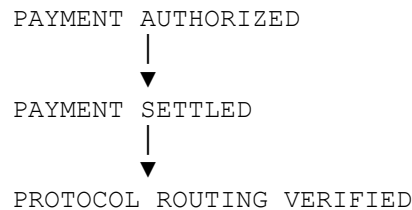
Any emergency mechanism must operate through a separately governed protocol-level control plane and must not provide a hidden bypass around the payment invariant.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.  
Licensed under GNU AGPL-3.0 rules.

## 18. PAYMENT PROCESSOR & SETTLEMENT ORCHESTRATION

### 18.1 Separation of Payment Authorization and Protocol Settlement

The Haven architecture must distinguish between three separate financial states:



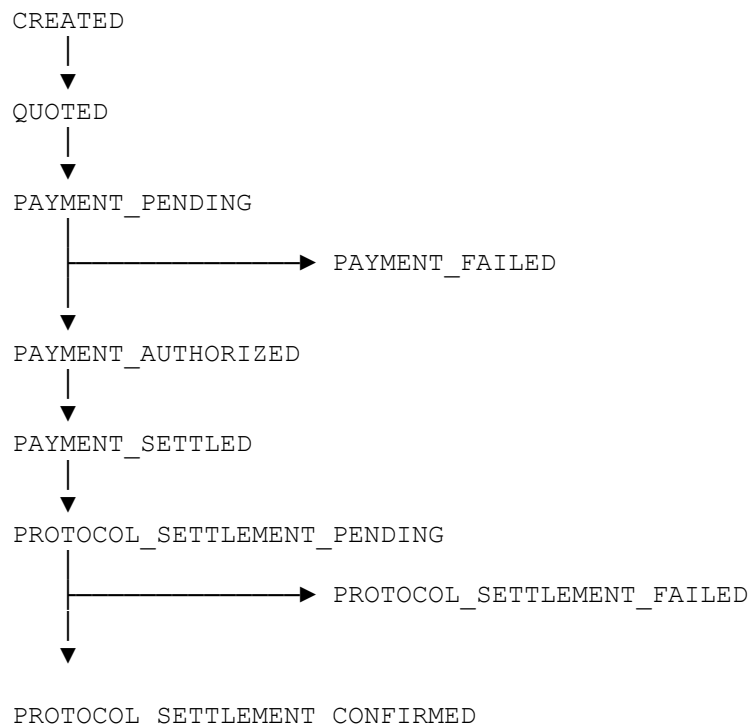
These states must never be collapsed into a single Boolean field.

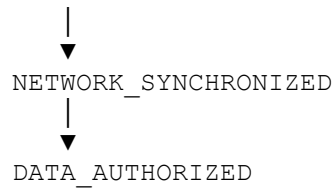
A successful card, bank transfer, wallet, or other payment processor response establishes only that the commercial payment mechanism accepted the transaction.

It does not independently establish that the Haven protocol fee reached the required routing enclosure.

### 18.2 Canonical Financial State Model

Each transaction should progress through the following states:





A transaction must not jump directly from `PAYMENT_SETTLED` to `DATA_AUTHORIZED`.

## 18.3 Payment Processor Adapter

The payment processor integration should be isolated behind a provider-neutral adapter interface.

Reference interface:

```
interface PaymentProcessorAdapter {  
  
    createPaymentIntent(  
        transactionId,  
        amountMinor,  
        currency,  
        idempotencyKey  
    )  
  
    authorizePayment(  
        paymentIntentId  
    )  
  
    retrievePayment(  
        paymentReference  
    )  
  
    verifyWebhook(  
        rawPayload,  
        signature  
    )  
  
    mapPaymentStatus(  
        providerStatus  
    )  
}
```

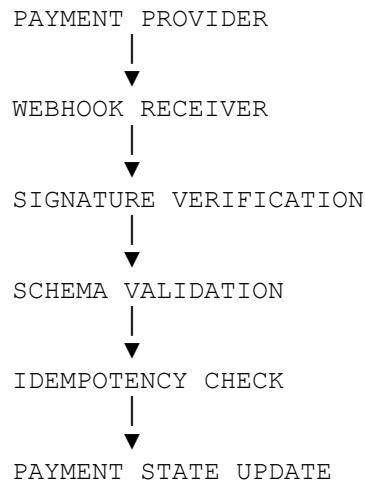
Provider-specific implementation details must remain outside the Haven protocol core.

This allows the enterprise to change payment processors without modifying the authoritative protocol routing rules.

## 18.4 Webhook Verification

Payment processor webhooks must be treated as untrusted input until cryptographically verified.

The webhook pipeline is:



The implementation must reject:

- invalid signatures;
- malformed payloads;
- unknown transaction references;
- duplicate events;
- expired event timestamps where provider policy supports timestamp validation; and
- contradictory state transitions.

## 18.5 Idempotency

Every payment initiation must have an immutable idempotency key.

Example:

```
{
  "idempotencyKey": "haven-pos-20260902-000001",
  "transactionId": "POS-TXN-000001",
  "amountMinor": 1010000,
  "currency": "NGN"
}
```

The same idempotency key must not produce multiple protocol obligations.

If a payment processor retries a request, the system must return the original transaction result rather than create a second fee obligation.

## 18.6 Duplicate Settlement Protection

The settlement observer must detect duplicate blockchain evidence.

A canonical settlement identity may be constructed from:

```
chainId
+
transactionHash
+
logIndex
+
splitterAddress
```

For native transfers without a protocol event log, the implementation must use the transaction hash and an independently validated transaction structure.

Duplicate evidence must not create duplicate ledger entries.

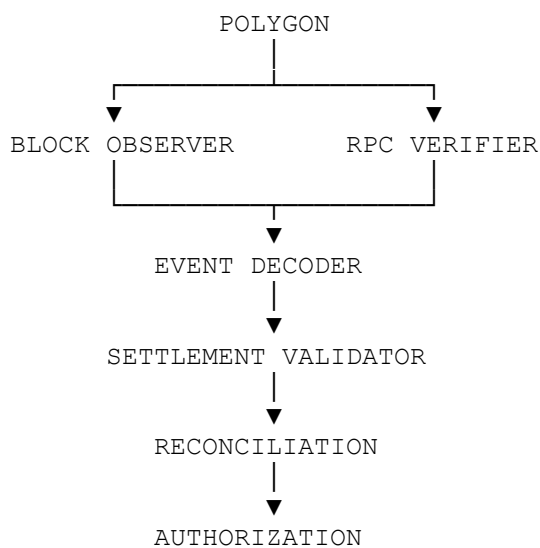
## 19. SETTLEMENT OBSERVATION & RECONCILIATION ENGINE

### 19.1 Settlement Observer

The Settlement Observer is responsible for independently determining whether the expected Haven routing event actually occurred.

It should consume blockchain data without trusting the client POS.

Reference flow:



### 19.2 Required Settlement Evidence

A settlement record should contain, at minimum:

```
{
  "chainId": 137,
  "transactionHash": "0xEXPECTED_TRANSACTION_HASH",
  "blockNumber": 12345678,
  "splitterAddress": "0xEXPECTED_SPLITTER_ADDRESS",
  "eventName": "FeeReceived",
  "networkId": "HAVEN-NET-001",
  "transactionId": "POS-TXN-000001",
  "eventId": "EVT-000001",
  "amountWei": "10000000000000000",
  "clientAmountWei": "9500000000000000",
  "havenAmountWei": "5000000000000000",
  "confirmationState": "CONFIRMED"
}
```

The actual production amount must correspond to the configured settlement asset and payment architecture.

The example is structural rather than a statement of the actual Polygon asset denomination used by every Haven deployment.

## 19.3 Settlement Validation Rules

A settlement is valid only when all required conditions pass:

1. Correct chain
2. Correct splitter address
3. Transaction exists
4. Transaction succeeded
5. Required contract event exists
6. Event originated from expected splitter
7. Expected amount received
8. Expected allocation observed
9. Transaction correlation is valid
10. Required confirmation threshold reached

A single failed mandatory condition produces:

SETTLEMENT\_INVALID

## 19.4 Reconciliation Algorithm

Reference logic:

```
expectedFee =
    protocolFee(transactionId)

observedSettlement =
    findSettlement(transactionId, eventId)

IF observedSettlement == null:
    status = MISSING_CHAIN_SETTLEMENT

ELSE IF observedSettlement.contract != EXPECTED_SPLITTER:
    status = CONTRACT_MISMATCH

ELSE IF observedSettlement.amount != expectedFee:
    status = AMOUNT_MISMATCH

ELSE IF observedSettlement.clientAllocation
    != expectedClientAllocation:
    status = DESTINATION_MISMATCH

ELSE IF observedSettlement.havenAllocation
    != expectedHavenAllocation:
    status = DESTINATION_MISMATCH
```

```

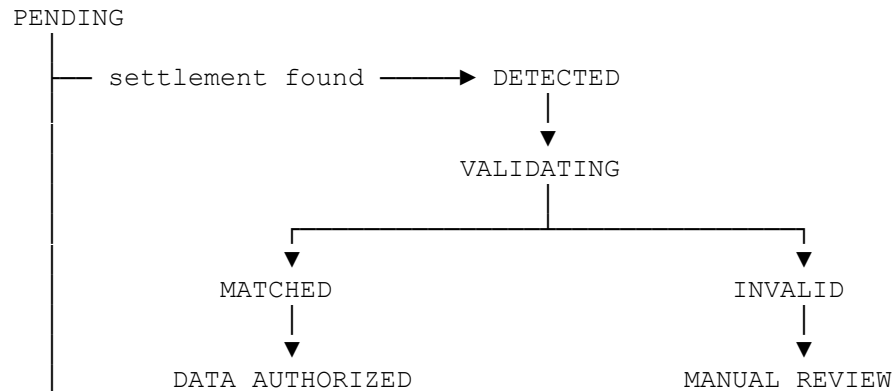
ELSE IF confirmations < REQUIRED_CONFIRMATIONS:
    status = PENDING_CONFIRMATION

ELSE:
    status = MATCHED

```

The reconciliation engine must operate deterministically.

## 19.5 Reconciliation State Machine



An invalid settlement must never be automatically treated as a successful payment.

## 19.6 Reconciliation Categories

The engine should explicitly distinguish:

```

MATCHED
PENDING_CHAIN
MISSING_CHAIN_SETTLEMENT
AMOUNT_MISMATCH
DESTINATION_MISMATCH
CONTRACT_MISMATCH
DUPLICATE
REORGED
CONFIRMATION_INSUFFICIENT
CORRELATION_MISMATCH
MANUAL_REVIEW
RESOLVED

```

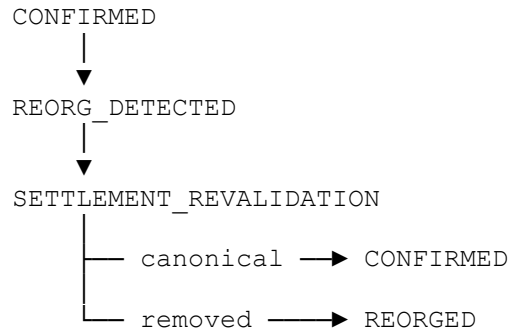
These categories should be persisted as append-oriented records.

The system should not overwrite the historical fact that a mismatch occurred.

## 19.7 Chain Reorganization Handling

A transaction previously observed as confirmed must remain subject to chain reorganization handling according to the deployment's blockchain finality policy.

If a settlement becomes non-canonical:



A reorged settlement must not continue authorizing protocol participation solely because an earlier observation existed.

## 19.8 Authorization Rule

The final authorization condition should be equivalent to:

```
DATA_AUTHORIZED =  
    PaymentSettled  
    AND ProtocolSettlementMatched  
    AND SynchronizationFresh  
    AND NetworkSynchronized  
    AND ContractIdentityValid  
    AND ConfigurationIntegrityValid  
    AND CircuitBreakerClosed
```

This creates a layered authorization boundary.

No single client-side status flag can satisfy the entire condition.

**The commercial payment may succeed while the Haven protocol remains locked.**

That distinction is fundamental to maintaining the Zero-Trust Payment Routing Enclosure.

## 20. SECURITY HARDENING & THREAT MODEL

### 20.1 Security Objective

The Haven payment architecture must assume that participating enterprise environments are potentially compromised.

The protocol therefore cannot depend on the integrity of:

- client frontend code;
- client backend code;
- POS configuration;
- client database records;
- environment variables;
- application administrators;
- payment status flags;
- locally generated synchronization flags; or
- manually maintained settlement records.

The security objective is to ensure that compromise of the client application does not provide a path to bypass the Haven economic obligation or falsely authorize Haven network participation.

### 20.2 Threat Model

The baseline threat model includes:

T1 Malicious developer  
T2 Compromised POS application  
T3 Modified frontend  
T4 Modified backend  
T5 Database manipulation  
T6 Configuration tampering  
T7 Payment-routing substitution  
T8 RPC manipulation  
T9 Replay attack  
T10 Duplicate settlement  
T11 Webhook forgery  
T12 Stale synchronization state  
T13 Contract-address substitution  
T14 Unauthorized treasury substitution  
T15 Blockchain reorganization  
T16 Credential compromise

The architecture must treat these as expected adversarial conditions rather than exceptional events.

## 20.3 Trust Boundary Classification

Each component must be assigned a trust classification.

| Component                                  | Trust Level    | Authoritative?                              |
|--------------------------------------------|----------------|---------------------------------------------|
| Polygon consensus                          | Highest        | Yes                                         |
| Immutable Splitter                         | Highest        | Yes                                         |
| Independently verified settlement evidence | High           | Yes                                         |
| Haven Edge Adapter                         | Enforcement    | Locally authoritative for release decisions |
| Payment Processor                          | External trust | No                                          |
| Client Backend                             | Untrusted      | No                                          |
| Client Database                            | Untrusted      | No                                          |
| POS Frontend                               | Untrusted      | No                                          |
| Consumer Device                            | Untrusted      | No                                          |

The adapter may make a local enforcement decision, but it must derive that decision from independently verifiable protocol conditions.

## 20.4 Attack: Fee Suppression

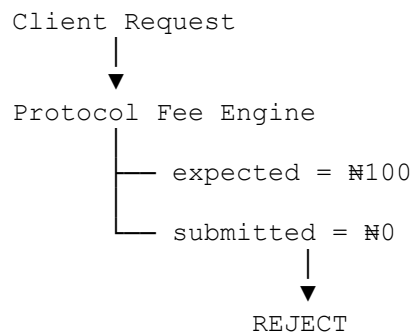
### Attack

A developer modifies the POS application:

```
havenFee = 0
```

### Required Response

The server-side checkout engine independently calculates the obligation.



The frontend cannot suppress the protocol requirement.

## 20.5 Attack: Splitter Substitution

### Attack

A malicious administrator changes:

```
SPLITTER_ADDRESS
```

to an attacker-controlled contract.

### Required Response

The adapter compares the configured address against the deployment manifest and independently validates the expected contract identity.

The unauthorized address must produce:

```
CONTRACT_IDENTITY_INVALID
```

The circuit breaker opens.

## 20.6 Attack: Treasury Substitution

The Splitter contract must not expose a production mechanism allowing the client to dynamically replace the approved treasury.

The production contract therefore uses immutable constructor-defined routing parameters:

```
address public immutable clientTreasury;  
address public immutable havenVault;  
uint16 public immutable clientBps;  
uint16 public immutable havenBps;
```

This prevents an application administrator from changing the destination through a normal client API.

## 20.7 Attack: Synchronization Forgery

A compromised client backend may attempt:

```
{  
  "networkSynchronized": true  
}
```

The adapter must ignore this value.

Instead:

Client Claim



REJECTED AS AUTHORITY

The adapter queries the approved contract directly.

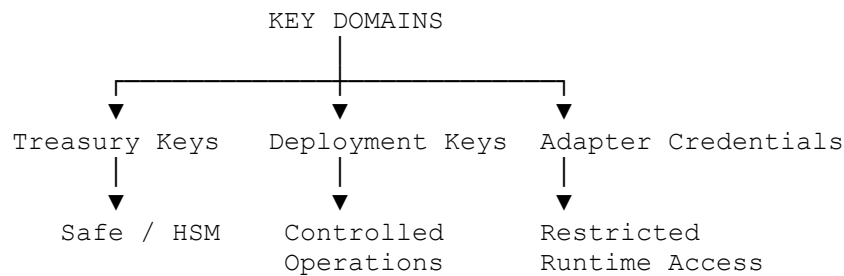
Only independently verified protocol state can satisfy the synchronization condition.

## 21. CRYPTOGRAPHIC & OPERATIONAL CONTROLS

### 21.1 Key Separation

Haven deployments must separate cryptographic responsibilities.

At minimum:

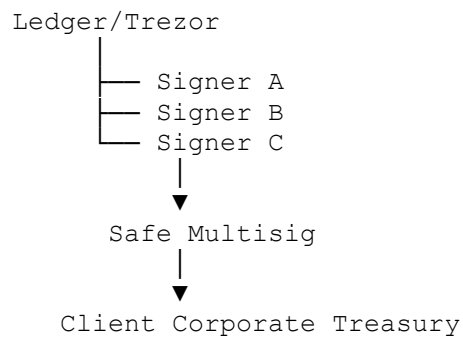


No single application credential should control all protocol functions.

### 21.2 Treasury Custody

The corporate treasury should be controlled by a hardware-backed multisignature wallet.

Reference architecture:

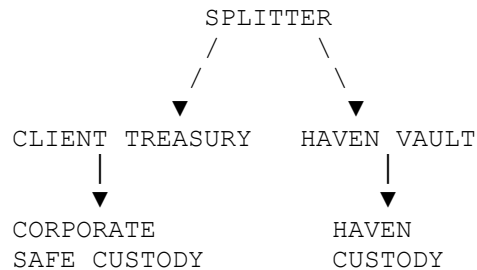


The payment routing contract should send the client allocation directly to the approved treasury address.

The POS application must never hold the treasury private key.

## 21.3 Haven Vault Separation

The Haven allocation should terminate at a separately controlled Haven vault.



The separation ensures that the client cannot redirect the Haven allocation through its own application infrastructure.

## 21.4 Private-Key Exposure Rules

Private keys must never be:

- embedded in frontend JavaScript;
- stored in source control;
- transmitted through application APIs;
- stored in plaintext environment variables;
- placed in POS configuration files;
- logged;
- returned by administrative endpoints; or
- exposed to ordinary application developers.

Where signing is required, the preferred architecture is hardware-backed or HSM-backed signing.

## 21.5 Administrative Privilege Separation

Administrative roles should be separated into at least:

PROTOCOL OPERATOR  
TREASURY OPERATOR  
INFRASTRUCTURE OPERATOR  
SECURITY OPERATOR  
AUDITOR

These roles should not automatically inherit one another's privileges.

For example:

```
Infrastructure Operator
      ≠
Treasury Signer
```

and:

```
POS Administrator
      ≠
Protocol Administrator
```

The separation reduces the blast radius of credential compromise.

## 21.6 Immutable Deployment Manifest

Every production deployment must maintain a signed or otherwise integrity-protected deployment manifest.

Reference:

```
{
  "protocol": "Haven Protocol",
  "version": "1.0",
  "network": "polygon",
  "chainId": 137,
  "splitterAddress": "0xEXPECTED_SPLITTER_ADDRESS",
  "clientTreasury": "0xEXPECTED_TREASURY_ADDRESS",
  "havenVault": "0xEXPECTED_VAULT_ADDRESS",
  "clientBps": 9500,
  "havenBps": 500,
  "verificationIntervalSeconds": 60
}
```

The manifest should be treated as part of the deployment's security evidence.

## 21.7 Deployment Verification

Before activating a network:

1. Verify chain ID.
2. Verify splitter address.
3. Verify deployed bytecode.
4. Verify contract interface.
5. Verify immutable treasury.
6. Verify immutable Haven vault.
7. Verify allocation basis points.
8. Verify synchronization interface.
9. Verify adapter configuration digest.
10. Execute controlled test transaction.
11. Verify settlement evidence.

12. Verify circuit-breaker behavior.
13. Record deployment evidence.
14. Approve production activation.

No production network should be considered synchronized merely because deployment completed successfully.

## 21.8 Security Logging

Security logs should capture protocol decisions without storing unnecessary clinical information.

Required security events include:

```
CONTRACT_VERIFICATION_FAILED
CHAIN_ID_MISMATCH
CONFIGURATION_TAMPERED
SETTLEMENT_MISSING
SETTLEMENT_MISMATCH
SYNCHRONIZATION_FAILED
CIRCUIT_BREAKER_OPENED
CIRCUIT_BREAKER_RECOVERING
CIRCUIT_BREAKER_CLOSED
UNAUTHORIZED_ROUTE_DETECTED
REPLAY_DETECTED
DUPLICATE_SETTLEMENT_DETECTED
```

Logs must contain correlation identifiers rather than patient-identifying payloads wherever possible.

## 21.9 Security Principle

The security architecture follows a single governing rule:

**Client control must never equal protocol control.**

A client may operate its healthcare systems.

It may administer its POS.

It may maintain its databases.

It may manage its internal workflows.

But it must not be able to unilaterally redefine:

```
Where protocol fees go.
Whether a required fee exists.
What contract is authoritative.
Whether the network is synchronized
```

Whether an unauthorized route is acceptable.

Those decisions belong to the protocol enforcement boundary.

## 22. PRODUCTION API & JSON SCHEMA CONTRACTS

### 22.1 API Design Principle

The Haven integration API must expose protocol operations through explicit, versioned contracts.

No client integration should depend on undocumented behavior.

Every production endpoint must define:

- request schema;
- response schema;
- authentication requirements;
- authorization requirements;
- idempotency behavior;
- error semantics;
- timeout behavior;
- retry behavior; and
- audit requirements.

The API must remain subordinate to the on-chain payment enforcement layer.

An API response stating `synchronized: true` cannot override a failed blockchain verification.

### 22.2 Protocol Fee Quote Endpoint

#### Endpoint

POST /api/v1/protocol/fee/quote

#### Request

```
{
  "schemaVersion": "1.0",
  "transactionId": "POS-TXN-000001",
  "networkId": "HAVEN-NET-001",
  "eventType": "ENCOUNTER_CREATED",
  "currency": "NGN",
  "subtotalMinor": 1000000,
  "idempotencyKey": "haven-quote-000001"
}
```

## Response

```
{
  "schemaVersion": "1.0",
  "transactionId": "POS-TXN-000001",
  "feeType": "FIXED_PROTOCOL_EVENT_FEE",
  "currency": "NGN",
  "havenFeeMinor": 10000,
  "mandatory": true,
  "protocolVersion": "1.0",
  "expiresAt": "2026-09-02T10:15:00Z"
}
```

The quote service should calculate the fee using server-side protocol configuration.

## 22.3 Protocol Authorization Endpoint

### Endpoint

POST /api/v1/protocol/authorize

The endpoint should be called only after the payment and settlement workflow reaches the appropriate state.

Reference request:

```
{
  "schemaVersion": "1.0",
  "transactionId": "POS-TXN-000001",
  "eventId": "EVT-000001",
  "paymentReference": "PAY-000001",
  "settlementReference": "0xEXPECTED_TRANSACTION_HASH"
}
```

Reference response:

```
{
  "authorized": true,
  "authorizationState": "DATA_AUTHORIZED",
  "transactionId": "POS-TXN-000001",
  "eventId": "EVT-000001",
  "verifiedAt": "2026-09-02T10:05:00Z"
}
```

The authorization service must independently verify the settlement evidence before returning `authorized: true`.

## 22.4 Rejection Response Contract

All protocol failures should use machine-readable error codes.

Reference:

```
{
  "schemaVersion": "1.0",
  "authorized": false,
  "code": "HAVEN_NETWORK_LOCKED",
  "reason": "PROTOCOL_SETTLEMENT_MISSING",
  "retryable": true,
  "correlationId": "CORR-000001"
}
```

Recommended error taxonomy:

```
HAVEN_FEE_REQUIRED
HAVEN_FEE_MISMATCH
PAYMENT_NOT_SETTLED
PROTOCOL_SETTLEMENT_MISSING
PROTOCOL_SETTLEMENT_MISMATCH
CONTRACT_IDENTITY_INVALID
CHAIN_ID_INVALID
SYNCHRONIZATION_INVALID
SYNCHRONIZATION_STALE
CONFIGURATION_TAMPERED
DUPLICATE_EVENT
REPLAY_DETECTED
HAVEN_NETWORK_LOCKED
```

Error messages returned to external clients should remain deliberately limited.

Internal security diagnostics belong in protected operational logs.

## 22.5 Production JSON Schema

The following schema defines the protocol authorization response:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://haven.protocol/schemas/protocol-authorization-
response.schema.json",
  "title": "Haven Protocol Authorization Response",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "authorized",
    "authorizationState"
  ],
  "properties": {
    "authorized": {
```

```

    "type": "boolean"
  },
  "authorizationState": {
    "type": "string",
    "enum": [
      "UNAUTHORIZED",
      "PENDING",
      "DATA_AUTHORIZED",
      "LOCKED"
    ]
  },
  "transactionId": {
    "type": "string",
    "minLength": 1,
    "maxLength": 128
  },
  "eventId": {
    "type": "string",
    "minLength": 1,
    "maxLength": 128
  },
  "verifiedAt": {
    "type": "string",
    "format": "date-time"
  },
  "code": {
    "type": "string",
    "maxLength": 128
  },
  "reason": {
    "type": "string",
    "maxLength": 256
  },
  "retryable": {
    "type": "boolean"
  },
  "correlationId": {
    "type": "string",
    "maxLength": 128
  }
}
}

```

Production implementations should additionally publish schema versioning and compatibility policy.

## 23. EVENT INTEGRITY, IDEMPOTENCY & REPLAY PROTECTION

### 23.1 Event Identity

Every Haven event must have a globally unique protocol identifier.

Reference:

```
eventId = UUIDv4
```

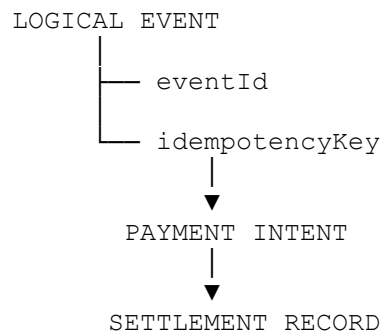
The event ID must remain immutable throughout its lifecycle.

A retry of the same logical event must retain the original identifier rather than generate a new event.

### 23.2 Idempotency Key

Each transaction must also possess an idempotency key.

The relationship is:



The system must prevent the same logical obligation from being represented as multiple payable events.

### 23.3 Replay Protection

A malicious actor may capture a previously valid event and submit it again.

The adapter must therefore maintain a replay-protection record.

Reference:

```
{
  "eventId": "EVT-000001",
  "firstSeenAt": "2026-09-02T10:00:00Z",
  "processingState": "PROCESSED",
  "payloadDigest": "sha256:EXPECTED_DIGEST"
}
```

A second event carrying the same immutable identity but conflicting content must be rejected.

## 23.4 Payload Integrity

The canonical event payload should produce a deterministic digest.

Conceptually:

```
payloadDigest =
SHA-256(
  canonicalize(eventEnvelope)
)
```

The canonicalization process must be deterministic.

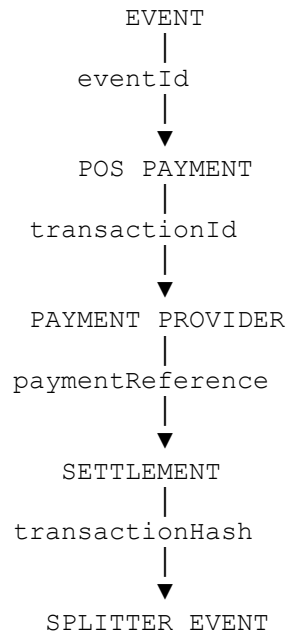
Equivalent JSON representations must not generate different digests solely because of property ordering or whitespace.

## 23.5 Correlation Integrity

The following identifiers should remain linked:

```
transactionId
eventId
paymentReference
settlementReference
transactionHash
```

Reference relationship:



If the chain of correlation cannot be established, authorization must fail.

## 23.6 Replay Window

The adapter should maintain bounded replay records according to deployment retention requirements.

The replay system should consider:

- event ID;
- idempotency key;
- payload digest;
- merchant/network identifier;
- event timestamp;
- processing state.

A replay attempt may produce:

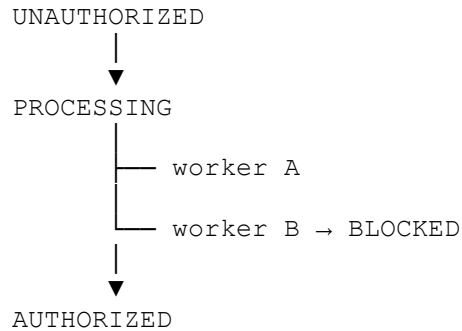
REPLAY\_DETECTED

without exposing the original event contents.

## 23.7 Concurrent Processing

Two workers must not independently authorize the same event.

A distributed lock or transactional database mechanism should protect the authorization transition:



The exact locking mechanism may vary by deployment.

The invariant must not.

## 23.8 Exactly-Once Authorization

Where infrastructure permits, Haven should provide **exactly-once authorization semantics**, even if underlying transport mechanisms are at-least-once.

This distinction is important.

The network may receive:

Event A  
Event A retry  
Event A retry

but the protocol must produce:

ONE authoritative authorization

and one corresponding economic obligation.

## 23.9 Database Transaction Boundary

Where the authorization service uses a relational database, state changes should occur within transactional boundaries.

A conceptual transaction:

```
BEGIN
```

```
Validate Event  
Validate Idempotency  
Validate Payment  
Validate Settlement  
Validate Synchronization
```

```
Create / update authorization record
```

```
COMMIT
```

If any mandatory validation fails:

```
ROLLBACK
```

No partial authorization state should be committed.

## 23.10 Auditability

Every authorization transition should produce an immutable audit record containing:

```
authorizationId  
eventId  
transactionId  
previousState  
newState  
decision  
reasonCode  
verificationTimestamp  
adapterInstance  
protocolVersion
```

Clinical payloads should not be copied into the audit record unless explicitly required by the deployment's lawful data-governance model.

The audit trail exists to prove **why the protocol allowed or denied network participation**, not to become a duplicate clinical database.

## 24. DEPLOYMENT, VALIDATION & ACTIVATION PROCEDURE

### 24.1 Production Deployment Principle

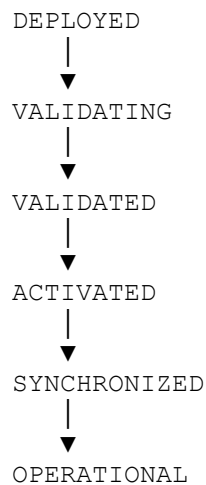
A Haven network must never transition directly from deployment to operational authorization.

Deployment establishes infrastructure.

Validation establishes correctness.

Activation establishes permission to participate.

These are separate lifecycle states:



A deployed but unvalidated network must remain locked.

### 24.2 Pre-Deployment Controls

Before contract deployment, the implementation team must establish a signed deployment manifest containing:

Protocol Version  
Chain ID  
Splitter Address Policy  
Client Treasury Address  
Haven Vault Address  
Client Allocation Basis Points  
Haven Allocation Basis Points  
Synchronization Authority  
Verification Interval  
Verification TTL  
Required Confirmation Threshold

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.  
Licensed under GNU AGPL-3.0 rules.

Approved RPC Endpoints  
Adapter Version  
Schema Versions

No production deployment should proceed while any required parameter remains undefined.

## 24.3 Smart Contract Deployment Checklist

The deployment operator must independently verify:

- [ ] Compiler version
- [ ] Solidity optimizer configuration
- [ ] Source code hash
- [ ] Build artifact hash
- [ ] Constructor parameters
- [ ] Client treasury address
- [ ] Haven vault address
- [ ] Client BPS
- [ ] Haven BPS
- [ ] Synchronization authority
- [ ] Target chain
- [ ] Deployment account
- [ ] Contract address
- [ ] Deployed bytecode

The deployed bytecode must correspond to the approved production artifact.

The address alone is insufficient evidence of contract identity.

## 24.4 Post-Deployment Contract Verification

After deployment, the adapter or deployment validation service must query:

```
eth_chainId  
eth_getCode  
clientTreasury()  
havenVault()  
clientBps()  
havenBps()
```

and, where applicable:

```
isNetworkSynchronized(NETWORK_ADDRESS)
```

Expected values must exactly match the approved manifest.

Any mismatch produces:

DEPLOYMENT\_VALIDATION\_FAILED

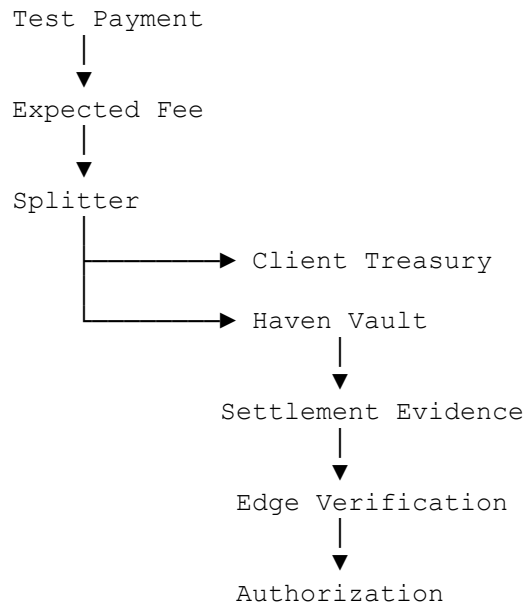
and prevents

activation.

## 24.5 Controlled Settlement Test

Before live consumer transactions are authorized, the deployment must execute a controlled test transaction using the approved routing mechanism.

The validation sequence is:



The test must verify both the transaction-level result and the downstream authorization behavior.

## 24.6 Negative Test Requirements

Production activation must include deliberate failure tests.

At minimum:

Test 1 Incorrect splitter address  
Test 2 Incorrect chain ID  
Test 3 Missing settlement  
Test 4 Incorrect fee amount  
Test 5 Incorrect destination  
Test 6 Synchronization = false  
Test 7 Stale synchronization

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.  
Licensed under GNU AGPL-3.0 rules.

Test 8 RPC unavailable  
Test 9 Configuration modified  
Test 10 Duplicate event  
Test 11 Replay attempt  
Test 12 Failed splitter transaction

Every negative test must demonstrate that the relevant Haven-governed operation is rejected or held.

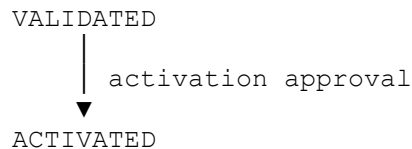
A system that passes only successful-path testing is not production validated.

## 25. GO-LIVE CONTROL PLANE

### 25.1 Activation Authority

Production activation must require an explicit controlled transition.

Reference:



The POS application must not possess the authority to activate itself.

Activation should occur through the infrastructure control plane using appropriately restricted administrative credentials.

### 25.2 Activation Preconditions

The network may transition to `ACTIVATED` only if:

|                                      |                     |
|--------------------------------------|---------------------|
| <code>contractIdentityValid</code>   | <code>= true</code> |
| <code>configurationValid</code>      | <code>= true</code> |
| <code>treasuryVerified</code>        | <code>= true</code> |
| <code>havenVaultVerified</code>      | <code>= true</code> |
| <code>splitConfigurationValid</code> | <code>= true</code> |
| <code>settlementTestPassed</code>    | <code>= true</code> |
| <code>negativeTestsPassed</code>     | <code>= true</code> |
| <code>adapterHealthy</code>          | <code>= true</code> |
| <code>rpcVerificationHealthy</code>  | <code>= true</code> |
| <code>schemaValidationHealthy</code> | <code>= true</code> |
| <code>auditPipelineHealthy</code>    | <code>= true</code> |

Equivalent logical condition:

```
ACTIVATE =  
  ContractValid  
  AND ConfigurationValid  
  AND RoutingValid  
  AND SettlementTestPassed  
  AND NegativeTestsPassed  
  AND AdapterHealthy  
  AND VerificationHealthy  
  AND AuditHealthy
```

## 25.3 Production Configuration Lock

Once activated, routing-critical configuration should become immutable from the client application layer.

Protected parameters include:

```
splitterAddress
clientTreasury
havenVault
clientBps
havenBps
chainId
networkId
synchronizationAuthority
```

A modification request must be treated as a deployment-level change rather than a runtime application setting.

Where the underlying contract itself is immutable, any legitimate routing change requires deployment of a separately governed contract and a formal protocol migration procedure.

## 25.4 Migration Prohibition

A client must not be able to silently migrate from:

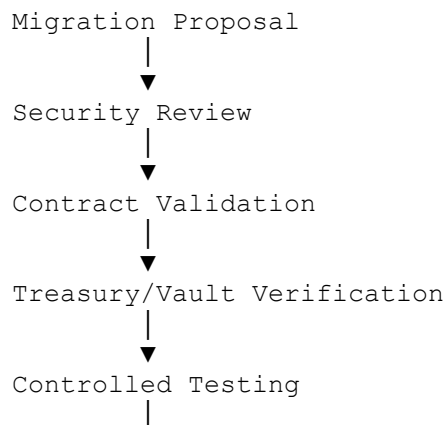
Approved Splitter

to:

Alternative Splitter

through an ordinary configuration update.

Any migration must have:



▼  
Activation

The old and new routing states must remain auditable.

## 25.5 Runtime Health Model

The production control plane should continuously evaluate:

Blockchain Connectivity  
Contract Identity  
Synchronization State  
Settlement Observation  
Configuration Integrity  
Adapter Health  
Queue Health  
Database Health  
Audit Pipeline  
Schema Compatibility

Reference health state:

```
{
  "networkId": "HAVEN-NET-001",
  "status": "OPERATIONAL",
  "contractValid": true,
  "synchronized": true,
  "settlementObserverHealthy": true,
  "configurationValid": true,
  "adapterHealthy": true,
  "lastVerificationAt": "2026-09-02T10:00:00Z"
}
```

Operational dashboards must distinguish **healthy**, **degraded**, **locked**, and **unknown** states.

## 25.6 Unknown Is Not Healthy

The control plane must never convert uncertainty into an affirmative authorization state.

Therefore:

|         |                          |
|---------|--------------------------|
| TRUE    | → potentially authorized |
| FALSE   | → unauthorized           |
| UNKNOWN | → unauthorized           |

Examples of UNKNOWN include:

RPC timeout  
Malformed RPC response  
Contract call failure  
Missing settlement evidence  
Unavailable observer  
Stale verification  
Unrecognized bytecode  
Database inconsistency

The safe default is:

UNKNOWN → LOCK

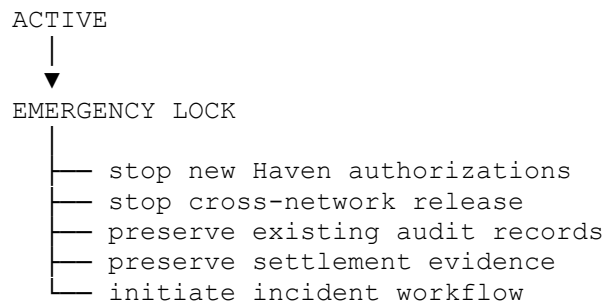
## 25.7 Emergency Lock

A controlled emergency lock may be initiated by authorized security infrastructure when a critical integrity condition is detected.

Example triggers:

Suspected private-key compromise  
Contract identity divergence  
Treasury address divergence  
Unexpected routing event  
Repeated settlement mismatch  
Critical adapter compromise  
Blockchain infrastructure anomaly

Emergency lock behavior:

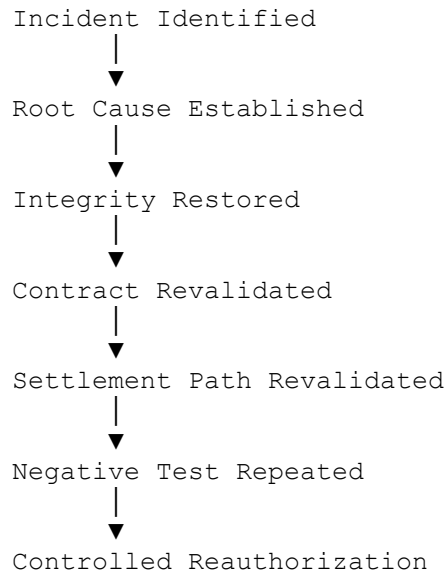


Emergency locking must not delete, rewrite, or conceal historical records.

## 25.8 Unlock Requirements

An emergency lock must not be removed merely by changing a local Boolean.

Unlock requires:



The objective is not simply to restore availability.

The objective is to restore **verifiable protocol integrity**.

## 25.9 Go-Live Acceptance Criterion

A Haven deployment is considered production-ready only when the following statement can be demonstrated:

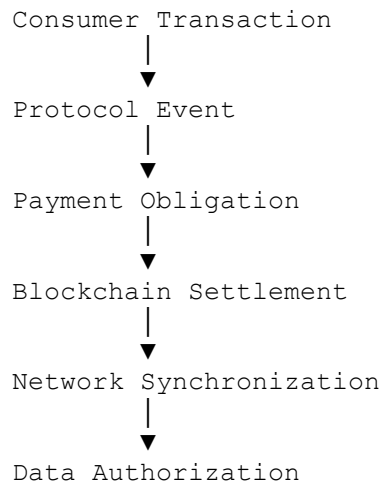
**A client-side administrator cannot disable the mandatory protocol fee, substitute the authoritative payment route, falsify synchronization, or authorize Haven-governed cross-network traffic without satisfying the independently verified protocol conditions.**

That condition represents the minimum acceptable security posture for production activation.

## 26. OBSERVABILITY, AUDIT & FINANCIAL RECONCILIATION

### 26.1 Observability Architecture

The Haven production environment must provide continuous visibility into the complete relationship between:



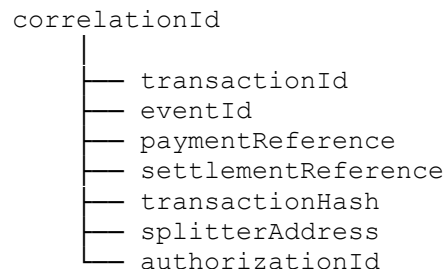
Observability must therefore operate across application, payment, blockchain, adapter, and enforcement layers.

The system must never rely exclusively on POS logs to establish protocol compliance.

### 26.2 Correlation Identifier

A single correlation identifier should connect the transaction lifecycle across system boundaries.

Reference:



The correlation identifier must not contain patient-identifying information.

It exists to establish technical and financial lineage.

## 26.3 Required Metrics

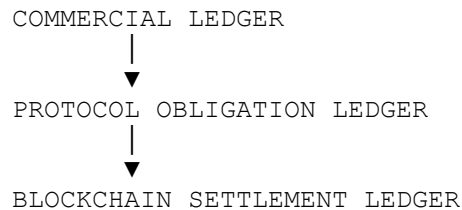
The production monitoring layer should expose metrics including:

```
haven_protocol_events_total
haven_protocol_fees_expected_total
haven_protocol_fees_settled_total
haven_protocol_settlements_matched_total
haven_protocol_settlements_missing_total
haven_protocol_settlements_mismatched_total
haven_network_lock_events_total
haven_network_unlock_events_total
haven_rpc_verification_failures_total
haven_contract_identity_failures_total
haven_configuration_integrity_failures_total
haven_replay_attempts_total
haven_duplicate_settlements_total
haven_authorization_denials_total
```

Metrics should be segmented by network, environment, protocol version, and deployment identifier where operationally appropriate.

## 26.4 Financial Reconciliation

The reconciliation engine must compare three independent ledgers:



The expected Haven fee should be derived from the protocol obligation ledger.

The observed settlement should be derived independently from blockchain evidence.

The commercial ledger confirms what the consumer was charged.

These sources must be reconciled rather than treated as interchangeable.

## 26.5 Daily Reconciliation

At minimum, a production deployment should perform periodic reconciliation across:

Expected Event Count  
Expected Fee Amount  
Payment Processor Settlement  
Observed Blockchain Settlement  
Client Allocation  
Haven Allocation  
Authorization Count  
Locked Event Count

Reference reconciliation equation:

Expected Protocol Fees  
=  
Matched Protocol Settlements  
+  
Valid Pending Settlements  
+  
Documented Exceptions

Undocumented differences must enter investigation.

## 26.6 Financial Invariant

For each successfully settled splitter transaction:

Incoming Amount  
=  
Client Allocation  
+  
Haven Allocation

For a 95/5 configuration:

Client Allocation = Incoming Amount × 9500 / 10000

Haven Allocation =  
Incoming Amount - Client Allocation

The remainder calculation prevents loss caused by integer division.

No floating-point representation may be used for blockchain settlement amounts.

## 26.7 Reconciliation Exception Handling

Exceptions must not be silently corrected.

The system should preserve:

Original Observation  
Exception Classification  
Detected At  
Affected Transaction  
Affected Event  
Expected Amount  
Observed Amount  
Investigation Owner  
Resolution  
Resolution Timestamp

A correction should create a new record rather than rewriting the original observation.

This preserves forensic integrity.

## **27. INCIDENT RESPONSE & FAILURE CONTAINMENT**

### **27.1 Incident Classification**

Haven payment incidents should be classified according to impact.

#### **SEV-1 — Protocol Integrity Compromise**

Examples:

Unauthorized treasury routing  
Compromised signing authority  
Splitter identity substitution  
Confirmed payment bypass with network authorization

Required action:

Immediate protocol lock  
Cross-network traffic halt  
Security escalation  
Evidence preservation  
Root-cause investigation

#### **SEV-2 — Settlement Integrity Failure**

Examples:

Repeated settlement mismatches  
Missing blockchain settlements  
Unexpected allocation  
Synchronization divergence

Required action:

Automatic network lock where threshold is exceeded  
Reconciliation  
Settlement investigation  
Controlled recovery

#### **SEV-3 — Infrastructure Degradation**

Examples:

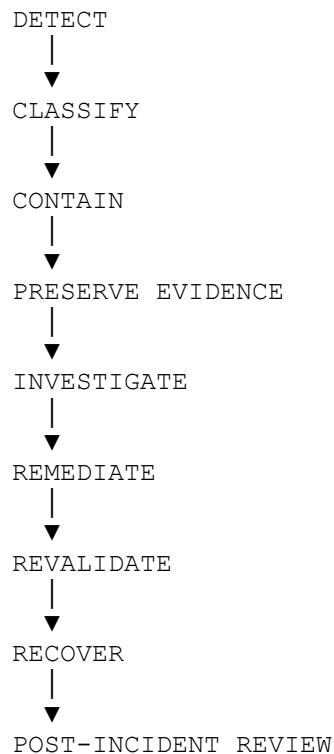
RPC provider outage  
Settlement observer delay  
Temporary queue saturation  
Non-critical monitoring failure

Required action:

Fail closed for affected authorization paths  
Fail over where configured  
Monitor recovery  
Preserve pending events

## 27.2 Incident Response Sequence

The standard response sequence is:



The incident system must prioritize containment over availability when protocol integrity is uncertain.

## 27.3 Automatic Containment

The following conditions should be capable of automatically opening the circuit breaker:

Contract identity mismatch  
Chain mismatch  
Invalid synchronization  
Stale synchronization beyond TTL  
Configuration digest mismatch  
Repeated settlement mismatch  
Unauthorized route detection  
Critical observer failure

The automatic lock prevents an operational team from needing to respond before the enforcement boundary becomes effective.

## 27.4 Evidence Preservation

During an incident, the system must preserve:

- Blockchain transaction hashes
- Relevant block numbers
- Contract address
- Decoded contract events
- RPC responses where legally and operationally appropriate
- Adapter security logs
- Configuration digests
- Deployment manifest
- Payment processor references
- Correlation identifiers
- Authorization decisions
- Circuit-breaker transitions

Clinical information should not be copied into incident evidence unless required by the incident and permitted by the applicable data-governance controls.

## 27.5 Incident Timeline

The incident system should produce a chronological timeline:

- T0 Normal operation
- T1 First anomaly
- T2 Detection
- T3 Circuit breaker opened
- T4 Settlement investigation
- T5 Root cause identified
- T6 Remediation
- T7 Contract revalidation
- T8 Synchronization restored
- T9 Controlled recovery
- T10 Production authorization resumed

Every transition should reference the applicable correlation and evidence identifiers.

## 27.6 Recovery Validation

Recovery requires more than restoring application availability.

Before reopening the network:

- [ ] Chain identity verified
- [ ] Splitter identity verified
- [ ] Treasury verified
- [ ] Haven vault verified
- [ ] Allocation verified
- [ ] Configuration verified
- [ ] Settlement path verified
- [ ] Synchronization verified
- [ ] Adapter integrity verified
- [ ] Pending events reconciled
- [ ] Replay protection verified
- [ ] Audit pipeline verified

Only after these conditions pass may the network transition from `LOCKED` to `RECOVERING`.

## 27.7 Post-Incident Review

Every material protocol incident should produce a post-incident record containing:

- Incident ID
- Severity
- Start Time
- Detection Time
- Containment Time
- Recovery Time
- Affected Network
- Affected Transactions
- Root Cause
- Security Impact
- Financial Impact
- Data Impact
- Corrective Actions
- Preventive Actions
- Owner
- Closure Approval

The objective is not merely to document what failed.

The review must determine whether the architecture allowed a violation of the core invariant:

**A participant must not receive Haven network authorization when the required payment routing condition has not been independently satisfied.**

## 28. PRODUCTION ACCEPTANCE TESTING & CONFORMANCE

### 28.1 Conformance Objective

The Haven Ledger Tokenomics and Point-of-Sale integration is production conformant only when the complete economic enforcement chain can be demonstrated from consumer checkout through independent protocol authorization.

Conformance is therefore evaluated across five domains:

1. Economic correctness
2. Smart contract correctness
3. Settlement correctness
4. Edge enforcement correctness
5. Failure and recovery correctness

Passing application-level tests alone does not establish Haven conformance.

### 28.2 Mandatory Acceptance Test Matrix

| Test ID   | Test                      | Expected Result              |
|-----------|---------------------------|------------------------------|
| POS-001   | Valid protocol event      | Correct Haven fee calculated |
| POS-002   | Fee removed from request  | Transaction rejected         |
| POS-003   | Fee manually reduced      | Transaction rejected         |
| POS-004   | Duplicate idempotency key | No duplicate obligation      |
| POS-005   | Valid payment             | Payment progresses normally  |
| POS-006   | Payment failure           | No protocol authorization    |
| CHAIN-001 | Correct Polygon chain     | Verification succeeds        |
| CHAIN-002 | Incorrect chain           | Circuit breaker opens        |
| CHAIN-003 | Expected splitter         | Contract accepted            |
| CHAIN-004 | Unknown contract          | Contract rejected            |
| SET-001   | Correct settlement        | Settlement matched           |
| SET-002   | Missing settlement        | Authorization denied         |
| SET-003   | Incorrect amount          | Settlement mismatch          |
| SET-004   | Incorrect destination     | Settlement mismatch          |
| SET-005   | Duplicate settlement      | Duplicate rejected           |
| SYNC-001  | Synchronization true      | Authorization eligible       |
| SYNC-002  | Synchronization false     | Network locked               |

| Test ID  | Test                   | Expected Result               |
|----------|------------------------|-------------------------------|
| SYNC-003 | Stale synchronization  | Network locked                |
| RPC-001  | RPC unavailable        | Fail closed                   |
| SEC-001  | Configuration modified | Tamper detected               |
| SEC-002  | Replay event           | Replay rejected               |
| SEC-003  | Unauthorized route     | Cross-network traffic blocked |
| REC-001  | Valid recovery         | Network reauthorized          |
| REC-002  | Failed recovery test   | Network remains locked        |

All acceptance results must be retained as deployment evidence.

## 28.3 Smart Contract Conformance

The deployed Splitter must satisfy the following invariants:

INVARIANT-01

Client treasury address cannot be modified.

INVARIANT-02

Haven vault address cannot be modified.

INVARIANT-03

Allocation basis points cannot be modified.

INVARIANT-04

Unauthorized callers cannot modify synchronization state.

INVARIANT-05

Incoming routed value cannot silently disappear.

INVARIANT-06

Failed allocation causes the routing transaction to revert.

INVARIANT-07

The contract accepts only the intended production configuration.

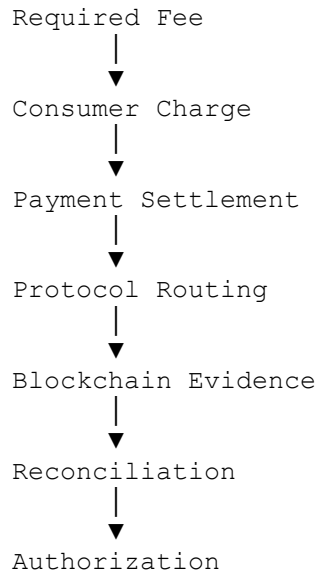
INVARIANT-08

Contract behavior corresponds to the approved production artifact.

These invariants should be validated through automated tests and, for a production deployment, independent security review.

## 28.4 Payment Conformance

For every Haven-governed event:



A missing stage must prevent the authorization stage from being reached.

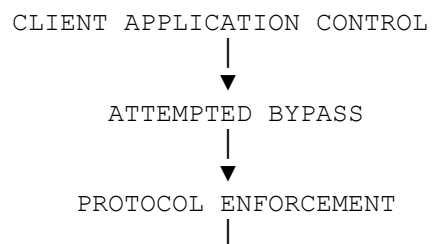
## 28.5 Security Acceptance Criterion

The most important acceptance test is an adversarial one.

A tester with legitimate administrative access to the client POS must attempt to:

- Remove the Haven fee
- Change the fee amount
- Change the splitter address
- Change the treasury
- Change the Haven vault
- Force synchronization true
- Disable the circuit breaker
- Submit an unauthorized settlement
- Replay a valid event

The expected outcome is:



▼  
DENIED

If any test permits unauthorized Haven network participation, the deployment must not be considered conformant.

## 28.6 Operational Acceptance

Production operations must additionally demonstrate:

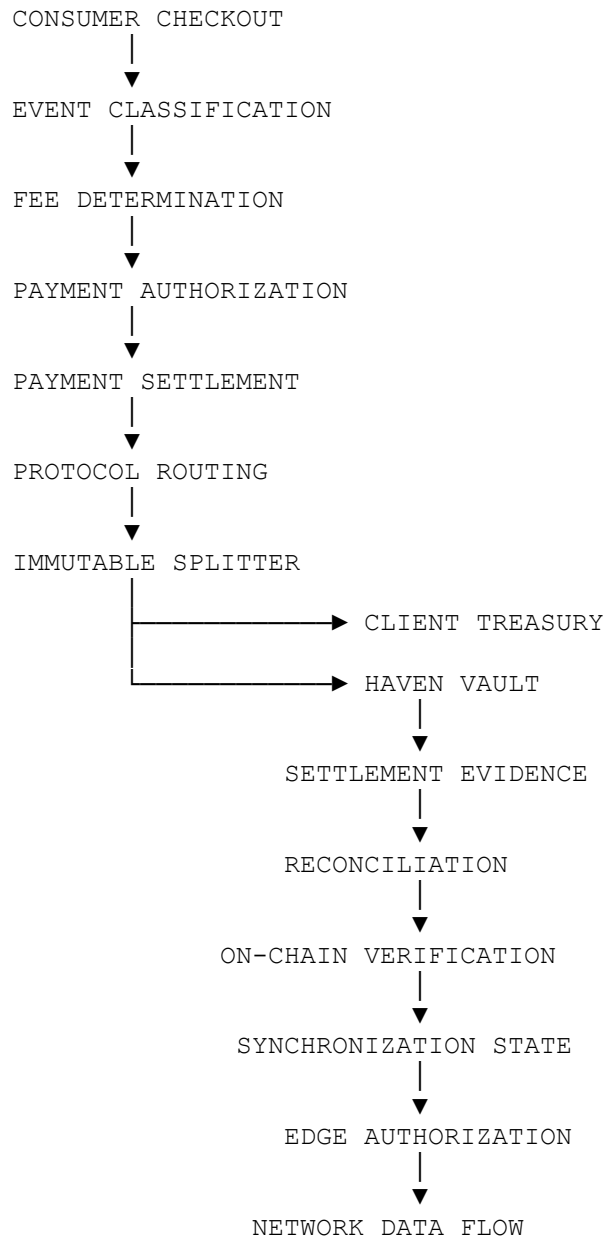
RPC failover  
Queue recovery  
Database recovery  
Settlement observer recovery  
Circuit-breaker recovery  
Configuration integrity validation  
Audit reconstruction  
Financial reconciliation  
Incident escalation

The acceptance process must test both normal operation and degraded conditions.

## 29. PRODUCTION OPERATING STANDARD & FINAL ARCHITECTURAL INVARIANTS

### 29.1 Production Operating Standard

The Haven Point-of-Sale integration is governed by the following operating sequence:



The payment mechanism is therefore not an isolated billing component.

It is part of the protocol's participation-control architecture.

## 29.2 Final Architectural Invariants

The following invariants constitute the production baseline.

### **INVARIANT 1 — Mandatory Economic Obligation**

Where a Haven-governed event requires a protocol fee, the obligation cannot be removed by client application logic.

### **INVARIANT 2 — Immutable Routing**

The production Splitter defines the authoritative routing destinations and allocation parameters.

Client application administrators cannot redefine them.

### **INVARIANT 3 — Independent Verification**

Client-reported payment or synchronization status is never sufficient evidence of protocol participation.

### **INVARIANT 4 — Settlement Before Authorization**

Successful consumer payment does not itself authorize Haven network participation.

Required protocol settlement must be independently verified.

### **INVARIANT 5 — Fail Closed**

UNKNOWN  
=  
UNAUTHORIZED

When protocol integrity cannot be established, Haven-governed cross-network operations stop.

### **INVARIANT 6 — No Split, No Data Traffic**

A network that cannot demonstrate compliance with the required payment-routing condition cannot continue Haven-governed cross-network coordination.

### **INVARIANT 7 — Client Control Is Not Protocol Control**

The enterprise retains control of its local healthcare applications.

It does not receive unilateral authority over Haven's protocol enforcement boundary.

## **INVARIANT 8 — No Clinical Data on Public Settlement Layer**

Blockchain settlement should carry only the minimum information necessary to establish economic and protocol integrity.

Clinical information remains outside the public settlement layer.

## **INVARIANT 9 — Historical Integrity**

Financial, authorization, security, and settlement observations must remain auditable.

Corrections create new records rather than rewriting historical evidence.

## **INVARIANT 10 — Recovery Requires Revalidation**

A locked network cannot be restored merely by changing a local status flag.

Recovery requires independent verification of protocol integrity.

## **29.3 Final Protocol Equation**

The complete Haven participation condition can be expressed as:

```
HAVEN_PARTICIPATION
=
VALID_EVENT
AND
VALID_FEE
AND
PAYMENT_SETTLED
AND
PROTOCOL_SETTLEMENT_MATCHED
AND
CORRECT_CONTRACT
AND
CORRECT_ROUTING
AND
SYNCHRONIZATION_FRESH
AND
NETWORK_SYNCHRONIZED
AND
CONFIGURATION_INTEGRITY_VALID
AND
CIRCUIT_BREAKER_CLOSED
```

If any mandatory condition evaluates to false or unknown:

```
HAVEN_PARTICIPATION = DENIED
```

## 29.4 Final Economic Principle

The Haven Protocol fee is an infrastructure pass-through obligation attached to the system event that requires Haven coordination.

The architecture does not depend on voluntary payment behavior after network participation has already occurred.

Instead:

ECONOMIC OBLIGATION  
↓  
SETTLEMENT  
↓  
VERIFICATION  
↓  
NETWORK PARTICIPATION

This ordering is fundamental.

The network does not first provide unrestricted coordination and attempt to recover the protocol fee afterward.

The economic condition is bound to authorization.

## 29.5 Final Security Principle

The Zero-Trust Payment Routing Enclosure Engine exists to make payment routing an enforceable protocol property rather than an application convention.

Its governing rule is:

**No valid routing evidence, no valid synchronization. No valid synchronization, no Haven-governed network participation.**

The architecture therefore converts payment compliance from a configurable software preference into a verifiable infrastructure condition.

# **Document Completion Statement**

## **THE HAVEN PROTOCOL**

### **Ledger Tokenomics & Point-of-Sale (PoS) Integration Guide**

**Document Number:** HAVEN-DOC-05

**Version:** 2.0

**Status:** Production Architecture Baseline

**Network:** Polygon EVM

**Primary Enforcement Layer:** On-Chain Payment Routing

**Economic Primitive:** Protocol Event Fee

**Primary Enforcement Model:** Zero-Trust Payment Routing Enclosure Engine

### **Governing Principle**

**NO PAYMENT ROUTE.**

**NO NETWORK PARTICIPATION.**

This document establishes the production baseline for integrating Haven economic obligations into participating enterprise point-of-sale environments while preserving separation between consumer payment processing, blockchain settlement, treasury custody, synchronization verification, and clinical data coordination.

The implementation team must treat the on-chain routing contract, independently verified settlement evidence, and fail-closed edge enforcement mechanism as the critical security boundary.

**END OF DOCUMENT 5**